

LAB 12 ASSIGNMENT

Infrastructure Automation using Terraform **Nginx Load Balancer and Apache Web Servers**

Name: Emaan Hanif

Roll Number: 17

Course:

Cloud Computing

Submitted To: Sir Waqas

Submission Date:

30 December 2025

Table of Contents

1. Executive Summary	3
o Brief overview of the assignment	
o Infrastructure deployed	
o Key achievements	
2. Architecture Design	4
o Architecture diagram	
o Component descriptions	
o Network topology	
o Security design	
3. Implementation Details	8
o Part 1: Infrastructure setup (with screenshots)	
o Part 2: Webserver module (with screenshots)	
o Part 3: Server scripts (with screenshots and code snippets)	
o Part 4: Deployment (with screenshots)	
o Part 5: Testing and verification (with screenshots)	
o Part 6: Cleanup (with screenshots)	
4. Testing Results	116
o Load balancing tests	
o Cache performance tests	
o High availability tests	
o Security tests	
o Performance metrics	
5. Challenges & Solutions	123
o Problems encountered	
o How you solved them	
o Lessons learned	
6. Conclusion	126
o Summary of work completed	
o Skills acquired	
o Future improvements	
7. Appendices	127
o Complete code listings	
o Configuration files	
o Additional screenshots	
o References	

1. Executive Summary

Brief Overview of the Assignment

This assignment involved deploying a production-ready, multi-tier web application infrastructure on AWS using Terraform modules and Nginx as a reverse proxy/load balancer. The infrastructure includes one Nginx server, three backend web servers (two primary, one backup), proper security configurations, caching, SSL/TLS, and high availability. Key tasks included project setup, variable configuration, module creation, server provisioning, and script automation for Apache and Nginx.

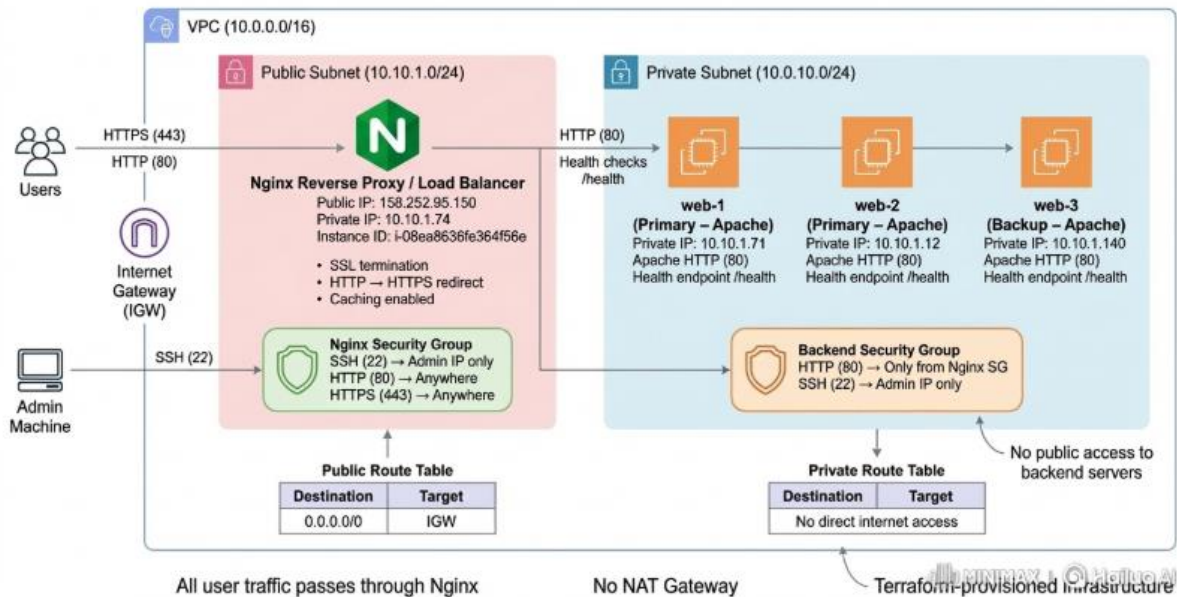
Infrastructure Deployed

- **Networking:** VPC, public subnet for Nginx, private subnets for backend servers, Internet Gateway, route tables.
- **Webservers:** One Nginx load balancer and three backend EC2 instances. Configuration and setup were implemented using custom Bash scripts (nginx-setup.sh and apache-setup.sh).
- **Security:** Nginx and backend security groups with restricted ingress/egress rules; backend accessible only via Nginx.
- **Server Configuration:** Apache installed on backend servers with dynamic metadata pages; Nginx configured with SSL, caching, reverse proxy, load balancing, and health checks.
- **High Availability:** Backup backend server ensures service continuity if primary servers fail.

Key Achievements

1. **Project Setup & Variables:** Organized Terraform structure, defined validated variables, and automated configuration via terraform.tfvars.
2. **Networking Module:** Created reusable module for VPC, subnets, IGW, and route tables with tagging and outputs.
3. **Security Module:** Configured Nginx and backend security groups with precise access rules.
4. **Webserver Module & Deployment:** Deployed Nginx and backend servers using a reusable EC2 module with key pairs and user-data scripts.
5. **Server Scripts:** Automated Apache and Nginx setup including metadata pages, SSL, caching, load balancing, and security headers.
6. **Infrastructure Deployment:** Successfully initialized, validated, applied Terraform configuration, and verified resources in AWS Console.
7. **Testing & Validation:** Ensured load balancing, caching, high availability, and security headers worked as intended.
8. **Bonus Enhancements:** Implemented custom error pages, rate limiting, and automated backend health checks.

Architecture diagram



Component descriptions

- **VPC** → 10.0.0.0/16

Provides an isolated network for all resources.

All subnets, EC2 instances, route tables, and gateways exist inside the VPC.

- **Internet Gateway (IGW)** → Connects public subnet to internet

Attached to the VPC.

Enables inbound and outbound internet connectivity.

Allows client traffic to reach the Nginx server.

- **Public Subnet** → 10.10.1.0/24, hosts Nginx EC2

Hosts internet-facing resources.

Associated with a route table that allows internet access via IGW.

- **Private Subnet** – >10.0.10.0/24, hosts backend web servers (web-1, web-2, web-3 backup)

Not directly accessible from the internet.

- **Nginx EC2 Instance**

Acts as a reverse proxy and load balancer.

Receives HTTP and HTTPS requests from users.

Forwards requests to backend servers.

- **Backend EC2 Instances**

Run Apache web server.

web-1 and web-2 act as primary servers.

web-3 acts as a backup server.

Only accessible through the Nginx server.

- **Security Groups**

Act as virtual firewalls.

Allow SSH access only from authorized IPs.

Allow HTTP/HTTPS access as per defined rules.

- Nginx SG: Allow 80/443 from Internet, 22 from admin IP
- Backend SG: Allow 80 only from Nginx SG

- **Availability Zone (AZ) Distribution**

All resources are deployed in a single Availability Zone.

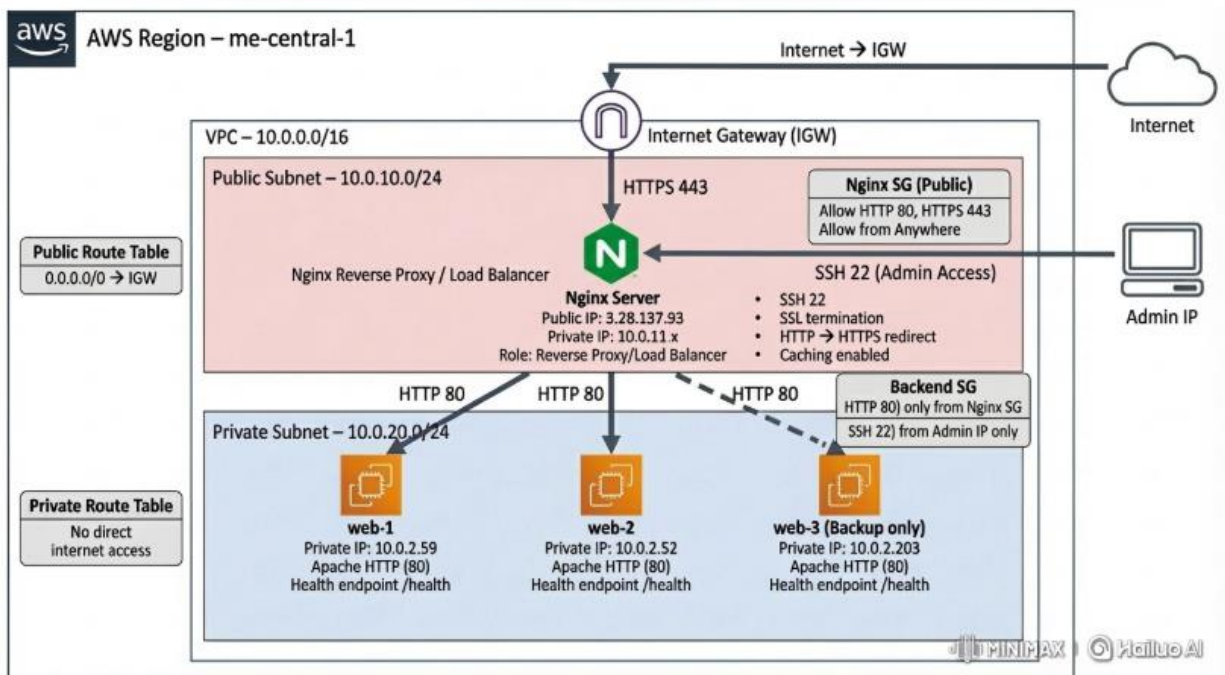
- **Route Tables**

Control traffic routing within the VPC.

Public route table routes traffic to the Internet Gateway.

Private route table allows internal communication only.

Network topology



2.4 Network Topology

client Request Flow:

- All external HTTP/HTTPS requests from users enter via the **Internet**.
- Requests first hit the **Nginx server** in the **public subnet**.

Nginx Role:

- Acts as **reverse proxy** and **load balancer**.
- Provides **SSL termination**, caching, and security headers.

Backend Server Access:

- **Private subnet servers** (web-1, web-2) handle normal requests.
- **Backup server** (web-3) only receives traffic if a primary server fails.
- Backend servers **do not communicate directly with the Internet**—only via Nginx.

Security Groups enforce traffic rules:

- Nginx SG: HTTP/HTTPS from anywhere, SSH from admin IP

- Backend SG: HTTP only from Nginx SG
- If one backend fails, Nginx redirects traffic to other healthy servers.

Security Design

Security in this infrastructure is implemented using **AWS Security Groups**, which act as virtual firewalls to control inbound and outbound traffic at the instance level.

Nginx Security Group (Public-Facing)

- Allows **HTTP (80)** and **HTTPS (443)** traffic from the internet to serve client requests.
- Allows **SSH (22)** access only from the administrator's IP for secure management.
- Acts as the **only public entry point**, reducing the attack surface.

Backend Security Group (Private Servers)

- Allows **HTTP (80)** traffic **only from the Nginx Security Group**.
- Blocks direct internet access to backend servers.
- Ensures backend servers are reachable **only through the reverse proxy**.

Traffic Isolation & Control

- All external traffic is terminated at Nginx before reaching backend servers.
- Backend servers are isolated from public exposure, improving security.
- Security group rules enforce **least-privilege access**.

2.6 Load Balancing Strategy

- Nginx distributes traffic using **round-robin** load balancing
- Two backend servers handle active traffic
- One backend server is configured as a **backup**
- If a primary backend fails, traffic is automatically redirected to the backup server

This strategy improves reliability and fault tolerance.

Part 1: Infrastructure setup

Task1:

- ❖ create all necessary files and directories
- ❖ implement proper .gitignore to exclude sensitive files
- ❖ document the project structure in README.md

Create Project Root Directory

mkdir Lab12_Assignment

```
● @emanhanif1 →/workspaces/Lab12_Assignment (main) $ mkdir Assignment2
```

This command creates a new directory named **Lab12_Assignment** to serve as the main (root) folder for organizing all files and resources of the project.

cd Lab12_Assignment

```
● @emanhanif1 →/workspaces/Lab12_Assignment (main) $ cd Assignment2
○ @emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ █
```

This command changes the current working directory to **Lab12_Assignment** so you can start working inside the project folder.

Create Root Terraform Files

touch main.tf variables.tf outputs.tf locals.tf terraform.tfvars README.md .gitignore

```
● @emanhanif1 →/workspaces/Lab12_Assignment (main) $ cd Assignment2
● @emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ touch main.tf variables.tf outputs.tf locals.tf terraform.tfvars README.md .gitignore
```

This command creates all required root Terraform configuration files, documentation, and Git ignore file to initialize the project structure before adding modules and scripts.

Core Terraform files were created at the root level:

- ❖ main.tf → Defines primary infrastructure resources
- ❖ variables.tf → Stores input variables
- ❖ outputs.tf → Displays important output values
- ❖ locals.tf → Defines reusable local values
- ❖ terraform.tfvars → Holds variable values
- ❖ README.md → documents the project
- ❖ .gitignore → prevents sensitive or unnecessary files from being tracked

Create Modules Directory Structure

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ touch modules/networking/main.tf modules/networking/variables.tf modules/networking/outputs.tf
```

- Creates a **modules** directory to store reusable Terraform components
- Separates infrastructure into **networking**, **security**, and **webserver** modules
- Now create files inside each module:

Networking module

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ touch modules/networking/main.tf modules/networking/variables.tf modules/networking/outputs.tf
```

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ touch modules/networking/main.tf modules/networking/variables.tf modules/networking/outputs.tf
```

This command creates module with main.tf, variables.tf, and outputs.tf files

Security module

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ mkdir -p modules/networking modules/security modules/webserver
```

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ touch modules/security/main.tf modules/security/variables.tf modules/security/outputs.tf
```

This command creates module with main.tf, variables.tf, and outputs.tf files

Webserver module

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ mkdir -p modules/networking modules/security modules/webserver
```

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ touch modules/webserver/main.tf modules/webserver/variables.tf modules/webserver/outputs.tf
```

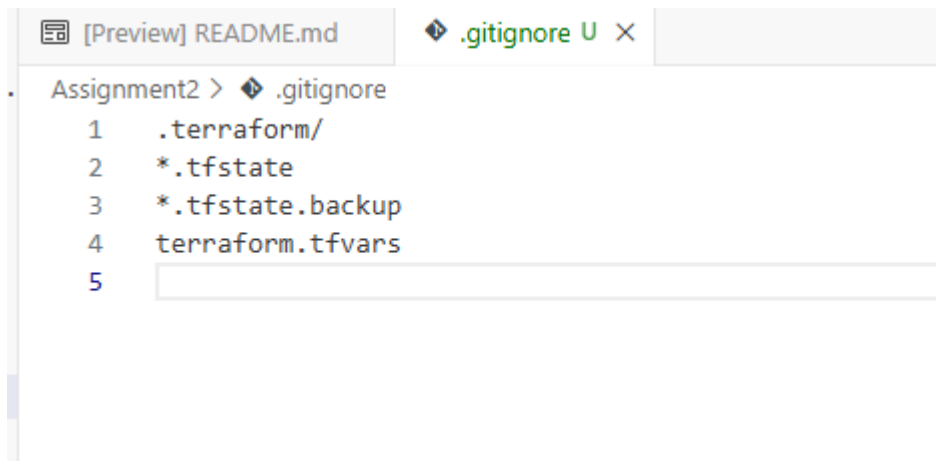
This command creates security module with main.tf, variables.tf, and outputs.tf files

Create Scripts Directory

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ mkdir scripts
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ touch scripts/nginx-setup.sh scripts/apache-setup.sh
```

- ❖ Creates a **scripts** directory to store automation shell scripts
- ❖ nginx-setup.sh configures the **Nginx load balancer**
- ❖ apache-setup.sh installs and configures **Apache** on backend servers
- ❖ Automates server setup to ensure **consistent configuration**

Implement proper .gitignore to exclude sensitive file



? Adds a .gitignore file to prevent sensitive files from being tracked

- ❖ Document Project Structure in README.md

Assignment2 >  README.md > abc # Assignment2 — Terraform Project (Part 1) > abc ## Project Structure

```
1  # Assignment2 - Terraform Project (Part 1)
2
3  ## Purpose
4  This README documents the project structure and basic information for the
5  Terraform setup.
6  It shows a clean and modular organization of files and folders for Assignment2.
7  ---
8
9  ## Project Overview
10 This project contains Terraform configurations organized in a dedicated
11 directory (`Assignment2`) to prevent committing sensitive files and large state
12 files.
13 Modules and scripts are separated for clarity and reusability.
14 ---
15
16 ## Project Structure
17
18 Assignment2/
19 |— main.tf
20 |— variables.tf
21 |— outputs.tf
22 |— locals.tf
23 |— terraform.tfvars
24 |— .gitignore
25 |— modules/
26 |   |— networking/
```



```

├── main.tf
├── variables.tf
├── outputs.tf
├── security/
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
├── webserver/
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
├── scripts/
│   ├── nginx-setup.sh
│   └── apache-setup.sh
└── README.md

```

```

# Main Project Structure
#
# ├── main.tf
# ├── variables.tf
# ├── outputs.tf
# ├── security/
# │   ├── main.tf
# │   ├── variables.tf
# │   └── outputs.tf
# ├── webserver/
# │   ├── main.tf
# │   ├── variables.tf
# │   └── outputs.tf
# ├── scripts/
# │   ├── nginx-setup.sh
# │   └── apache-setup.sh
# └── README.md

```

Modules & Scripts Explanation

- **Modules:**

- ``networking``: Contains VPC, subnets, and routing logic.
- ``security``: Contains security groups and firewall rules.
- ``webserver``: Contains web server setup (Apache/Nginx EC2 configuration).

- ``nginx-setup.sh``: Installs and configures Nginx for load balancing.
- ``apache-setup.sh``: Installs and configures Apache backend servers.

.gitignore Note

Sensitive files and Terraform state files are excluded:

```

.terraform/
*.tfstate
*.tfstate.backup
terraform.tfvars
*.log

```

> This ensures the repository stays clean and no sensitive information is committed.

The README file documents:

- Project overview
- Directory structure
- Purpose of each module

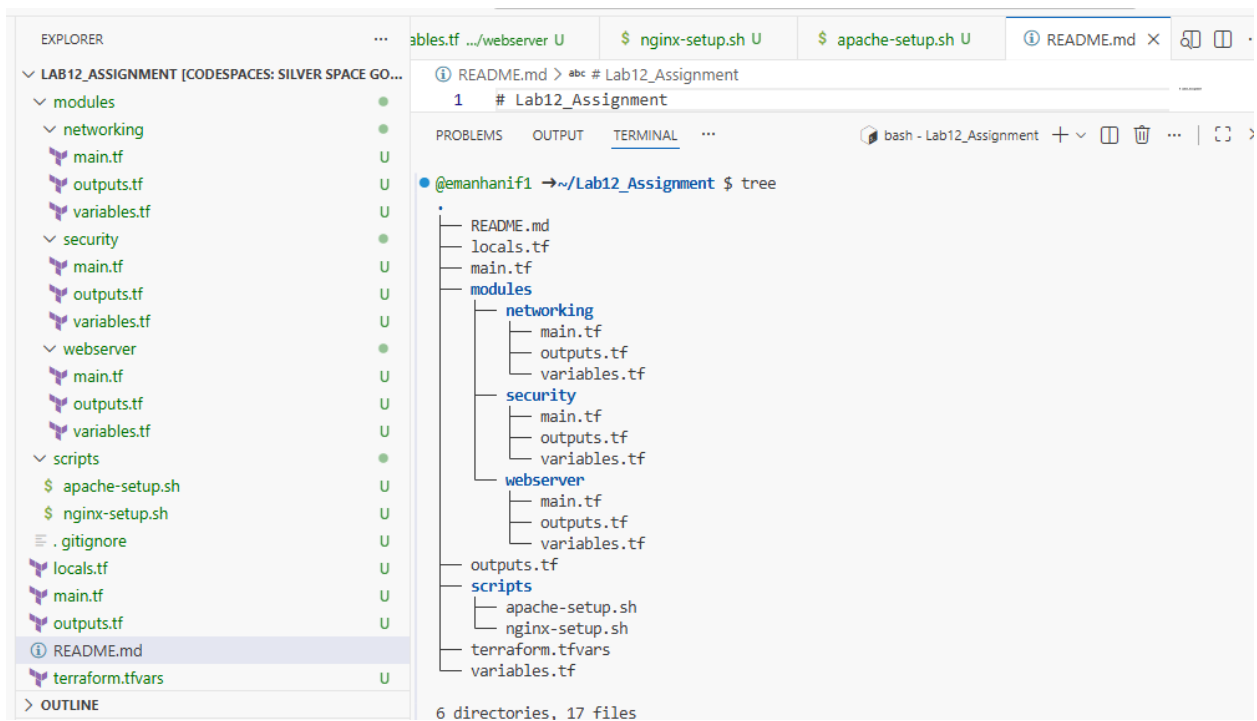
Helps others to understand the project quickly.

Verify Project Structure

Create all necessary files and directories

The tree command also verifies that:

- All required directories and files were created correctly
- Project structure matches Terraform best practices



```
EXPLORER
LAB 12_ASSIGNMENT [CODESPACES: SILVER SPACE GO...]
├── modules
│   ├── networking
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   ├── security
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   └── webserver
│       ├── main.tf
│       ├── outputs.tf
│       └── variables.tf
├── scripts
│   ├── apache-setup.sh
│   ├── nginx-setup.sh
│   ├── .gitignore
│   ├── locals.tf
│   ├── main.tf
│   ├── outputs.tf
│   ├── README.md
│   └── terraform.tfvars
└── OUTLINE

TERMINAL
@emmanhanif1 → ~/Lab12_Assignment $ tree
.
├── README.md
├── locals.tf
├── main.tf
├── modules
│   ├── networking
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   ├── security
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   └── webserver
│       ├── main.tf
│       ├── outputs.tf
│       └── variables.tf
├── outputs.tf
└── scripts
    ├── apache-setup.sh
    ├── nginx-setup.sh
    ├── terraform.tfvars
    └── variables.tf

6 directories, 17 files
```

1.2 Variable Configuration

Define all required variables in variables.tf:

Required Variables:

- vpc_cidr_block (string)
- subnet_cidr_block (string)
- availability_zone (string)
- env_prefix (string)
- instance_type (string)
- public_key (string)
- private_key (string)
- backend_servers (list of objects with name and script_path)

Tasks:

- ☐ Add validation rules for CIDR blocks
- ☐ Add descriptions for all variables
- ☐ Set appropriate defaults where applicable
- ☐ Populate terraform.tfvars with your values

Variables Defined In Variables.tf:

- ❖ Declares input variables used across the Terraform project
- ❖ Defines values for VPC, subnets, and CIDR blocks
- ❖ Specifies instance types and AMI IDs
- ❖ Stores key pair name and region
- ❖ Makes the configuration **flexible and reusable**

```

variables.tf
1  variable "vpc_cidr_block" {
2      description = "CIDR block for the VPC"
3      type        = string
4      default     = "10.1.0.0/16"
5      validation {
6          condition = can(cidrnetmask(var.vpc_cidr_block))
7          error_message = "Invalid VPC CIDR block format."
8      }
9  }
10
11 variable "subnet_cidr_block" {
12     description = "CIDR block for the subnet"
13     type        = string
14     default     = "10.1.10.0/24"
15     validation {
16         condition = can(cidrnetmask(var.subnet_cidr_block))
17         error_message = "Invalid subnet CIDR block format."
18     }
19 }
20
21 variable "availability_zone" {
22     description = "AWS availability zone for instances"
23     type        = string
24     default     = "us-east-1a"
25 }
26
27 variable "env_prefix" {
28     description = "Environment prefix (prod/dev)"
29     type        = string
30     default     = "lab12"
31 }
32
33 variable "instance_type" {
34     description = "EC2 instance type"
35     type        = string
36     default     = "t3.micro"

```

SSH public key path

```

variable "public_key" {
    description = "Path to the SSH public key"
    type        = string
    default     = "~/.ssh/id_ed25519.pub"
}

```

SSH private key path

```

variable "private_key" {
    description = "Path to the SSH private key"
    type        = string
    default     = "~/.ssh/id_ed25519"
}

```



```
# Backend servers list of objects (3 servers)
variable "backend_servers" {
  description = "List of backend servers with their name and setup script path"
  type = list(object({
    name      = string
    script_path = string
  }))
  default = [
    {
      name      = "web-1"
      script_path = "scripts/apache-setup.sh"
    },
    {
      name      = "web-2"
      script_path = "scripts/apache-setup.sh"
    },
    {
      name      = "web-3"
      script_path = "scripts/apache-setup.sh"
    }
  ]
}
```

- **vpc_cidr_block**
Defines the CIDR range for the Virtual Private Cloud (VPC).
- **subnet_cidr_block**
Specifies the CIDR range for the subnet inside the VPC.
- **availability_zone**
Determines the AWS Availability Zone where resources are deployed.
- **env_prefix**
Used as a naming prefix to identify the environment (e.g., dev or prod).
- **instance_type**
Specifies the EC2 instance type used for web servers.
- **public_key & private_key**
Used for secure SSH access to EC2 instances.
- **backend_servers**
A list of backend servers, each containing a server name and its associated setup script path.

❖ **Set appropriate defaults where applicable**

- **Set default values** for variables like instance_type and env_prefix to simplify deployment
- **Reduces manual input** when applying Terraform
- **Enhances reusability** of scripts across different environments
- Ensures **consistent configuration** without always specifying every variable

```

26
27 variable "env_prefix" {
28     description = "Environment prefix (prod/dev)"
29     type        = string
30     default     = "lab12"
31 }
32
33 variable "instance_type" {
34     description = "EC2 instance type"
35     type        = string
36     default     = "t3.micro"

```

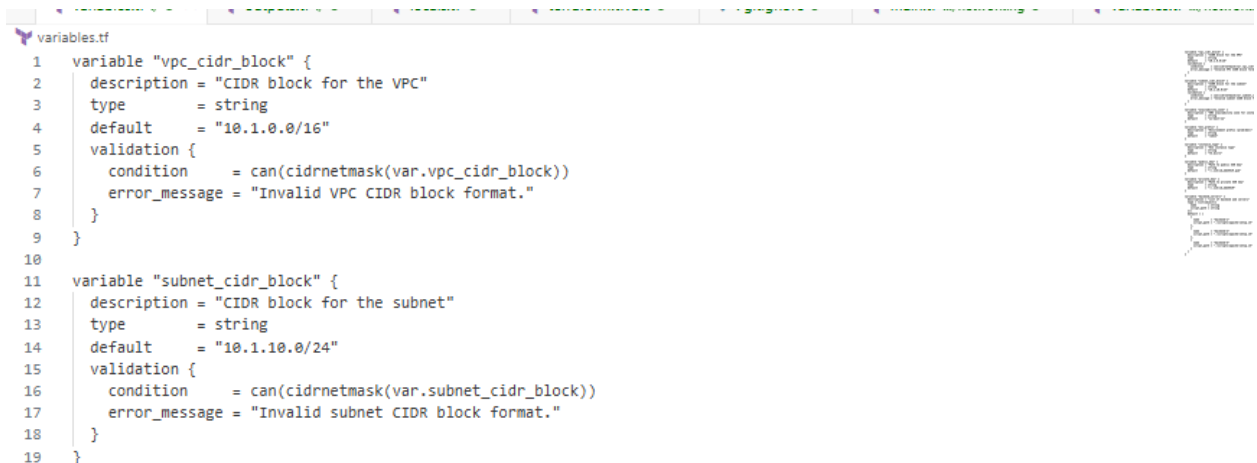
❖ Added validation rules

Added validation rules for `vpc_cidr_block` and `subnet_cidr_block` to allow only valid CIDR formats

🔗 Provided default values for all variables to simplify deployment

🔗 Prevents configuration errors during Terraform execution

🔗 Ensures consistent and error-free setup across different environments



```

variables.tf
1  variable "vpc_cidr_block" {
2      description = "CIDR block for the VPC"
3      type        = string
4      default     = "10.1.0.0/16"
5      validation {
6          condition = can(cidrnetmask(var.vpc_cidr_block))
7          error_message = "Invalid VPC CIDR block format."
8      }
9  }
10
11 variable "subnet_cidr_block" {
12     description = "CIDR block for the subnet"
13     type        = string
14     default     = "10.1.10.0/24"
15     validation {
16         condition = can(cidrnetmask(var.subnet_cidr_block))
17         error_message = "Invalid subnet CIDR block format."
18     }
19 }

```

❖ Added descriptions for all variables to explain their purpose

🔗 Improves **readability** and **maintainability** of the Terraform code

🔗 Helps users understand what each variable controls before assigning values

```

variables.tf
1  variable "vpc_cidr_block" {
2    description = "CIDR block for the VPC"
3    type        = string
4    default     = "10.1.0.0/16"
5    validation {
6      condition = can(cidrnetmask(var.vpc_cidr_block))
7      error_message = "Invalid VPC CIDR block format."
8    }
9  }
10
11 variable "subnet_cidr_block" {
12   description = "CIDR block for the subnet"
13   type        = string
14   default     = "10.1.10.0/24"
15   validation {
16     condition = can(cidrnetmask(var.subnet_cidr_block))
17     error_message = "Invalid subnet CIDR block format."
18   }
19 }

```

❖ Populated terraform.tfvars with values

- Assigned lab-specific values to all variables.

[Preview] README.md X .gitignore U README.md U variables.tf U terrafoi

Assignment2 > terraform.tfvars

```

1  vpc_cidr_block      = "10.0.0.0/16"
2  subnet_cidr_block  = "10.0.10.0/24"
3  availability_zone    = "me-central-1a"
4  env_prefix          = "prod"
5  instance_type       = "t3.micro"
6  public_key          = "~/ssh/id_ed25519.pub"
7  private_key         = "~/ssh/id_ed25519"
8
9  backend_servers = [
10   { name = "web-1", script_path = "scripts/apache-setup.sh" },
11   { name = "web-2", script_path = "scripts/apache-setup.sh" },
12   { name = "web-3", script_path = "scripts/apache-setup.sh" }
13 ]
14

```

Terraform Variable Values

- Network CIDR ranges:** Ensure proper subnetting and avoid IP conflicts
- AWS availability zone:** Control resource placement for high availability
- EC2 instance type:** Set server capacity for performance and cost efficiency

- **SSH key paths:** Enable secure access to instances
- **Backend server definitions:** Specify which servers handle traffic and backups

1.3 Networking Module (5 marks)

Create a networking module that provisions:

- VPC with specified CIDR block
- Subnet with public IP assignment enabled
- Internet Gateway
- Route table with default route to IGW
- Associate route table with subnet

Module Location: modules/networking/

Required Outputs:

- vpc_id
- subnet_id
- igw_id
- route_table_id

Tasks:

- ☐ Create VPC resource
- ☐ Create subnet with map_public_ip_on_launch = true
- ☐ Create and attach Internet Gateway
- ☐ Configure routing table
- ☐ Add proper tags to all resources using env_prefix

Purpose of Networking Module

- To create basic AWS network infrastructure
- To provide internet access to EC2 instances
- To keep networking resources modular and reusable

dules > networking >  main.tf

```
1  # 1. Create VPC
2  resource "aws_vpc" "this" {
3      cidr_block = var.vpc_cidr_block
4      tags = {
5          Name = "${var.env_prefix}-vpc"
6      }
7  }
8
9  # 2. Create Subnet
10 resource "aws_subnet" "this" {
11     vpc_id            = aws_vpc.this.id
12     cidr_block        = var.subnet_cidr_block
13     availability_zone  = var.availability_zone
14     map_public_ip_on_launch = true
15
16     tags = {
17         Name = "${var.env_prefix}-subnet"
18     }
19 }
20
21 # 3. Create Internet Gateway
22 resource "aws_internet_gateway" "this" {
23     vpc_id = aws_vpc.this.id
24
25     tags = {
26         Name = "${var.env_prefix}-igw"
27     }
28 }
```

```
1  # 1. Create VPC
2  resource "aws_vpc" "this" {
3      cidr_block = var.vpc_cidr_block
4      tags = {
5          Name = "${var.env_prefix}-vpc"
6      }
7  }
8
9  # 2. Create Subnet
10 resource "aws_subnet" "this" {
11     vpc_id            = aws_vpc.this.id
12     cidr_block        = var.subnet_cidr_block
13     availability_zone  = var.availability_zone
14     map_public_ip_on_launch = true
15
16     tags = {
17         Name = "${var.env_prefix}-subnet"
18     }
19 }
20
21 # 3. Create Internet Gateway
22 resource "aws_internet_gateway" "this" {
23     vpc_id = aws_vpc.this.id
24
25     tags = {
26         Name = "${var.env_prefix}-igw"
27     }
28 }
```

```

resource "aws_internet_gateway" "this" {
}

# 4. Create Route Table
resource "aws_route_table" "this" {
  vpc_id = aws_vpc.this.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

# 5. Associate Route Table with Subnet
resource "aws_route_table_association" "this" {
  subnet_id      = aws_subnet.this.id
  route_table_id = aws_route_table.this.id
}

```

TASKS COMPLETED:

Create VPC (main.tf)

- Use aws_vpc
- Set CIDR block using variable
- Enable DNS support
- Add tags using env_prefix

dules > networking >  main.tf

```

1  # 1. Create VPC
2  resource "aws_vpc" "this" {
3    cidr_block = var.vpc_cidr_block
4    tags = {
5      Name = "${var.env_prefix}-vpc"
6    }
7  }

```

```

1  resource "aws_vpc" "this" {
2    cidr_block = var.vpc_cidr_block
3    enable_dns_hostnames = true
4    enable_dns_support = true
5    tags = {
6      Name = "${var.env_prefix}-vpc"
7    }
8  }

```

Create Public Subnet

- Use `aws_subnet`
- Attach it to VPC
- Enable:

`map_public_ip_on_launch = true`

```
3
3 # 2. Create Subnet
3 resource "aws_subnet" "this" {
1   vpc_id           = aws_vpc.this.id
2   cidr_block       = var.subnet_cidr_block
3   availability_zone = var.availability_zone
4   map_public_ip_on_launch = true
5
5   tags = {
7     Name = "${var.env_prefix}-subnet"
3   }
3 }
3
```

Create Internet Gateway

- Use `aws_internet_gateway`
- Attach to VPC
- Tag it properly

```
3
1 # 3. Create Internet Gateway
2 resource "aws_internet_gateway" "this" {
3   vpc_id = aws_vpc.this.id
4
5   tags = {
5     Name = "${var.env_prefix}-igw"
7   }
3 }
```

Create Route Table

- Use `aws_route_table`

- Add default route:

0.0.0.0/0 → internet gateway

```
# 4. Create Route Table
resource "aws_route_table" "this" {
  vpc_id = aws_vpc.this.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}
```

Associate Route Table with Subnet

- Use `aws_route_table_association`
- Connect route table to public subnet

```
# 5. Associate Route Table with Subnet
resource "aws_route_table_association" "this" {
  subnet_id      = aws_subnet.this.id
  route_table_id = aws_route_table.this.id
}
```

Define Variables in `variables.tf`


```

Assignment2 > modules > networking > variables.tf
1  variable "vpc_cidr_block" {
2    | description = "CIDR block for the VPC"
3    | type       = string
4  }
5
6  variable "subnet_cidr_block" {
7    | description = "CIDR block for the subnet"
8    | type       = string
9  }
10
11 variable "availability_zone" {
12   | description = "Availability zone for the subnet"
13   | type       = string
14 }
15
16 variable "env_prefix" {
17   | description = "Environment prefix for tagging"
18   | type       = string
19 }
20

```

Define Variables in Output.tf

view] README.md	.gitignore U	main.tf .../security U	variables.tf U	main.tf .../networking U
-----------------	--------------	------------------------	----------------	--------------------------

```

Assignment2 > modules > networking > outputs.tf
1  output "vpc_id" {
2    | value = aws_vpc.this.id
3  }
4
5  output "subnet_id" {
6    | value = aws_subnet.this.id
7  }
8
9  output "igw_id" {
10   | value = aws_internet_gateway.this.id
11 }
12
13 output "route_table_id" {
14   | value = aws_route_table.this.id
15 }
16

```

Call module

Assignment2 >  main.tf

```
1 provider "aws" {
2     shared_credentials_files = ["~/.aws/credentials"]
3     shared_config_files      = ["~/.aws/config"]
4 }
5 # Module call
6 module "networking" {
7     source                = "./modules/networking"
8     vpc_cidr_block        = var.vpc_cidr_block
9     subnet_cidr_block     = var.subnet_cidr_block
10    availability_zone      = var.availability_zone
11    env_prefix             = var.env_prefix
12 }
```

Initialize Terraform

terraform init

```
● @emanhanif1 → /workspaces/Lab12_Assignment/Assignment2 (main) $ terraform init
```

```
Initializing the backend...
Initializing modules...
```

```
Initializing provider plugins...
```

- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.27.0

```
Terraform has been successfully initialized!
```

```
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
```

```
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Apply deployment

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ terraform apply
```

```
+ tags_all = {  
+   "Name" = "prod-vpc"  
}  
}
```

Plan: 5 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

```
module.networking.aws_vpc.this: Creating...  
module.networking.aws_vpc.this: Creation complete after 2s [id=vpc-0ba1bcf63d7f30bf5]  
module.networking.aws_internet_gateway.this: Creating...  
module.networking.aws_internet_gateway.this: Creation complete after 0s [id=igw-080720630546527d3]  
module.networking.aws_route_table.this: Creating...  
module.networking.aws_route_table.this: Creation complete after 1s [id=rtb-06583758fa4e260ff]  
module.networking.aws_subnet.this: Still creating... [10s elapsed]  
module.networking.aws_subnet.this: Creation complete after 11s [id=subnet-09d2a34248d1250e6]  
module.networking.aws_route_table_association.this: Creating...  
module.networking.aws_route_table_association.this: Creation complete after 0s [id=rtbassoc-0ff7ae73099876eca]
```

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

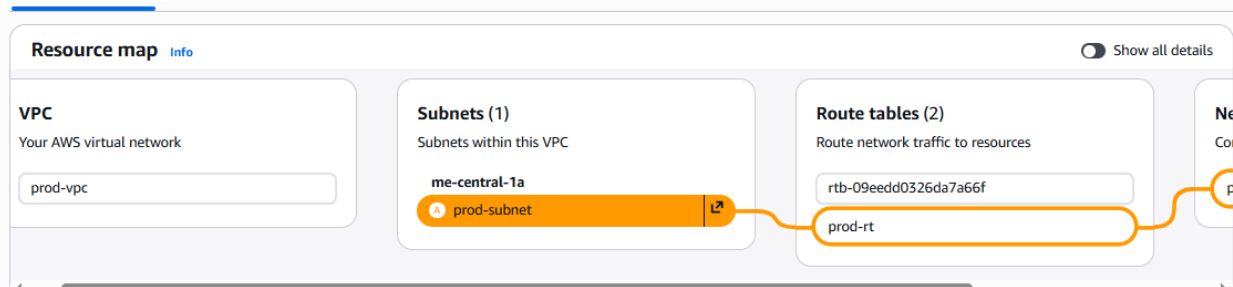
```
Assignment2 > outputs.tf
```

```
1  output "vpc_id" {  
2    | value = module.networking.vpc_id  
3  }  
4  
5  output "subnet_id" {  
6    | value = module.networking.subnet_id  
7  }  
8  
9  output "igw_id" {  
10   | value = module.networking.igw_id  
11 }  
12  
13 output "route_table_id" {  
14   | value = module.networking.route_table_id  
15 }  
16
```

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ terraform output  
igw_id = "igw-080720630546527d3"  
route_table_id = "rtb-06583758fa4e260ff"  
subnet_id = "subnet-09d2a34248d1250e6"  
vpc_id = "vpc-0ba1bcf63d7f30bf5"
```

Module Outputs

- vpc_id – VPC ID
- subnet_id – Subnet ID
- igw_id – Internet Gateway ID
- route_table_id – Route Table ID



1.4 Security Module (5 marks)

Create a security module that provisions:

Two Security Groups:

1. **Nginx Security Group** (for reverse proxy/load balancer):

- ❖ Ingress: Port 22 (SSH) from your IP only
- ❖ Ingress: Port 80 (HTTP) from anywhere (0.0.0.0/0)
- ❖ Ingress: Port 443 (HTTPS) from anywhere (0.0.0.0/0)
- ❖ Egress: All traffic

2. **Backend Security Group** (for web servers):

- ❖ Ingress: Port 22 (SSH) from your IP only
- ❖ Ingress: Port 80 (HTTP) from Nginx security group only
- ❖ Egress: All traffic

Module Location: modules/security/

Required Variables:

- vpc_id

- env_prefix
- my_ip

Required Outputs:

- nginx_sg_id
- backend_sg_id

Create variables in variables.tf

Purpose: Define all inputs required to create security groups dynamically and make the module reusable.

Variables Defined:

- vpc_id – Specifies the VPC where the security groups will be created.
- env_prefix – Prefix used in resource names for identification (e.g., prod-nginx-sg).
- my_ip – The user's public IP, used to restrict SSH (port 22) access to only the admin.

```
signment2 > modules > security > variables.tf
1  variable "vpc_id" {
2      description = "VPC ID where security groups will be created"
3      type        = string
4  }
5
6  variable "env_prefix" {
7      description = "Environment prefix for resource naming"
8      type        = string
9  }
10
11 variable "my_ip" {
12     description = "Your public IP in CIDR format (e.g. x.x.x.x/32)"
13     type        = string
14 }
15
```

Nginx Security Group (main.tf)

Explanation

Resource: aws_security_group for the Nginx server (public-facing).

Ingress Rules:

- SSH (port 22) → allowed only from the user's IP (my_ip).
- HTTP (port 80) → allowed from anywhere (0.0.0.0/0).
- HTTPS (port 443) → allowed from anywhere (0.0.0.0/0).

❖ **Egress Rules:**

- All outbound traffic is allowed.

❖ **Tags:**

- Added using env_prefix for environment identification and clarity.

❖ **Purpose:**

- Serves as the public-facing reverse proxy and load balancer.

```
isignmentz > modules > security > main.tf
1  resource "aws_security_group" "nginx_sg" {
17      ingress {
18      }
19  }
20
21  # HTTPS from anywhere
22  ingress {
23      from_port    = 443
24      to_port      = 443
25      protocol     = "tcp"
26      cidr_blocks  = ["0.0.0.0/0"]
27  }
28
29  # Allow all outbound traffic
30  egress {
31      from_port    = 0
32      to_port      = 0
33      protocol     = "-1"
34      cidr_blocks  = ["0.0.0.0/0"]
35  }
36
37  tags = {
38      Name = "${var.env_prefix}-nginx-sg"
39  }
40  }
```

Create Backend Security Group (main.tf)

Resource: aws_security_group for backend servers.

Ingress Rules:

- SSH (port 22) → allowed only from the user's IP (my_ip).
- HTTP (port 80) → allowed only from the Nginx security group (nginx_sg_id).

Egress Rules:

- All outbound traffic is allowed.

Tags:

- Added using env_prefix for environment identification.

Purpose:

- Restricts direct access to backend servers from the intern

```
resource "aws_security_group" "backend_sg" {
  name     = "${var.env_prefix}-backend-sg"
  vpc_id   = var.vpc_id

  # SSH from your IP only
  ingress {
    from_port     = 22
    to_port       = 22
    protocol      = "tcp"
    cidr_blocks   = [var.my_ip]
  }

  # HTTP only from Nginx SG (IMPORTANT POINT)
  ingress {
    from_port     = 80
    to_port       = 80
    protocol      = "tcp"
    security_groups = [aws_security_group.nginx_sg.id]
  }

  # Allow all outbound traffic
  egress {
    from_port     = 0
```

```
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  tags = {
    Name = "${var.env_prefix}-backend-sg"
  }
}
```

Define Outputs in Outputs (outputs.tf)

```
Assignment2 > modules > security > outputs.tf
1  output "nginx_sg_id" {
2    |   value = aws_security_group.nginx_sg.id
3    | }
4
5  output "backend_sg_id" {
6    |   value = aws_security_group.backend_sg.id
7    | }
8  |
```

The module exposes the following outputs:

- nginx_sg_id – ID of the Nginx security group
- backend_sg_id – ID of the backend security group

These outputs allow other modules or root-level Terraform code to reference the security groups.

Call security module in root main.tf


```
.ssignment2 > main.tf
1  provider "aws" {
2      shared_credentials_files = [ ~/.aws/credentials ]
3      shared_config_files      = [ "~/.aws/config" ]
4  }
5  # Module call
6  module "networking" {
7      source          = "./modules/networking"
8      vpc_cidr_block  = var.vpc_cidr_block
9      subnet_cidr_block = var.subnet_cidr_block
10     availability_zone = var.availability_zone
11     env_prefix        = var.env_prefix
12 }
13
14 # Module call
15 module "security" {
16     source      = "./modules/security"
17     vpc_id      = module.networking.vpc_id
18     env_prefix  = var.env_prefix
19     my_ip       = local.my_ip
20 }
21
```

This connects the security module to the **main infrastructure**, passing necessary variables.

terraform init then

terraform apply -auto-approve

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
module.networking.aws_vpc.this: Refreshing state... [id=vpc-0ba1bcf63d7f30bf5]
module.networking.aws_subnet.this: Refreshing state... [id=subnet-09d2a34248d1250e6]
module.networking.aws_internet_gateway.this: Refreshing state... [id=igw-080720630546527d3]
module.security.aws_security_group.nginx_sg: Refreshing state... [id=sg-00ab12453f5a34a8b]
module.networking.aws_route_table.this: Refreshing state... [id=rtb-06583758fa4e260ff]
module.security.aws_security_group.backend_sg: Refreshing state... [id=sg-0b2971b0b6eb026b2]
module.networking.aws_route_table_association.this: Refreshing state... [id=rtbassoc-0ff7ae73099876eca]

Changes to Outputs:
  + backend_sg_id = "sg-0b2971b0b6eb026b2"
  + nginx_sg_id   = "sg-00ab12453f5a34a8b"
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
backend_sg_id = "sg-0b2971b0b6eb026b2"
igw_id        = "igw-080720630546527d3"
nginx_sg_id   = "sg-00ab12453f5a34a8b"
route_table_id = "rtb-06583758fa4e260ff"
subnet_id     = "subnet-09d2a34248d1250e6"
vpc_id        = "vpc-0ba1bcf63d7f30bf5"
```

Check outputs

Terraform output

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ terraform output
backend_sg_id = "sg-0b2971b0b6eb026b2"
igw_id        = "igw-080720630546527d3"
nginx_sg_id   = "sg-00ab12453f5a34a8b"
route_table_id = "rtb-06583758fa4e260ff"
subnet_id     = "subnet-09d2a34248d1250e6"
vpc_id        = "vpc-0ba1bcf63d7f30bf5"
```

- To show this Screenshot: assignment_part1_security_module. png

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ cat modules/security/main.tf
```

```
vpc_id = var.vpc_id

# SSH from your IP only
ingress {
  from_port = 22
  to_port   = 22
  protocol  = "tcp"
  cidr_blocks = [var.my_ip]
}

# HTTP only from Nginx SG (IMPORTANT POINT)
ingress {
  from_port = 80
  to_port   = 80
  protocol  = "tcp"
  security_groups = [aws_security_group.nginx_sg.id]
}

# Allow all outbound traffic
egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}

tags = {
  Name = "${var.env_prefix}-backend-sg"
}
```

sg-00ab12453f5a34a8b - prod-nginx-sg

sg-00ab12453f5a34a8b - prod-nginx-sg

Actions

Details

Security group name

prod-nginx-sg

Security group ID

sg-00ab12453f5a34a8b

Description

Managed by Terraform

VPC ID

vpc-0ba1bcf63d7f30bf5

Owner

789925699467

Inbound rules count

3 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Tags

Inbound rules (3)



Manage tags

Edit inbound rules

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
-	sgr-05217d45127d888af	IPv4	HTTPS	TCP	443	0.0.0.0/0
-	sgr-06a2cbc317e5ee591	IPv4	SSH	TCP	22	4.240.0.0/16
-	sgr-0e1ac901c0e49f54d	IPv4	HTTP	TCP	80	0.0.0.0/0

sg-0b2971b0b6eb026b2 - prod-backend-sg

[Actions](#)

Details

Security group name
prod-backend-sg

Security group ID
sg-0b2971b0b6eb026b2

Description
Managed by Terraform

VPC ID
[vpc-0ba1bcf63d7f30bf5](#)

Owner
789925699467

Inbound rules count
2 Permission entries

Outbound rules count
1 Permission entry

[Inbound rules](#) | [Outbound rules](#) | [Sharing](#) | [VPC associations](#) | [Tags](#)

Inbound rules (2)

[Manage tags](#)[Edit inbound rules](#)[<](#) [1](#) [>](#) [{](#)

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-038ccefa57744fb95	IPv4	SSH	TCP	22
☐	-	sgr-07cd42bb8f4ed9c11	-	HTTP	TCP	80

1.5 Locals Configuration

Create locals.tf with:

- Dynamic IP detection for my_ip
- Resource naming conventions
- Common tags
- Backend server configurations

```
locals.tf
2  # Data Source - Dynamic Public IP
3
4
5  data "http" "my_ip" {
6    url = "https://icanhazip.com"
7  }
8
9
10 # Local Values
11
12
13 locals {
14
15   # Automatically detect your public IP and add /32
16   my_ip = "${chomp(data.http.my_ip.response_body)}/32"
17
18   # Common tags for all resources
19   common_tags = {
20     Environment = var.env_prefix
21     Project      = "Lab12-Assignment"
22     ManagedBy    = "Terraform"
23   }
24
25   # Resource naming convention
26   name_prefix = "${var.env_prefix}-lab12"
27
28   # Backend web servers configuration
29   backend_servers = [
```

- locals.tf is used to define reusable values in one place.

Dynamic IP Detection

- Terraform automatically detects the **public IP of the user** using the HTTP data source (<https://icanhazip.com>).

-

```
data "http" "my_ip" {
  url = "https://icanhazip.com"
}

locals {
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"
}
```

-

Stored in a local variable (my_ip) for use in **SSH access and security groups**.

- Stored in local variable
- Ensures only the current user can access EC2 instances securely.

2. Common Tags

- Defined a **common tags block** to standardize tagging across all resources:

```
locals {  
  common_tags = {  
    Environment = var.env_prefix  
    Project      = "Assignment-2"  
    ManagedBy    = "Terraform"  
  }  
}
```

- Helps in **resource identification, cost tracking, and management.**

3. Backend Server Configuration

- Defined a **list of backend servers** with:

```
locals {
  backend_servers = [
    {
      name      = "web-1"
      suffix    = "1"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name      = "web-2"
      suffix    = "2"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name      = "web-3"
      suffix    = "3"
      script_path = "./scripts/apache-setup.sh"
    }
  ]
}
```

This list is later used in the **Webserver module** to create EC2 instances dynamically.

4. Reusable Local Values

`common_tags` and `backend_servers` are reusable values.

`my_ip` is also reusable throughout the project.

Part 2 Webserver Module

2.1 Module Design

Define Variables in variables.tf

```
modules > webserver > variables.tf
1  variable "env_prefix" {
2    description = "Environment prefix (e.g. prod, dev)"
3    type        = string
4  }
5
6  variable "instance_name" {
7    description = "Base name of the EC2 instance"
8    type        = string
9  }
10
11 variable "instance_type" {
12   description = "EC2 instance type"
13   type        = string
14 }
15
16 variable "availability_zone" {
17   description = "Availability zone for EC2"
18   type        = string
19 }
20
21 variable "vpc_id" {
22   description = "VPC ID"
23   type        = string
24 }
25
26 variable "subnet_id" {
27   description = "Subnet ID"
28   type        = string
29 }
30
31 variable "security_group_id" {
```



```

variable "security_group_id" {
  description = "Security group ID to attach"
  type       = string
}

variable "public_key" {
  description = "Path to public SSH key"
  type       = string
}

variable "script_path" {
  description = "Path to user data script"
  type       = string
}

variable "instance_suffix" {
  description = "Unique suffix for instance"
  type       = string
}

variable "common_tags" {
  description = "Common tags for resources"
  type       = map(string)
}

```

assignment_part2_webserver_module_variables.png

- Define all input variables needed to create EC2 instances:
 - ❖ env_prefix → environment name (dev/prod)
 - ❖ instance_name → name of the instance
 - ❖ instance_type → EC2 type (t3.micro etc.)
 - ❖ availability_zone → AWS AZ
 - ❖ vpc_id → VPC ID from networking module
 - ❖ subnet_id → subnet ID
 - ❖ security_group_id → SG ID
 - ❖ public_key → SSH public key path
 - ❖ script_path → path to Apache/Nginx setup script
 - ❖ instance_suffix → unique identifier per instance
 - ❖ common_tags → tags for resources

Purpose: Make module reusable for multiple servers.

Output

```
modules > webserver > outputs.tf
1  output "instance_id" {
2      description = "EC2 instance ID"
3      value       = aws_instance.this.id
4  }
5
6  output "public_ip" {
7      description = "Public IP address"
8      value       = aws_instance.this.public_ip
9  }
10
11 output "private_ip" {
12     description = "Private IP address"
13     value       = aws_instance.this.private_ip
14 }
15
```

instance_id

- ❖ Outputs the unique AWS EC2 instance ID.
- ❖ Useful for tracking and referencing in other modules or Terraform outputs.

public_ip

- ❖ Outputs the public IP assigned to the instance.
- ❖ Needed to access Nginx or test connectivity from outside the VPC.

private_ip

- ❖ Outputs the private IP assigned to the instance.

ROOT OUTPUT.TF

In the **root outputs.tf**, we write outputs that collect and display information from webserver after Terraform applies the configuration.

Assignment2 > outputs.tf

```
26  -----
27  output "backend_web1_instance_id" {
28    | value = module.backend_web1.instance_id
29  }
30
31  output "backend_web1_public_ip" {
32    | value = module.backend_web1.public_ip
33  }
34
35  output "backend_web1_private_ip" {
36    | value = module.backend_web1.private_ip
37  }
38
39  output "backend_web2_instance_id" {
40    | value = module.backend_web2.instance_id
41  }
42
43  output "backend_web2_public_ip" {
44    | value = module.backend_web2.public_ip
45  }
46
47  output "backend_web2_private_ip" {
48    | value = module.backend_web2.private_ip
49  }
50
51  output "backend_web3_instance_id" {
52    | value = module.backend_web3.instance_id
53  }
54
55  output "backend_web3_public_ip" {
56    | value = module.backend_web3.public_ip
57  }
58
59  output "backend_web3_private_ip" {
60    | value = module.backend_web3.private_ip
61  }
```

Backend Server Outputs

Includes **web-1**, **web-2**, **web-3**:

- **Instance IDs** → for monitoring or SSH access.
- **Private IPs** → for internal communication (used in Nginx upstream block).
- **Public IPs** → for external access or testing if required.

Nginx Server Outputs

- Outputs **instance ID** and **public IP** for SSH access and testing HTTPS/HTTP traffic.

Create Resources in main.tf

Key Pair

- ❖ Create a unique key pair per instance using instance_suffix
- ❖ Use the provided public_key

```
modules > webserver > main.tf
1  # Fetch latest Amazon Linux 2023 AMI
2  data "aws_ami" "amazon_linux" {
3      most_recent = true
4      owners      = ["amazon"]
5
6      filter {
7          name     = "name"
8          values   = ["al2023-ami-*-x86_64"]
9      }
10 }
11
12 # Create Key Pair (unique per instance)
13 resource "aws_key_pair" "this" {
14     key_name     = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}"
15     public_key   = file(var.public_key)
16
17     tags = merge(
18         var.common_tags,
19         {
20             Name = "${var.env_prefix}-${var.instance_name}-key"
21         }
22     )
23 }
24
```

Create Resources in main.tf

EC2 Instance

- ❖ Use Amazon Linux 2023 AMI
- ❖ Enable public IP
- ❖ Attach security_group_id
- ❖ Launch in the specified subnet and AZ
- ❖ Run user data script (script_path) to setup server automatically
- ❖ Apply tags (common_tags + instance_name + env_prefix)

```

modules > webserver > main.tf
1  # Fetch latest Amazon Linux 2023 AMI
2  data "aws_ami" "amazon_linux" {
3      most_recent = true
4      owners      = ["amazon"]
5
6      filter {
7          name     = "name"
8          values   = ["al2023-ami-*-x86_64"]
9      }
10 }
11
12 # Create Key Pair (unique per instance)
13 resource "aws_key_pair" "this" {
14     key_name     = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}"
15     public_key   = file(var.public_key)
16
17     tags = merge(
18         var.common_tags,
19         {
20             Name = "${var.env_prefix}-${var.instance_name}-key"
21         }
22     )
23 }
24
25 # Create EC2 Instance
26 resource "aws_instance" "this" {
27     ami                = data.aws_ami.amazon_linux.id
28     instance_type      = var.instance_type
29     availability_zone   = var.availability_zone
30     subnet_id          = var.subnet_id
31     vpc_security_group_ids = [var.security_group_id]
32
33     associate_public_ip_address = true
34     key_name                     = aws_key_pair.this.key_name
35     user_data                    = file(var.script_path)
36
37     tags = merge(
38         var.common_tags,
39         {
40             Name = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}"
41         }
42     )
43 }

```

assignment_part2_webserver_module_main.png

Call the module in root main.tf for Nginx and backend servers with different variables.

```
Assignment2 > main.tf
21  module "backend_web1" {
33      common_tags      = local.common_tags
34  }
35
36
37  module "backend_web2" {
38      source              = "./modules/webserver"
39      env_prefix          = var.env_prefix
40      instance_name       = "web"
41      instance_suffix     = local.backend_servers[1].suffix
42      instance_type       = var.instance_type
43      availability_zone    = var.availability_zone
44      vpc_id              = module.networking.vpc_id
45      subnet_id           = module.networking.subnet_id
46      security_group_id   = module.security.backend_sg_id
47      public_key          = var.public_key
48      script_path         = local.backend_servers[1].script_path
49      common_tags         = local.common_tags
50  }
51
52  module "backend_web3" {
53      source              = "./modules/webserver"
54      env_prefix          = var.env_prefix
55      instance name       = "web"
```

Run terraform init

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ terraform init
```

Initializing the backend...

Initializing modules...

- backend_web1 in modules/webserver
- backend_web2 in modules/webserver
- backend_web3 in modules/webserver

Initializing provider plugins...

- Reusing previous version of hashicorp/http from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/http v3.5.0
- Using previously-installed hashicorp/aws v6.27.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

terraform apply -auto-approve

After this, Terraform will create all resources (VPC, subnets, EC2 instances, security groups, etc.).

```
@emanhanif1 → /workspaces/Lab12_Assignment/Assignment2 (main) $ terraform apply -auto-approve

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.
|

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

backend_sg_id = "sg-0b2971b0b6eb026b2"
backend_web1_instance_id = "i-0392e8f3517bee0ff"
backend_web1_private_ip = "10.0.10.177"
backend_web1_public_ip = "40.172.88.89"
backend_web2_instance_id = "i-056e26d7a2a24007f"
backend_web2_private_ip = "10.0.10.153"
backend_web2_public_ip = "3.28.184.51"
backend_web3_instance_id = "i-0e49ebc94db2388cc"
```

web-1, web-2, web-3:

- ❖ Instance IDs → for monitoring or SSH access.
- ❖ Private IPs → for internal communication (used in Nginx upstream block).
- ❖ Public IPs → for external access or testing if required.

After terraform apply, you can run:

terraform output

to see **all outputs** defined in your outputs.tf

```
@emanhanif1 → /workspaces/Lab12_Assignment/Assignment2 (main) $ terraform output
backend_sg_id = "sg-0b2971b0b6eb026b2"
backend_web1_instance_id = "i-0392e8f3517bee0ff"
backend_web1_private_ip = "10.0.10.177"
backend_web1_public_ip = "40.172.88.89"
backend_web2_instance_id = "i-056e26d7a2a24007f"
backend_web2_private_ip = "10.0.10.153"
backend_web2_public_ip = "3.28.184.51"
backend_web3_instance_id = "i-0e49ebc94db2388cc"
backend_web3_private_ip = "10.0.10.89"
backend_web3_public_ip = "158.252.86.112"
```

assignment_part2_webserver_module_outputs

2.2 Module Usage

Objective

Instantiate the reusable **webserver module** in the root main.tf to deploy:

- ❖ 1 Nginx server using nginx-setup.sh
- ❖ 3 backend servers (web-1, web-2, web-3) using apache-setup.sh

The backend servers are created dynamically using **for_each**, making the module reusable and scalable.

Create Nginx server module in main.tf

```
# Nginx Server
module "nginx_server" {
  source          = "./modules/webserver"
  env_prefix      = var.env_prefix
  instance_name   = "nginx-proxy"
  instance_type   = var.instance_type
  availability_zone = var.availability_zone

  vpc_id          = module.networking.vpc_id
  subnet_id       = module.networking.subnet_id
  security_group_id = module.security.nginx_sg_id

  public_key      = var.public_key
  script_path     = "./scripts/nginx-setup.sh"
  instance_suffix = "nginx"

  common_tags     = local.common_tags
}
```

assignment_part2_main_tf_modules.png

❖ Variables passed:

- instance_name = "nginx-proxy"
- security_group_id = module.security.nginx_sg_id
- script_path = "./scripts/nginx-setup.sh"
- instance_suffix = "nginx"
- Other common variables: env_prefix, instance_type, availability_zone, vpc_id, subnet_id, public_key, common_tags

Create backend server module instances using `for_each`

- ❖ Module block: module "backend_servers" with **for_each**
- ❖ Iterates over `local.backend_servers` list
- ❖ Variables passed dynamically for each server:

`instance_name = each.value.name`

`security_group_id = module.security.backend_sg_id`

`script_path = each.value.script_path`

`instance_suffix = each.value.suffix`

```
# Backend Servers (3 instances)
module "backend_servers" {
  for_each = { for server in local.backend_servers : server.name => server }

  source           = "../modules/webserver"
  env_prefix       = var.env_prefix
  instance_name    = each.value.name
  instance_type    = var.instance_type
  availability_zone = var.availability_zone

  vpc_id           = module.networking.vpc_id
  subnet_id        = module.networking.subnet_id
  security_group_id = module.security.backend_sg_id

  public_key       = var.public_key
  script_path      = each.value.script_path
  instance_suffix  = each.value.suffix
  common_tags      = local.common_tags
}
```

assignment_part2_main_tf_modules.png

common variables are used: `env_prefix`, `instance_type`, `availability_zone`, `vpc_id`, `subnet_id`, `public_key`, `common_tags`

Terraform Apply `-auto-approve`

- This command **deploys all Terraform resources defined in the configuration files** (main.tf, modules, etc.).

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ terraform apply -auto-approve
  set as cleartext in state

  (and 10 more similar warnings elsewhere)
```

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

```
backend_sg_id = "sg-0b2971b0b6eb026b2"
backend_web1_instance_id = "i-0392e8f3517bee0ff"
backend_web1_private_ip = "10.0.10.177"
backend_web1_public_ip = "40.172.88.89"
backend_web2_instance_id = "i-056e26d7a2a24007f"
backend_web2_private_ip = "10.0.10.153"
backend_web2_public_ip = "3.28.184.51"
backend_web3_instance_id = "i-0e49ebc94db2388cc"
backend_web3_private_ip = "10.0.10.89"
backend_web3_public_ip = "158.252.86.112"
igw_id = "igw-080720630546527d3"
nginx_sg_id = "sg-00ab12453f5a34a8b"
route_table_id = "rtb-06583758fa4e260ff"
subnet_id = "subnet-09d2a34248d1250e6"
vpc_id = "vpc-0ba1bcf63d7f30bf5"
```

Terraform Output

- Displays **all outputs defined in outputs.tf** after deployment.

```

@emanhanif1 → /workspaces/Lab12_Assignment (main) $ terraform output
backend_instance_ids = {
  "web-1" = "i-01ac5020921cf4139"
  "web-2" = "i-04e9a0da483ef5692"
  "web-3" = "i-0cff5d924c4b7de4a"
}
backend_private_ips = {
  "web-1" = "10.10.1.71"
  "web-2" = "10.10.1.12"
  "web-3" = "10.10.1.140"
}
backend_public_ips = {
  "web-1" = "3.28.130.160"
  "web-2" = "158.252.82.178"
  "web-3" = "40.172.215.19"
}
nginx_instance_id = "i-008aeb366fe364f56"
nginx_private_ip = "10.10.1.74"
nginx_public_ip = "158.252.95.150"
@emanhanif1 → /workspaces/Lab12_Assignment (main) $

```

📌 **Instance IDs** → Unique identifiers for each EC2 instance. Useful for **monitoring, SSH access, and management**.

📌 **Private IPs** → Internal IP addresses of instances.

- ❖ Used for **internal communication** within the VPC.
- ❖ For example, Nginx uses **backend private IPs** to route traffic in the upstream block.

📌 **Public IPs** → External IP addresses.

- ❖ Used for **testing, SSH from admin IP**, and accessing the Nginx load balancer from the internet.

ensure proper security group assignment

```

----- nginx -----
security_group_id = module.security.nginx_sg_id
----- backend -----

security_group_id = module.security.backend_sg_id

```

- ❖ Nginx SG allows HTTP/HTTPS from **internet**
- ❖ Backend SG allows HTTP only from **Nginx SG**

Pass all required variables

- All variables like env_prefix, instance_type, availability_zone, vpc_id, subnet_id, public_key, script_path and common_tags are passed correctly.

Key Points

- ❖ **Reusability:** Same module is used for Nginx and backend servers, only variable values change.
- ❖ **Dynamic deployment:** Using for_each allows creating multiple backend servers with a single block.
- ❖ **Security:** Correct security group assigned for Nginx and backend servers.

Part 3: Server Configuration Scripts

3.1 Apache Backend Server Script

Update & Install Apache

```
yum update -y  
yum install httpd -y
```

- ❖ Updates system packages.
- ❖ Installs Apache HTTP Server.

Starts and enables Apache service

- ❖ systemctl start httpd
- ❖ systemctl enable httpd

Starts Apache and ensures it **runs on boot**.

- The page includes:
 - ❖ Nice **background gradient and container styling**
 - ❖ Labels and values for all metadata

- ❖ Timestamp and server status
- ❖ CSS makes the page visually appealing for the assignment.

```
# Get metadata token (IMDSv2)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
```

scripts/apache-setup.sh

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ cd /workspaces/Lab12_Assignment/Assignment2/scripts
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2/scripts (main) $
```

Key Features

- #!/bin/bash and set -e for safe execution
- Updates system
- Installs Apache
- Starts & enables Apache service:
- Uses **IMDSv2** to fetch instance metadata:
 - Private IP, Public IP, Public DNS, Instance ID
- Sets hostname: hostnamectl set-hostname myapp-webserver
- Creates **custom HTML page** at /var/www/html/index.html
 - Displays hostname, IPs, instance ID, deployment timestamp, server status
- Sets proper permissions: chmod 644 /var/www/html/index.html

```

<!DOCTYPE html>
<html>
<head>
  <title>Backend Web Server</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 50px;
      background: #ffc0cb; /* Light pink background */
      color: white;
    }
    .container {
      background: rgba(255, 255, 255, 0.2);
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 8px 32px rgba(31, 38, 135, 0.37);
    }
    h1 {
      color: maroon; /* Maroon heading */
      text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
    }
  
```

Assignment2 > scripts > \$ apache-setup.sh

```

3   <head>
5   <style>
22   .info {
23     margin: 15px 0;
24     padding: 10px;
25     background: rgba(255,255,255,0.1);
26     border-radius: 5px;
27     color: black;
28   }
29   .label { font-weight: bold; color: #8b0000; } /* Dark red labels */
30 </style>
31 </head>
32 <body>
33   <div class="container">
34     <h1>Backend Web Server - Assignment 2</h1>
35     <div class="info"><span class="label">Hostname:</span> web-1</div>
36     <div class="info"><span class="label">Instance ID:</span> i-0123456789abcdef</div>
37     <div class="info"><span class="label">Private IP:</span> 10.0.1.15</div>
38     <div class="info"><span class="label">Public IP:</span> 3.29.21.56</div>
39     <div class="info"><span class="label">Public DNS:</span> ec2-3-29-21-56.compute-1.amazonaws.com</div>
40     <div class="info"><span class="label">Deployed:</span> 2025-12-28 16:00</div>
41     <div class="info"><span class="label">Status:</span> ☒ Active and Running</div>
42     <div class="info"><span class="label">Managed By:</span> Terraform</div>
43   </div>
44 </body>
45 </html>

```

- File name: assignment_part3_apache_script.png

Backend Web Page Verification

Steps:

- ❖ Open backend server public IP in browser: <http://3.29.5.181/>
- ❖ Page displayed server metadata correctly.

Now your backend servers are accessible via **HTTP** from any browser.

```
Complete!
[ec2-user@ip-10-10-1-71 ~]$ sudo systemctl start httpd
sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service
[ec2-user@ip-10-10-1-71 ~]$ sudo systemctl status httpd
• httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Sun 2025-12-21 19:35:24 UTC; 9s ago
```

☐	prod-nginx-pr...	i-0eee88943b7e23a56	✔ Running	🔍	t3.micro	✔ 3/3 checks passec	View alarms +	me-central-1a	-
☐	prod-web-3-3	i-0d2a690e1f3347e2a	✔ Running	🔍	t3.micro	✔ 3/3 checks passec	View alarms +	me-central-1a	-
☐	prod-web-2	i-056e26d7a2a24007f	✔ Running	🔍	t3.micro	✔ 3/3 checks passec	View alarms +	me-central-1a	-
☐	prod-web-2-2	i-0105c8dd5a8070ae1	✔ Running	🔍	t3.micro	✔ 3/3 checks passec	View alarms +	me-central-1a	-
☐	prod-web-1-1	i-00c0eff1448559bcf	✔ Running	🔍	t3.micro	✔ 3/3 checks passec	View alarms +	me-central-1a	-
☐	apache-test	i-095274a7158d26110	✔ Running	🔍	t3.micro	⌚ Initializing	View alarms +	me-central-1c	ec2-3-2E

```

@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2/scripts (main) $ sudo ./apache-setup.sh
Get:29 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libgirepository-2.0-0 amd64 2.80.0-6ubuntu3.5 [73.5 kB]
Get:30 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libglib2.0-dev-bin amd64 2.80.0-6ubuntu3.5 [138 kB]
Get:31 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 gir1.2-glib-2.0 amd64 2.80.0-6ubuntu3.5 [183 kB]
Get:32 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libglib2.0-0t64 amd64 2.80.0-6ubuntu3.5 [1544 kB]
Get:33 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 python3-apt-common all 2.7.7ubuntu5.1 [20.8 kB]
Get:34 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 python3-apt amd64 2.7.7ubuntu5.1 [169 kB]
Get:35 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 bsdxtrautils amd64 2.39.3-9ubuntu6.4 [73.7 kB]
Get:36 http://archive.ubuntu.com/ubuntu noble/main amd64 libnftables1 amd64 1.0.9-1build1 [358 kB]
Get:37 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libpng-dev amd64 1.6.43-5ubuntu0.1 [264 kB]
Get:38 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libpng16-16t64 amd64 1.6.43-5ubuntu0.1 [187 kB]
Get:39 http://archive.ubuntu.com/ubuntu noble/main amd64 nftables amd64 1.0.9-1build1 [69.8 kB]
Get:40 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libgprofng0 amd64 2.42-4ubuntu2.8 [849 kB]
Get:41 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libctf0 amd64 2.42-4ubuntu2.8 [94.5 kB]
Get:42 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libctf-nobfd0 amd64 2.42-4ubuntu2.8 [97.9 kB]
Get:43 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 binutils-x86-64-linux-gnu amd64 2.42-4ubuntu2.8 [2463 kB]
Get:44 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libbinutils amd64 2.42-4ubuntu2.8 [576 kB]
Get:45 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 binutils amd64 2.42-4ubuntu2.8 [18.1 kB]
Get:46 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 binutils-common amd64 2.42-4ubuntu2.8 [240 kB]
Get:47 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libsframe1 amd64 2.42-4ubuntu2.8 [15.6 kB]
Get:48 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libmysqlclient-dev amd64 8.0.44-0ubuntu0.24.04.2 [1591 kB]
Get:49 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libmysqlclient21 amd64 8.0.44-0ubuntu0.24.04.2 [1254 kB]
Get:50 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libpq-dev amd64 16.11-0ubuntu0.24.04.1 [152 kB]
Get:51 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libpq5 amd64 16.11-0ubuntu0.24.04.1 [145 kB]
Get:52 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libunbound8 amd64 1.19.2-1ubuntu3.7 [443 kB]
Get:53 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 linux-libc-dev amd64 6.8.0-90.91 [1932 kB]
Fetched 129 MB in 5s (26.2 MB/s)

```

3. SSH into the EC2 instance:

ssh -i your-key.pem ec2-user@<EC2_PUBLIC_IP>

```

@emanhanif1 →~/ .ssh $ nano ~/.ssh/terraform_key.pem
@emanhanif1 →~/ .ssh $ chmod 400 ~/.ssh/terraform_key.pem
@emanhanif1 →~/ .ssh $ ssh -i ~/.ssh/terraform_key.pem ec2-user@3.28.192.129

#
~\ ##### Amazon Linux 2023
~\ #####\
~\ \####|
~\ \#/ https://aws.amazon.com/linux/amazon-linux-2023
~\ V~' ->
~\ /
~\ /
~\ /m/'

Last login: Sun Dec 28 19:48:42 2025 from 119.154.15.243
[ec2-user@ip-172-31-9-110 ~]$

```

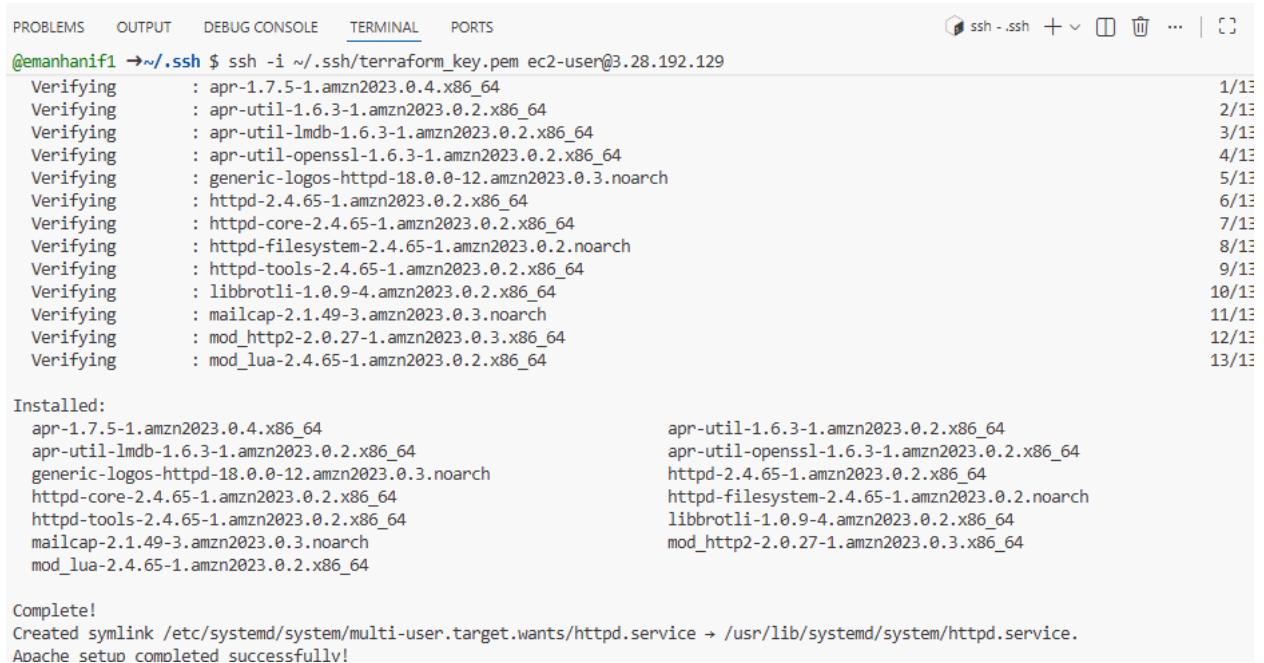


```
[ec2-user@ip-172-31-9-110 ~]$ mkdir -p scripts
[ec2-user@ip-172-31-9-110 ~]$ nano scripts/apache-setup.sh
```

4. Run the script:

`sudo ./apache-setup.sh`

```
[ec2-user@ip-172-31-9-110 ~]$ chmod +x scripts/apache-setup.sh
[ec2-user@ip-172-31-9-110 ~]$ sudo ./scripts/apache-setup.sh
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
@emanhanif1 → ~/.ssh $ ssh -i ~/.ssh/terraform_key.pem ec2-user@3.28.192.129
Verifying      : apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Verifying      : apr-util-1.6.3-1.amzn2023.0.2.x86_64 2/13
Verifying      : apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64 3/13
Verifying      : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 4/13
Verifying      : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 5/13
Verifying      : httpd-2.4.65-1.amzn2023.0.2.x86_64 6/13
Verifying      : httpd-core-2.4.65-1.amzn2023.0.2.x86_64 7/13
Verifying      : httpd-filesystem-2.4.65-1.amzn2023.0.2.noarch 8/13
Verifying      : httpd-tools-2.4.65-1.amzn2023.0.2.x86_64 9/13
Verifying      : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 10/13
Verifying      : mailcap-2.1.49-3.amzn2023.0.3.noarch 11/13
Verifying      : mod_http2-2.0.27-1.amzn2023.0.3.x86_64 12/13
Verifying      : mod_lua-2.4.65-1.amzn2023.0.2.x86_64 13/13

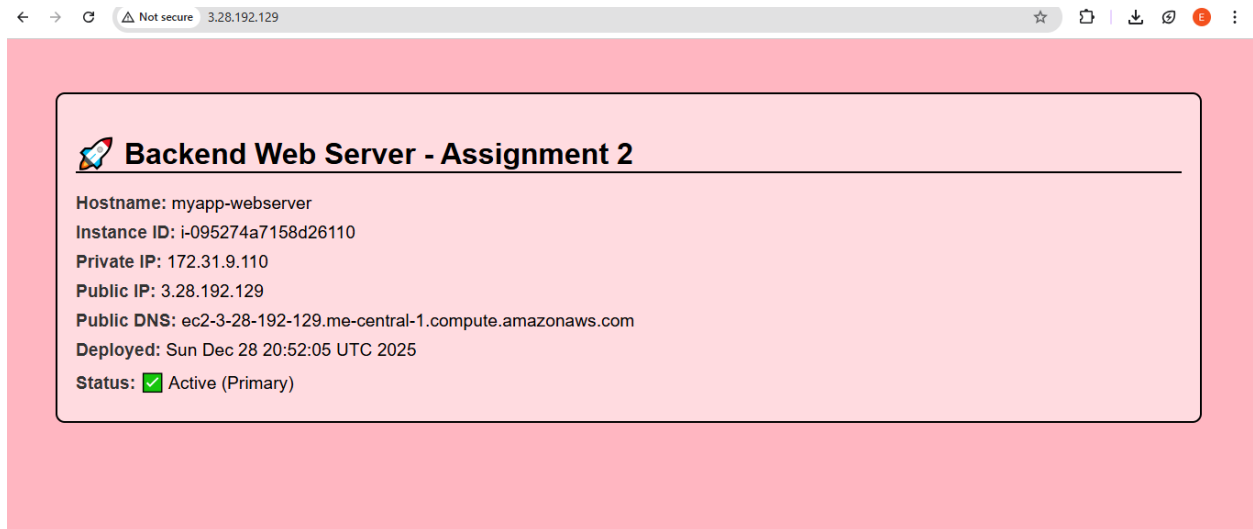
Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
httpd-core-2.4.65-1.amzn2023.0.2.x86_64
httpd-filesystem-2.4.65-1.amzn2023.0.2.noarch
httpd-tools-2.4.65-1.amzn2023.0.2.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch
mod_lua-2.4.65-1.amzn2023.0.2.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
httpd-2.4.65-1.amzn2023.0.2.x86_64
httpd-filesystem-2.4.65-1.amzn2023.0.2.noarch
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mod_http2-2.0.27-1.amzn2023.0.3.x86_64

Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
Apache setup completed successfully!
```

5. Open the browser and check:

Refresh your browser at <http://3.28.192.129>

- Make sure all info is displayed: hostname, private/public IP, instance ID, timestamp, status.



Browser screenshot showing backend web page with server info

3.2 Nginx Server Setup Script

Objective:

Automate the configuration of the Nginx frontend server using a Bash script:

- Update system packages
- Install Nginx
- Generate a self-signed SSL certificate
- Configure HTTPS with HTTP-to-HTTPS redirect
- Set up load balancing with upstream backend servers
- Enable caching, proper logging, security headers, and health checks

Implementation

Script Location:

scripts/nginx-setup.sh

Step 1: SSL Certificate Generation

Nginx can serve HTTPS

1. Update system + install Nginx/OpenSSL

```
# Update and install Nginx
yum update -y
yum install -y nginx openssl
systemctl start nginx
systemctl enable nginx
```

2. Create directories:

```
mkdir -p /etc/ssl/private /etc/ssl/certs
```

```
# Create SSL directories
mkdir -p /etc/ssl/private
mkdir -p /etc/ssl/certs
```

Generate self-signed certificate:

```
# Generate self-signed certificate
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/selfsigned.key \
  -out /etc/ssl/certs/selfsigned.crt \
  -subj "/CN=$PUBLIC_IP" \
  -addext "subjectAltName=IP:$PUBLIC_IP" \
  -addext "basicConstraints=CA:FALSE" \
  -addext "keyUsage=digitalSignature,keyEncipherment" \
  -addext "extendedKeyUsage=serverAuth"

echo "Self-signed certificate created for IP: $PUBLIC_IP"
```

```
# SSL Configuration
ssl_certificate /etc/ssl/certs/selfsigned.crt;
ssl_certificate_key /etc/ssl/private/selfsigned.key;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;
ssl_prefer_server_ciphers on;
```

Implement Caching

Goal: Improve performance

In http block of nginx.conf:

```
nginx 1.20.1 nginx-setup.sh
# Proxy settings
location / {
    proxy_pass http://backend_servers;

    # Proxy headers
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # Cache settings
    proxy_cache my_cache;
    proxy_cache_valid 200 60m;
    proxy_cache_valid 404 10m;
    proxy_cache_key "$scheme$request_method$host$request_uri";
    proxy_cache_bypass $http_cache_control;
    add_header X-Cache-Status $upstream_cache_status;
```

Step3

Update Upstream Block with Real IPs

In your nginx-setup.sh, find the **upstream backend_servers** section and replace placeholders with real IPs:

```

ano 7.2 nginx-setup.sh *
# Primary servers (active load balancing)
server 10.0.10.177:80;
server 10.0.10.153:80;

# Backup server (only used when primary servers are down)
server 10.0.10.89:80 backup;

```

❓ **Primary servers:** web-1 & web-2 → load balanced

❓ **Backup server:** web-3 → only used if primary servers fail

Add Security Headers

Goal: Hardening Nginx

In HTTPS server block:

```

# HTTPS Server
server {
    listen 443 ssl http2;
    server_name _;

    # SSL Configuration
    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    # Security Headers
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
}

```

Configure HTTP to HTTPS Redirect

Goal: Force HTTPS

HTTP server block:

```

# HTTP Server (redirect to HTTPS)
server {
    listen 80;
    server_name _;

    location / {
        return 301 https://$host$request_uri;
    }

    # Allow health checks over HTTP
    location /health {
        access_log off;
        return 200 "Nginx is healthy\n";
        add_header Content-Type text/plain;
    }
}

```

Go into nginx

```

@emanhanif1 →/workspaces/Lab12_Assignment (main) $ ssh -i ~/.ssh/terraform_key ec2-user@3.28.137.93
Warning: Identity file /home/codespace/.ssh/terraform_key not accessible: No such file or directory.

```

```

#
##### Amazon Linux 2023
#####\
\###|
\##/ https://aws.amazon.com/linux/amazon-linux-2023
V~'-'>
~m/'

```

Last login: Mon Dec 29 19:42:46 2025 from 3.29.147.40

```
[ec2-user@ip-10-0-10-110 ~]$ nginx -v
```

nginx version: nginx/1.28.0

```
[ec2-user@ip-10-0-10-110 ~]$ sudo systemctl status nginx
```

```

● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
   Active: active (running) since Mon 2025-12-29 00:07:23 UTC; 1 day 12h ago
     Process: 38491 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 38493 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 38494 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)

```

```
[ec2-user@ip-10-0-10-110 ~]$ sudo nginx -t
```

nginx: [warn] the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:53

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok

nginx: configuration file /etc/nginx/nginx.conf test is successful

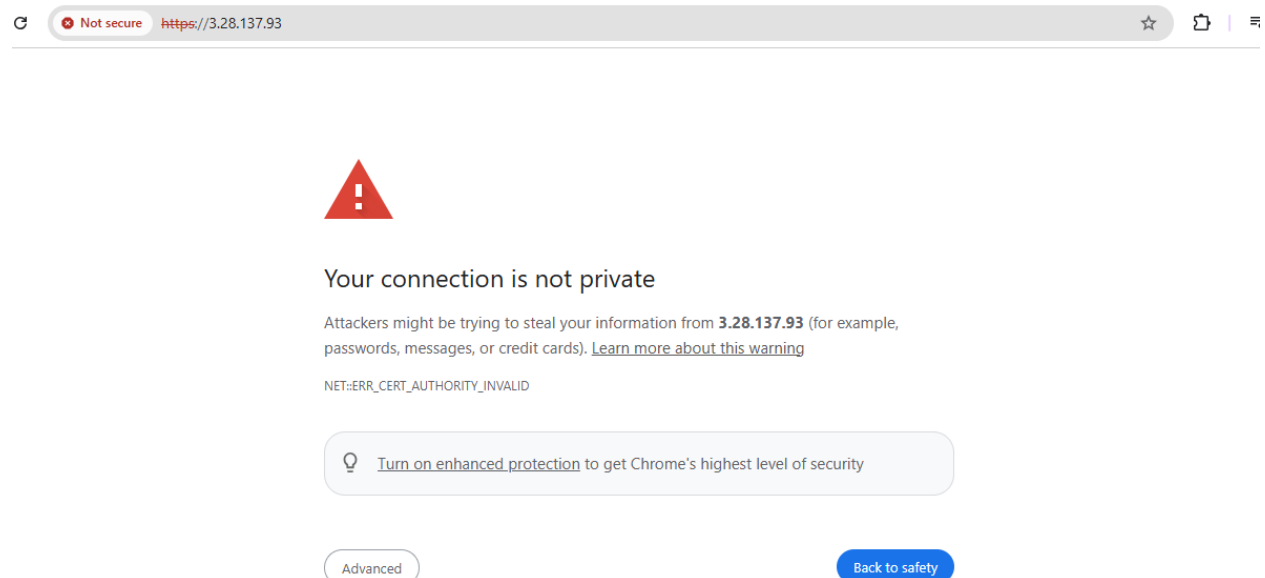
```
[ec2-user@ip-10-0-10-110 ~]$ sudo systemctl restart nginx
```

```
[ec2-user@ip-10-0-10-110 ~]$ curl -I http://localhost/health
HTTP/1.1 200 OK
Server: nginx/1.28.0
Date: Tue, 30 Dec 2025 13:28:53 GMT
Content-Type: application/octet-stream
Content-Length: 17
Connection: keep-alive
Content-Type: text/plain

[ec2-user@ip-10-0-10-110 ~]$
```

🔗 this proves that Nginx configuration is active, the syntax errors are gone, and the specific /health location you defined is working perfectly.

🔗 **Service Running:** The 200 OK status confirms that the Nginx service is not only active but correctly serving the customized configuration from your script.



Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Security group ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info		
sgr-0b3e6bb3e6d9eca77	SSH	TCP	22	Custom	0.0.0.0		Delete
sgr-0d941f4c5ea70df16	HTTP	TCP	80	Custom	0.0.0.0		Delete
-	HTTPS	TCP	443	Anyw...	0.0.0.0		Delete

[Add rule](#)

⚠️ Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

You will see a "Your connection is not private" warning because we are using a **self-signed certificate**. Click **Advanced** and then **Proceed** to see the final page.

Not secure <https://3.28.192.129>

☆ 🖨️

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Verification

- Open frontend server public IP in browser: `http://<nginx-public-ip>/`
- Verify HTTPS works: `https://<nginx-public-ip>/`
- Check health endpoint: `https://<nginx-public-ip>/health`

Verify Nginx Default Page

- Open your **browser** and go to:
- `https://<NGINX_PUBLIC_IP>`
- You may get a warning about the SSL certificate (self-signed → normal).
- This will be your **default Nginx page, before backend IPs are configured**.
- Take a **screenshot** → `assignment_part3_nginx_default_page.png` (deliverable).

Part 4: Infrastructure Deployment

Deploy the infrastructure using Terraform.

Verify all resources in AWS Console.

```
@manhanif1 →/workspaces/Lab12_Assignment (main) $ ssh-keygen -t rsa -b 2048 -f ~/.ssh/terraform_key.pem -N ""
Generating public/private rsa key pair.
/home/codespace/.ssh/terraform_key.pem already exists.
Overwrite (y/n)? n
```

```
@manhanif1 →/workspaces/Lab12_Assignment (main) $ ls ~/.ssh/terraform_key
/home/codespace/.ssh/terraform_key
@manhanif1 →/workspaces/Lab12_Assignment (main) $
```

That means **don't need to generate a new one** for Terraform.

```
@emanhanif1 →/workspaces/Lab12_Assignment (main) $ terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/http from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/http v3.5.0
- Using previously-installed hashicorp/aws v6.27.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

2. Validate configuration

terraform validate

```
@emanhanif1 →/workspaces/Lab12_Assignment (main) $ terraform validate
Success! The configuration is valid.
```

3. Plan deployment

terraform plan

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ terraform plan
}
# (8 unchanged attributes hidden)
}
```

Plan: 0 to add, 8 to change, 0 to destroy.

Changes to Outputs:

```
~ backend_web1_public_ip = "40.172.88.89" -> (known after apply)
~ backend_web2_public_ip = "3.28.184.51" -> (known after apply)
~ backend_web3_public_ip = "158.252.86.112" -> (known after apply)
```

Warning: Value is base64 encoded

```
with module.nginx_server.aws_instance.server,
on modules/webserver/main.tf line 16, in resource "aws_instance" "server":
16:   user_data      = file(var.script_path)
```

The value is base64 encoded. If you want to use base64 encoding, please use the user_data_base64 argument. user_data attribute is set as cleartext in state

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

4. Apply configuration

terraform apply -auto-approve

```
@emanhanif1 → /workspaces/Lab12_Assignment (main) $ terraform apply -auto-approve
Plan: 0 to add, 3 to change, 0 to destroy.
```

```
Changes to Outputs:
  ~ nginx_public_ip      = "158.252.95.150" -> (known after apply)
module.security.aws_security_group.nginx: Modifying... [id=sg-0e306e98e9ba96898]
module.security.aws_security_group.nginx: Modifications complete after 1s [id=sg-0e306e98e9ba96898]
module.security.aws_security_group.backend: Modifying... [id=sg-0251ea5e4ce796a4c]
module.nginx_server.aws_instance.this: Modifying... [id=i-008aeb366fe364f56]
module.security.aws_security_group.backend: Modifications complete after 1s [id=sg-0251ea5e4ce796a4c]
module.nginx_server.aws_instance.this: Still modifying... [id=i-008aeb366fe364f56, 00m10s elapsed]
module.nginx_server.aws_instance.this: Still modifying... [id=i-008aeb366fe364f56, 00m20s elapsed]
module.nginx_server.aws_instance.this: Still modifying... [id=i-008aeb366fe364f56, 00m30s elapsed]
module.nginx_server.aws_instance.this: Still modifying... [id=i-008aeb366fe364f56, 00m40s elapsed]
module.nginx_server.aws_instance.this: Still modifying... [id=i-008aeb366fe364f56, 00m50s elapsed]
module.nginx_server.aws_instance.this: Modifications complete after 53s [id=i-008aeb366fe364f56]
```

Apply complete! Resources: 0 added, 3 changed, 0 destroyed.

Outputs:

```
backend_instance_ids = {
  "web-1" = "i-01ac5020921cf4139"
  "web-2" = "i-04e9a0da483ef5692"
  "web-3" = "i-0cff5d924c4b7de4a"
}
backend_private_ips = {
  "web-1" = "10.10.1.71"
  "web-2" = "10.10.1.12"
  "web-3" = "10.10.1.140"
}
backend_public_ips = {
  "web-1" = "3.29.5.181"
  "web-2" = "3.29.21.56"
  "web-3" = "3.29.30.73"
}
nginx_instance_id = "i-008aeb366fe364f56"
nginx_private_ip = "10.10.1.74"
nginx_public_ip = "158.252.77.173"
```

4.2 Output Configuration

Create comprehensive outputs in outputs.tf:

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ terraform output
```

```
backend_web3_private_ip = "10.0.10.89"  
backend_web3_public_ip = "3.28.43.87"  
configuration_guide = <<EOT  
=====
```

```
DEPLOYMENT SUCCESSFUL!  
=====
```

Next Steps:

1. SSH into Nginx server: `ssh ec2-user@3.28.137.93`
2. Edit Nginx config: `sudo vim /etc/nginx/nginx.conf`
3. Update backend IPs in upstream block:
 - BACKEND_IP_1: 10.0.10.59
 - BACKEND_IP_2: 10.0.10.52
 - BACKEND_IP_3: 10.0.10.203
4. Restart Nginx: `sudo systemctl restart nginx`
5. Test: `https://3.28.137.93`

Backend Servers:

- web-1: 51.112.43.218 (private: 10.0.10.59)
- web-2: 51.112.186.36 (private: 10.0.10.52)
- web-3: 40.172.215.170 (private: 10.0.10.203)

```
=====
```

EOT

```
igw_id = "igw-080720630546527d3"  
nginx_instance_id = "i-0eee88943b7e23a56"  
nginx_public_ip = "3.28.137.93"  
nginx_sg_id = "sg-00ab12453f5a34a8b"  
route_table_id = "rtb-06583758fa4e260ff"  
subnet_id = "subnet-09d2a34248d1250e6"  
vpc_id = "vpc-0ba1bcf63d7f30bf5"
```

assignment_part4_terraform_output

Each output has a **clear, helpful description** (description = "...") explaining exactly what it is.

Networking Outputs

Shows **VPC and subnet IDs** created by Terraform.

- Useful for network troubleshooting or reference in other modules.

Nginx Server Outputs

- Provides **public IP** to SSH or test Nginx.
- Instance ID for reference or debugging.

Backend Servers Outputs

- Shows **all backend servers** (web-1, web-2, web-3) info.
- You can see **public & private IPs**, useful for configuring Nginx upstream.

```
@emanhanif1 →/workspaces/Lab12_Assignment/Assignment2 (main) $ cat outputs.json
Steps:\n1. SSH into Nginx server: ssh ec2-user@3.28.137.93\n2. Edit Nginx config: sudo vim /etc/nginx/nginx.conf\n3. Update backend IP
s in upstream block:\n - BACKEND_IP_1: 10.0.10.59\n - BACKEND_IP_2: 10.0.10.52\n - BACKEND_IP_3: 10.0.10.203\n4. Restart Nginx:
sudo systemctl restart nginx\n5. Test: https://3.28.137.93\n\nBackend Servers:\n- web-1: 51.112.43.218 (private: 10.0.10.59)\n
- web-2: 51.112.186.36 (private: 10.0.10.52)\n - web-3: 40.172.215.170 (private: 10.0.10.203)\n=====
=====\n"
  },
  "igw_id": {
    "sensitive": false,
    "type": "string",
    "value": "igw-080720630546527d3"
  },
  "nginx_instance_id": {
    "sensitive": false,
    "type": "string",
    "value": "i-0eee88943b7e23a56"
  },
  "nginx_public_ip": {
    "sensitive": false,
    "type": "string",
    "value": "3.28.137.93"
  },
  "nginx_sg_id": {
    "sensitive": false,
    "type": "string",
    "value": "sg-00ab12453f5a34a8b"
  },
  "route_table_id": {
    "sensitive": false,
    "type": "string",
    "value": "rtb-06583758fa4e260ff"
  },
  },
```

The cat outputs.json screenshot confirms that you have all the mandatory data points:

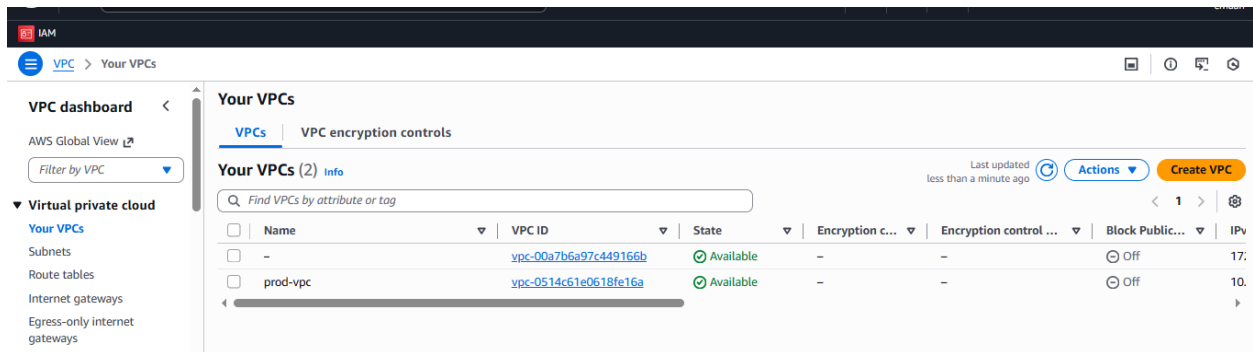
- **Configuration Guide:** Shows the SSH command and backend IP addresses (10.0.10.59, 10.0.10.52, 10.0.10.203).
- **Nginx Server Info:** Displays the nginx_instance_id and nginx_public_ip (3.28.137.93).
- **Networking Info:** Shows the igw_id and route_table_id.

.3 AWS Console Verification

Verify all resources in AWS Console

Verify VPC Created

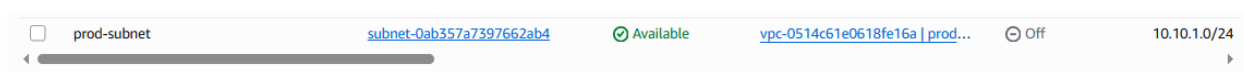
```
@emanhanif1 →/workspaces/Lab12_Assignment (main) $ terraform output vpc_id
"vpc-0514c61e0618fe16a"
@emanhanif1 →/workspaces/Lab12_Assignment (main) $
```



Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv
-	vpc-00a7b6a97c449166b	Available	-	-	Off	17.
prod-vpc	vpc-0514c61e0618fe16a	Available	-	-	Off	10.

Verify Subnet Created

```
@emanhanif1 →/workspaces/Lab12_Assignment (main) $ terraform output subnet_id
"subnet-0ab357a7397662ab4"
@emanhanif1 →/workspaces/Lab12_Assignment (main) $
```



Name	Subnet ID	State	VPC	Block Public...	IPv
prod-subnet	subnet-0ab357a7397662ab4	Available	vpc-0514c61e0618fe16a prod...	Off	10.10.1.0/24

Verify Route Table Configured

- AWS Console:
 - VPC → Route Tables → check routes exist:
 - 0.0.0.0/0 → IGW for public subnet

b-0b50070174d33c799 🔍 📄 🔄

rtb-0b50070174d33c799 / prod-rt Actions ▾

Details info
Route table ID
 rtb-0b50070174d33c799
VPC
vpc-0514c61e0618fe16a | prod-vpc

Main
 No
Owner ID
 789925699467

Explicit subnet associations
[subnet-0ab357a7397662ab4](#) / prod-subnet

Edge associations
-

Routes | Subnet associations | Edge associations | Route propagation | Tags

Routes (2) Both ▾ Edit routes

Destination ▾	Target ▾	Status ▾	Propagated ▾	Route Origin ▾
0.0.0.0/0	igw-014c02f9da699906f	Active	No	Create Route

Verify Internet Gateway (IGW) Attached

- AWS Console:
 - VPC → Internet Gateways → check IGW attached to your VPC.

Internet gateways (3) info 🔍 📄 🔄 Actions ▾ Create internet gateway

☐	Name ▾	Internet gateway ID ▾	State ▾	VPC ID ▾	Owner
☐	prod-igw	igw-014c02f9da699906f	Attached	vpc-0514c61e0618fe16a prod-vpc	789925699467

Verify All 4 EC2 Instances Running

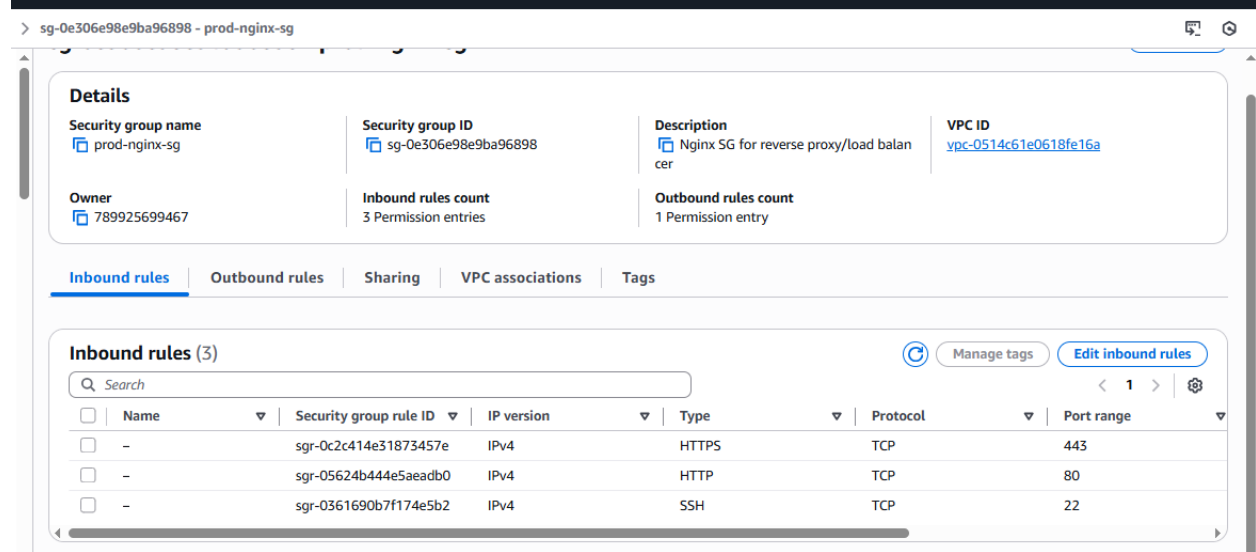
- AWS Console:
 - EC2 → Instances → status = **running**

☐	prod-web-3-3	i-0d2a690e1f3347e2a	Running	t3.micro	3/3 checks passed	View alarms +	me-central-1a	-
☐	prod-web-2	i-056e26d7a2a24007f	Running	t3.micro	3/3 checks passed	View alarms +	me-central-1a	-
☐	prod-web-2-2	i-0105c8dd5a8070ae1	Running	t3.micro	3/3 checks passed	View alarms +	me-central-1a	-
☐	prod-web-1-1	i-00c0eff1448559bcf	Running	t3.micro	3/3 checks passed	View alarms +	me-central-1a	-
☐	apache-test	i-095274a7158d26110	Running	t3.micro	3/3 checks passed	View alarms +	me-central-1c	ec2-3-2f

- Terraform:

terraform output nginx_public_ip
terraform output backend_servers_info

```
@emanhanif1 → /workspaces/Lab12_Assignment (main) $ terraform output nginx_public_ip
"158.252.77.173"
```



The screenshot shows the AWS Management Console for a security group named 'prod-nginx-sg'. The details section includes the Security group ID (sg-0e306e98e9ba96898), Description (Nginx SG for reverse proxy/load balancer), VPC ID (vpc-0514c61e0618fe16a), Owner (789925699467), Inbound rules count (3 Permission entries), and Outbound rules count (1 Permission entry). Below the details, the 'Inbound rules' tab is selected, showing a table with 3 rules.

Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sgr-0c2c414e31873457e	IPv4	HTTPS	TCP	443
-	sgr-05624b444e5aeadb0	IPv4	HTTP	TCP	80
-	sgr-0361690b7f174e5b2	IPv4	SSH	TCP	22

```
@emanhanif1 → /workspaces/Lab12_Assignment (main) $ terraform output backend_servers_info
{
  "web-1" = {
    "instance_id" = "i-01ac5020921cf4139"
    "private_ip" = "10.10.1.71"
    "public_ip" = "3.29.5.181"
  }
  "web-2" = {
    "instance_id" = "i-04e9a0da483ef5692"
    "private_ip" = "10.10.1.12"
    "public_ip" = "3.29.21.56"
  }
  "web-3" = {
    "instance_id" = "i-0cff5d924c4b7de4a"
    "private_ip" = "10.10.1.140"
    "public_ip" = "3.29.30.73"
  }
}
```

Verify Key Pairs Created

- AWS Console:
 - EC2 → Key Pairs → check your keys exist.

☐	Name	Type	Created	Fingerprint	ID
☐	prod-web-2-key	ed25519	2025/12/28 20:29 GMT+5	6e4aoli7x1iliBU2mFcWoQnRYhGDJ...	key-08b755e559da5227e
☐	prod-web-3-3-key	ed25519	2025/12/28 21:07 GMT+5	6e4aoli7x1iliBU2mFcWoQnRYhGDJ...	key-0c4f92dfbe9fd5d3e
☐	prod-web-2-2-key	ed25519	2025/12/28 21:07 GMT+5	6e4aoli7x1iliBU2mFcWoQnRYhGDJ...	key-055b777b1eec8aa40
☐	MyED25519Key	ed25519	2025/12/19 02:07 GMT+5	Ufjz1XaeuwGX/xaq8tx4TfIXfuhfXU...	key-0b61b7daa18374387
☐	prod-web-1-1-key	ed25519	2025/12/28 21:07 GMT+5	6e4aoli7x1iliBU2mFcWoQnRYhGDJ...	key-0ada6b2cd6d29ebe0
☐	prod-web-3-key	ed25519	2025/12/28 20:29 GMT+5	6e4aoli7x1iliBU2mFcWoQnRYhGDJ...	key-00736505f1ac29b67
☐	prod-web-1-1	rsa	2025/12/21 23:03 GMT+5	06:50:b4:33:7d:b9:d9:21:0e:a0:35:1...	key-077ad991de07020c1
☐	prod-nginx-proxy-nginx	rsa	2025/12/21 23:03 GMT+5	06:50:b4:33:7d:b9:d9:21:0e:a0:35:1...	key-0f2fc9f25af1d3c20
☐	prod-web-2-2	rsa	2025/12/21 23:03 GMT+5	06:50:b4:33:7d:b9:d9:21:0e:a0:35:1...	key-0ed65276117fd9326
☐	lab12-new-key	rsa	2025/12/28 23:52 GMT+5	e7:37:92:70:8b:e6:e5:be:c0:18:43:b...	key-04044dd7e93284008
☐	prod-nginx-proxy-nginx-key	ed25519	2025/12/28 21:07 GMT+5	6e4aoli7x1iliBU2mFcWoQnRYhGDJ...	key-0fd7bb3c478f523b0
☐	assignment	ed25519	2025/12/29 01:11 GMT+5	CWN/f2/FDSBx9h8Lp8dsXgvTDu7h...	key-0f4019e1c79cad909
☐	prod-web-1-key	ed25519	2025/12/28 20:29 GMT+5	6e4aoli7x1iliBU2mFcWoQnRYhGDJ...	key-0155d6814d9785cc6

Verify Security Groups (SG) Created

- AWS Console:
 - EC2 → Security Groups → check rules:
 - Nginx SG: 22 (SSH) from your IP, 80/443 from anywhere
 - Backend SG: 22 from your IP, 80 from Nginx SG

sg-0251ea5e4ce796a4c - prod-backend-sg

Details

Security group name prod-backend-sg	Security group ID sg-0251ea5e4ce796a4c	Description Backend SG for web servers	VPC ID vpc-0514c61e0618fe16a
Owner 789925699467	Inbound rules count 2 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules (2)

Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sgr-034bb104ad79c9394	-	HTTP	TCP	80
-	sgr-0adb373e9218709f8	IPv4	SSH	TCP	22

Part 5: Nginx Configuration & Testing

1. Access the Nginx Server

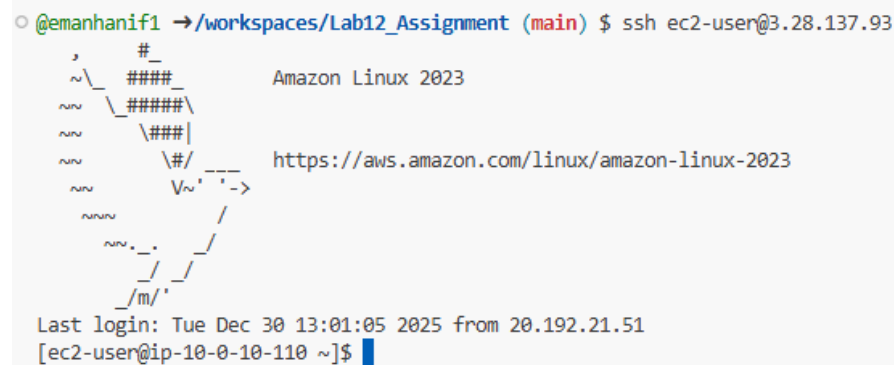
Connect to your Nginx instance using your terminal or PowerShell. Replace <nginx-public-ip> with your actual instance IP.

Bash

```
ssh ec2-user@<nginx-public-ip>
```

```
ssh ec2-user@3.28.137.93
```

Deliverable: Take a screenshot of your terminal after successful login. Save as assignment_part5_ssh_nginx.png.



```
@amanhanif1 →/workspaces/Lab12_Assignment (main) $ ssh ec2-user@3.28.137.93
#_
##### Amazon Linux 2023
#####\
#####|
\###|
\#/ https://aws.amazon.com/linux/amazon-linux-2023
V~' '->
/
/
/m/'
Last login: Tue Dec 30 13:01:05 2025 from 20.192.21.51
[ec2-user@ip-10-0-10-110 ~]$
```

assignment_part5_ssh_nginx.png.

This part of the assignment focuses on configuring Nginx as a **Load Balancer**. By updating the upstream block, you are telling Nginx how to distribute incoming traffic among your backend web servers.

The backup flag on your third server is particularly important—it ensures that traffic only goes to the third server if the first two are unavailable.

 [Step-by-Step Execution Guide](#)

2. Configure the Upstream Block

Open the configuration file. While the prompt suggests `/etc/nginx/nginx.conf`, sometimes load balancing blocks are kept in `/etc/nginx/conf.d/default.conf` depending on your installation.

Bash

```
sudo vim /etc/nginx/nginx.conf
```

Inside the `http` block, locate or add the upstream section. Use the **Private IPs** of your web servers:

```
        use_temp_path=off;

# Upstream backend servers
# PLACEHOLDER: Update these IPs after deployment
upstream backend_servers {
    # Primary servers (active load balancing)
    server 10.0.10.203:80;
    server 10.0.10.52:80;

    # Backup server (only used when primary servers are down)
    server 10.0.10.203:80 backup;
}
```

assignment_part5_nginx_conf_updated.png.

- Test Nginx configuration

```
[ec2-user@ip-10-0-10-110 ~]$ sudo nginx -t
nginx: [warn] the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:53
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-10-0-10-110 ~]$
```

assignment_part5_nginx_test.png.

restart Nginx service

```
[ec2-user@ip-10-0-10-110 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-110 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Tue 2025-12-30 13:41:37 UTC; 9s ago
     Process: 97298 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 97299 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 97300 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
  Main PID: 97301 (nginx)
    Tasks: 5 (limit: 1067)
   Memory: 4.5M
      CPU: 61ms
   CGroup: /system.slice/nginx.service
           └─97301 "nginx: master process /usr/sbin/nginx"
             └─97302 "nginx: worker process"
               └─97303 "nginx: worker process"
```

- Screenshot of the restart command and the "active" status. Save as assignment_part5_nginx_restart.png.

5.2 test Load Balancing

Test that Nginx is properly load balancing between web-1 and web-2.

Open Nginx URL

- Access the Nginx server using <https://<nginx-public-ip>>



Your connection is not private

Attackers might be trying to steal your information from **3.28.137.93** (for example, passwords, messages, or credit cards). [Learn more about this warning](#)

NET::ERR_CERT_AUTHORITY_INVALID

Turn on enhanced protection to get Chrome's highest level of security

Advanced

Back to safety

- Mention that a **self-signed certificate warning** will appear in the browser.

Accept Security Warning

- Click through the browser warning to proceed to the site.



Your connection isn't private

Attackers might be trying to steal your information from **158.252.77.173** (for example, passwords, messages, or credit cards).

NET::ERR_CERT_AUTHORITY_INVALID

Advanced

Go back

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

```
# Upstream backend servers
# PLACEHOLDER: Update these IPs after deployment
upstream backend_servers {
    # Primary servers (active load balancing)
    server 10.0.10.59:80;
    server 10.0.10.52:80;

    # Backup server (only used when primary servers are down)
    server 10.0.10.203:80 backup;
```

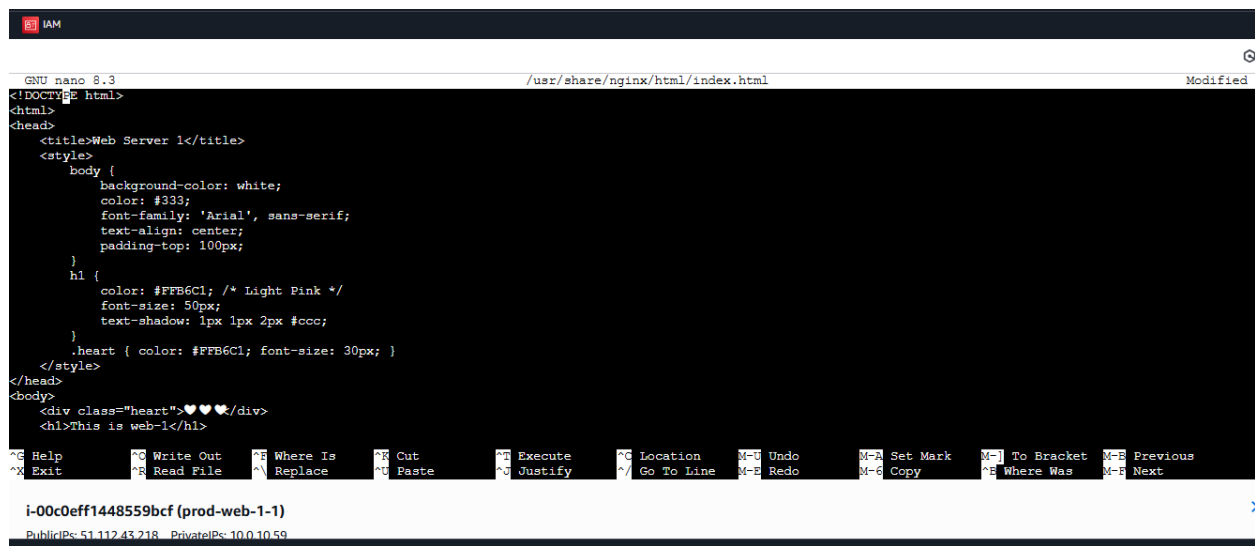
🔗 SSH into **web-1**.

```
ls /var/www/html
```

If that folder exists, use this command to edit your page:

```
Bash
```

```
sudo nano /var/www/html/index.html
```



```
GNU nano 8.3 /usr/share/nginx/html/index.html Modified
<!DOCTYPE html>
<html>
<head>
  <title>Web Server 1</title>
  <style>
    body {
      background-color: white;
      color: #333;
      font-family: 'Arial', sans-serif;
      text-align: center;
      padding-top: 100px;
    }
    h1 {
      color: #FFB6C1; /* Light Pink */
      font-size: 50px;
      text-shadow: 1px 1px 2px #ccc;
    }
    .heart { color: #FFB6C1; font-size: 30px; }
  </style>
</head>
<body>
  <div class="heart">♥♥♥♥</div>
  <h1>This is web-1</h1>
</body>
</html>
```

i-00c0eff1448559bcf (prod-web-1-1)

PublicIPs: 51.112.43.218 PrivateIPs: 10.0.10.59

SSH into web-2

```
ls /var/www/html
```

If that folder exists, use this command to edit your page:

Bash

```
sudo nano /var/www/html/index.html
```



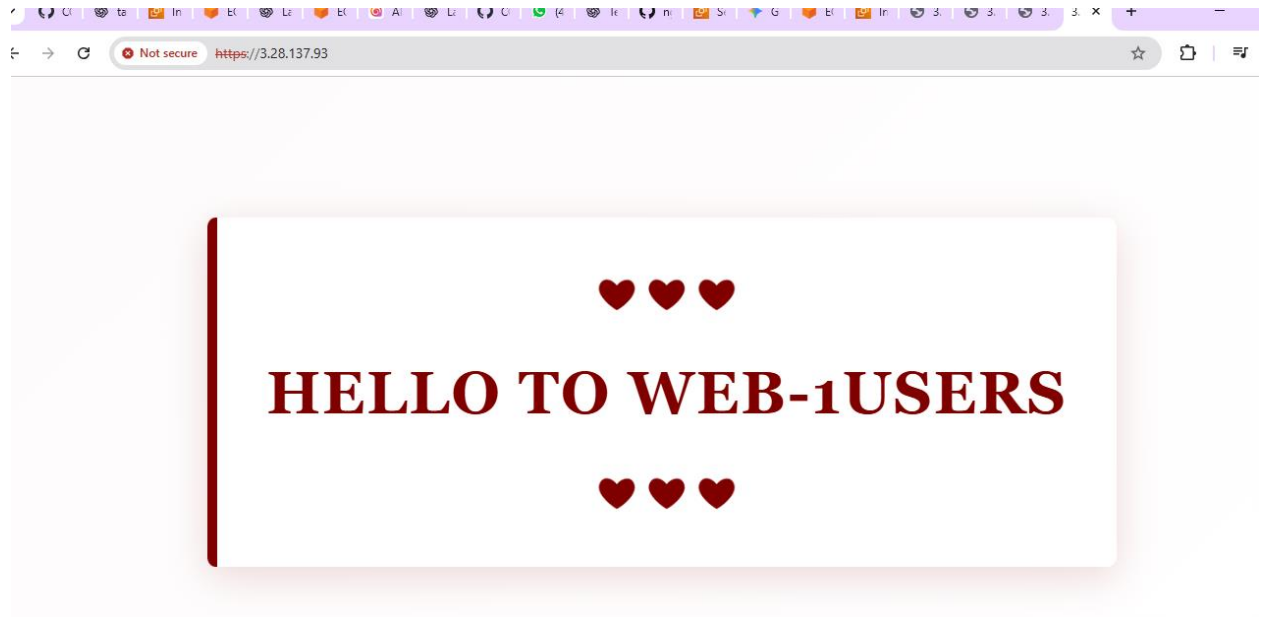
```
GNU nano 8.3 /var/www/html/index.html
<!DOCTYPE html>
<html>
<head>
  <style>
    body { background-color: white; text-align: center; padding-top: 100px; font-family: sans-serif; }
    h1 { color: #800000; font-size: 60px; } /* Maroon */
    .hearts { color: #800000; font-size: 40px; }
  </style>
</head>
<body>
  <div class="hearts">♥ ♥ ♥</div>
  <h1>This is web-2</h1>
  <div class="hearts">♥ ♥ ♥</div>
</body>
</html>
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

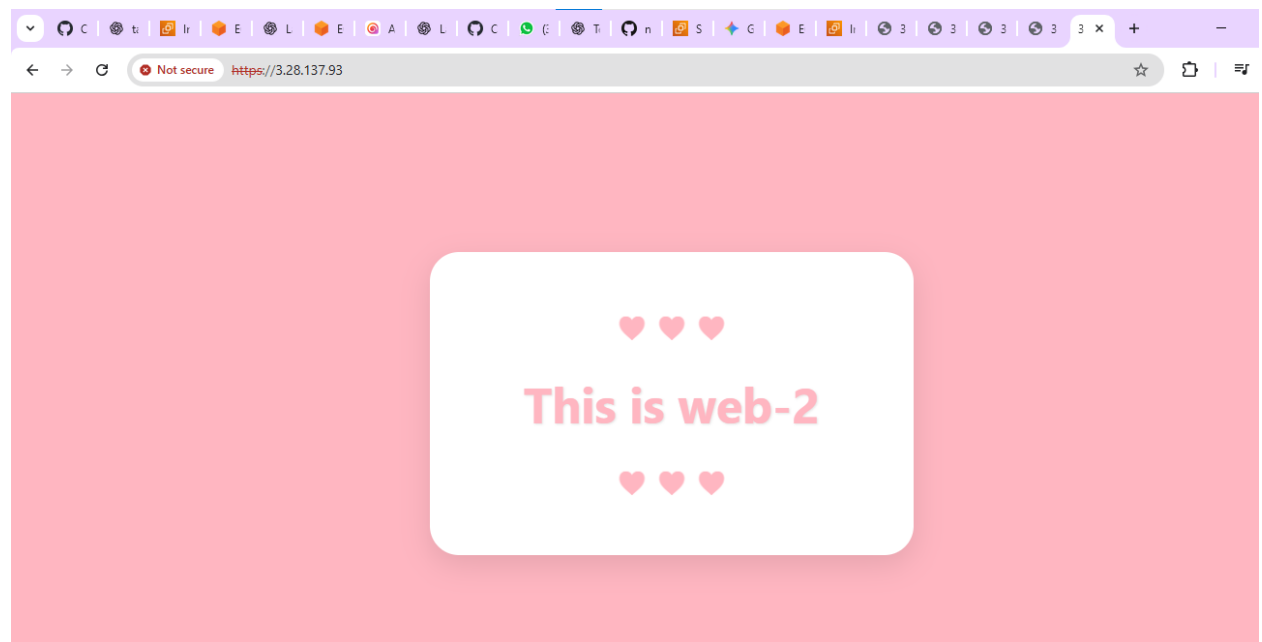
i-0105c8dd5a8070ae1 (prod-web-2-2)

```
[ec2-user@ip-10-0-10-110 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-110 ~]$ sudo systemctl restart nginx
```

Verify traffic alternates between web-1 and web-2



assignment_part5_web1_response.png



Web-2 response: assignment_part5_web2_response.png

Open your browser to <https://3.28.137.93>

Reload Page Multiple Times

- Refresh the page **at least 10 times** to test request distribution.
- Note: Each refresh should ideally go to a different backend server (web-1 or web-2).

Document Load Balancing Pattern

This showing the order of servers that responded to each refresh:

- Refresh 1 → web-1
- Refresh 2 → web-2
- Refresh 3 → web-1
- Refresh 4 → web-2
- ...

Verify Backup Server Is Idle

- Confirm that **web-3** (backup) does **not serve traffic** under normal conditions.

Summarize that **load balancing between web-1 and web-2 is working correctly** and backup server is only for failover.

5.3 Test Cache Functionality

Objective: To verify that the Nginx Reverse Proxy is correctly caching content from the backend servers to improve performance and reduce server load.

Cache directory Server-Side Cache Directory Inspection

Verified that Nginx is physically creating cache files on the disk in the designated directory.

- **Command Used:** `sudo ls -la /var/cache/nginx/`

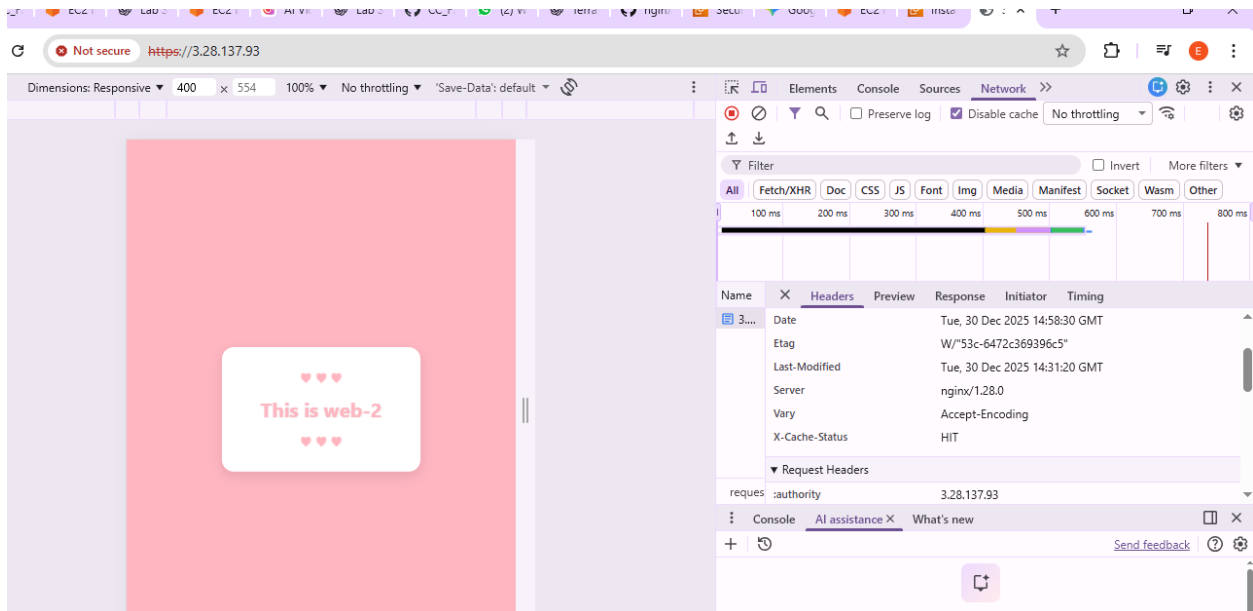
- **Observation:** The output showed multiple subdirectories (e.g., 2, 6, c, d), which are the hashed storage locations for cached objects.

The screenshot shows a VS Code interface with a terminal window. The terminal is connected to an EC2 instance via SSH. The user runs the command `sudo ls -la /var/cache/nginx/`. The output shows a directory listing of the nginx cache directory, including files like `nginx` and `nginx.128.0`. The interface also shows a file explorer on the left with a project structure including `.gitignore`, `terraform`, and `aws` directories. On the right, there is an 'Agent' sidebar with options like 'Generate Agent Instructions' and 'Build Workspace'.

```

# Primary servers (active load balancing)
server 10.0.10.177:80;
server 10.0.10.153:80;

@emanhanif1 →/workspaces/Lab12_Assignment (main) $ ssh -i ~/.ssh/terraform_key ec2-user@3.28.137.93
[ec2-user@ip-10-0-10-110 ~]$ sudo ls -la /var/cache/nginx/
total 0
drwx----- 11 nginx nginx 87 Dec 30 13:58 .
drwxr-xr-x  9 root  root 101 Dec 29 00:00 ..
drwx----- 3 nginx nginx 16 Dec 29 13:07 2
drwx----- 3 nginx nginx 16 Dec 29 00:07 3
drwx----- 4 nginx nginx 26 Dec 30 13:31 6
drwx----- 3 nginx nginx 16 Dec 30 13:31 7
drwx----- 3 nginx nginx 16 Dec 29 01:53 8
drwx----- 3 nginx nginx 16 Dec 29 16:11 9
drwx----- 4 nginx nginx 26 Dec 30 14:19 c
drwx----- 3 nginx nginx 16 Dec 30 13:58 d
drwx----- 3 nginx nginx 16 Dec 30 00:42 f
[ec2-user@ip-10-0-10-110 ~]$
  
```



Verify cache directory

```

[ec2-user@ip-10-0-10-110 ~]$ sudo ls -la /var/cache/nginx/
total 0
drwx-----. 11 nginx nginx 87 Dec 30 13:58 .
drwxr-xr-x. 9 root root 101 Dec 29 00:00 ..
drwx-----. 3 nginx nginx 16 Dec 29 13:07 2
drwx-----. 3 nginx nginx 16 Dec 29 00:07 3
drwx-----. 4 nginx nginx 26 Dec 30 13:31 6
drwx-----. 3 nginx nginx 16 Dec 30 13:31 7
drwx-----. 4 nginx nginx 26 Dec 30 14:59 8
drwx-----. 3 nginx nginx 16 Dec 29 16:11 9
drwx-----. 4 nginx nginx 26 Dec 30 14:19 c
drwx-----. 3 nginx nginx 16 Dec 30 13:58 d
drwx-----. 3 nginx nginx 16 Dec 30 00:42 f
[ec2-user@ip-10-0-10-110 ~]$

```

verification of Cache MISS (Initial Request)

To test the initial state, the Nginx cache was cleared and a request was sent using the curl command (or browser) to trigger the first fetch from the backend.

- **Action:** Executed `curl -I -k https://localhost` for the first time.
- **Observation:** The response header `X-Cache-Status` showed **MISS**, indicating Nginx fetched the data from the backend and stored it in the cache.

```

[ec2-user@ip-10-0-10-110 ~]$ curl -I -k https://localhost
HTTP/2 200
server: nginx/1.28.0
date: Tue, 30 Dec 2025 15:05:17 GMT
content-type: text/html; charset=UTF-8
content-length: 1148
vary: Accept-Encoding
last-modified: Tue, 30 Dec 2025 14:27:03 GMT
etag: "47c-6472c274f10f0"
x-cache-status: MISS
accept-ranges: bytes
[ec2-user@ip-10-0-10-110 ~]$

```

assignment_part5_cache_miss.png

Verification of Cache HIT (Subsequent Request)

The same request was repeated to ensure Nginx serves the content from its local cache rather than contacting the backend again.

Executed the curl command a second time.

The response header X-Cache-Status updated to **HIT**, confirming the caching mechanism is operational.

```
@emanhanif1 → /workspaces/Lab12_Assignment (main) $ ssh -i ~/.ssh/terraform_key ec2-user@3.28.137.93
server: nginx/1.28.0
date: Tue, 30 Dec 2025 15:05:17 GMT
content-type: text/html; charset=UTF-8
content-length: 1148
vary: Accept-Encoding
last-modified: Tue, 30 Dec 2025 14:27:03 GMT
etag: "47c-6472c274f10f0"
x-cache-status: MISS
accept-ranges: bytes

[ec2-user@ip-10-0-10-110 ~]$ curl -I -k https://localhost
HTTP/2 200
server: nginx/1.28.0
date: Tue, 30 Dec 2025 15:06:04 GMT
content-type: text/html; charset=UTF-8
content-length: 1148
vary: Accept-Encoding
last-modified: Tue, 30 Dec 2025 14:27:03 GMT
etag: "47c-6472c274f10f0"
x-cache-status: HIT
accept-ranges: bytes

[ec2-user@ip-10-0-10-110 ~]$
```

assignment_part5_cache_hit.png

Step 4: Access Log Analysis

Checked the Nginx access logs to verify that the cache status is being correctly logged for every incoming request.

- **Command**

```
sudo grep "Cache:" /var/log/nginx/access.log | tail -5
```

```
[ec2-user@ip-10-0-10-110 ~]$ sudo grep "Cache:" /var/log/nginx/access.log | tail -5
119.154.15.243 - - [30/Dec/2025:15:02:33 +0000] "GET / HTTP/2.0" 200 580 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: HIT
119.154.15.243 - - [30/Dec/2025:15:02:46 +0000] "GET / HTTP/2.0" 200 580 "-" "Mozilla/5.0 (Linux; Android 6.0; Nexus 5 Build/MRA58N) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Mobile Safari/537.36" "-" Cache: HIT
130.12.180.18 - - [30/Dec/2025:15:03:16 +0000] "GET / HTTP/1.1" 301 169 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:76.0) Gecko/20100101 Firefox/76.0" "-" Cache: -
127.0.0.1 - - [30/Dec/2025:15:05:17 +0000] "HEAD / HTTP/2.0" 200 0 "-" "curl/8.11.1" "-" Cache: MISS
127.0.0.1 - - [30/Dec/2025:15:06:04 +0000] "HEAD / HTTP/2.0" 200 0 "-" "curl/8.11.1" "-" Cache: HIT
[ec2-user@ip-10-0-10-110 ~]$
```

assignment_part5_access_log_cache

The logs clearly show the transition from MISS (or BYPASS) to HIT for the requested resources, along with timestamps and client IP addresses.

5.4 Test High Availability (Backup Server)

Test the backup server functionality by simulating primary server failure.

Simulating Primary Server 1 Failure

Primary servers (web-1 and web-2) are the active nodes. We start by stopping the service on the first node.

- SSH into web-1 and stopped the Apache service
- **Verification:** Checked service status with `sudo systemctl status httpd` to confirm it is "inactive (dead)".

```
[ec2-user@ip-10-0-10-59 ~]$ sudo systemctl stop httpd
[ec2-user@ip-10-0-10-59 ~]$
```

i-00c0eff1448559bcf (prod-web-1-1)

PublicIPs: 51.112.43.218 PrivateIPs: 10.0.10.59

run

sudo systemctl stop httpd.

```
o httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: inactive (dead) since Tue 2025-12-30 15:12:31 UTC; 13min ago
     Duration: 1d 16h 8min 50.696s
    Docs: man:httpd.service(8).
   Process: 27035 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
  Main PID: 27035 (code=exited, status=0/SUCCESS)
   Status: "Total requests: 597; Idle/Busy workers 100/0; Requests/sec: 0.00413; Bytes served/sec: 3 B/sec"
      CPU: 2min 36.477s
```

- **Verification:** Refresh your browser. You should now only see content from **web-2**.



Failover to web-3 (The Backup)

1. **Stop web-2:** SSH into **web-2**

run `sudo systemctl stop httpd`.

```
[ec2-user@ip-10-0-10-52 ~]$ sudo systemctl stop httpd
[ec2-user@ip-10-0-10-52 ~]$
```

i-0105c8dd5a8070ae1 (prod-web-2-2)

PublicIPs: 51.112.186.36 PrivateIPs: 10.0.10.52

Simulating Primary Server 2 Failure

With web-1 down, Nginx should still serve traffic from web-2. To trigger the backup, we must stop web-2 as well.

SSH into web-2 and stopped the Apache service

Take a screenshot of the web-2 terminal showing active: inactive

```
ec2-user@ip-10-0-10-52 ~]$ sudo systemctl status httpd
httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: inactive (dead) since Tue 2025-12-30 15:14:30 UTC; 10min ago
     Duration: 1d 16h 8min 57.248s
    Docs: man:httpd.service(8)
   Process: 27056 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
  Main PID: 27056 (code=exited, status=0/SUCCESS)
   Status: "Total requests: 598; Idle/Busy workers 100/0; Requests/sec: 0.00414; Bytes served/sec: 3 B/sec"
      CPU: 2min 5.896s

ec 28 23:05:31 ip-10-0-10-52.me-central-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server
ec 28 23:05:32 ip-10-0-10-52.me-central-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server
ec 28 23:05:32 ip-10-0-10-52.me-central-1.compute.internal httpd[27056]: Server configured, listening on: port 80
```

Save and Restart: After commenting the lines of cache settings, save the file and restart Nginx so the changes take effect:

```
Bash
sudo nginx -t
sudo systemctl restart nginx
```

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-10-0-10-110 ~]$ sudo systemctl restart nginx
```

Ensure Apache is running on **web-3** and the file exists:

```

mod_lua-2.4.65-1.amzn2023.0.2.x86_64
Complete!
[ec2-user@ip-10-0-10-203 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-10-203 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-10-0-10-203 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Tue 2025-12-30 15:21:58 UTC; 7min ago
     Docs: man:httpd.service(8)
  Main PID: 82003 (httpd)
    Status: "Total requests: 2; Idle/Busy workers 100/0;Requests/sec: 0.00477; Bytes served/sec: 3 B/sec"
    Tasks: 177 (limit: 1067)
   Memory: 13.6M
      CPU: 406ms
   CGroup: /system.slice/httpd.service
           └─82003 /usr/sbin/httpd -DFOREGROUND
             └─82146 /usr/sbin/httpd -DFOREGROUND
               └─82150 /usr/sbin/httpd -DFOREGROUND
                 └─82151 /usr/sbin/httpd -DFOREGROUND
                   └─82152 /usr/sbin/httpd -DFOREGROUND

Dec 30 15:21:58 ip-10-0-10-203.me-central-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Dec 30 15:21:58 ip-10-0-10-203.me-central-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Dec 30 15:21:58 ip-10-0-10-203.me-central-1.compute.internal httpd[82003]: Server configured, listening on: port 80
[ec2-user@ip-10-0-10-203 ~]$

```

i-0d2a690e1f3347e2a (prod-web-3-3)

PublicIPs: 40.172.215.170 PrivateIPs: 10.0.10.203

Nginx Error Log Verification

Nginx logs the connection failures to the primary servers, which helps in identifying why the failover occurred.

- **Command**

```
sudo tail -f /var/log/nginx/error.log
```

- **Observation:** The logs show "no live upstreams" or "connection refused" errors for the primary IPs justifying the switch to backup.

```
[ec2-user@ip-10-0-10-110 ~]$ sudo tail -n 20 /var/log/nginx/error.log
2025/12/30 15:42:05 [notice] 101118#101118: nginx/1.28.0
2025/12/30 15:42:05 [notice] 101118#101118: OS: Linux 6.1.158-180.294.amzn2023.x86_64
2025/12/30 15:42:05 [notice] 101118#101118: getrlimit(RLIMIT_NOFILE): 65535:65535
2025/12/30 15:42:05 [notice] 101120#101120: start worker processes
2025/12/30 15:42:05 [notice] 101120#101120: start worker process 101121
2025/12/30 15:42:05 [notice] 101120#101120: start worker process 101122
2025/12/30 15:42:05 [notice] 101120#101120: start cache manager process 101123
2025/12/30 15:42:05 [notice] 101120#101120: start cache loader process 101124
2025/12/30 15:42:42 [error] 101122#101122: *4 connect() failed (111: Connection refused) while connecting to upstream, client: 119.154.15.243, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.59:80/", host: "3.28.137.93"
2025/12/30 15:42:42 [warn] 101122#101122: *4 upstream server temporarily disabled while connecting to upstream, client: 119.154.15.243, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.59:80/", host: "3.28.137.93"
2025/12/30 15:42:42 [error] 101122#101122: *4 connect() failed (111: Connection refused) while connecting to upstream, client: 119.154.15.243, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.52:80/", host: "3.28.137.93"
2025/12/30 15:42:42 [warn] 101122#101122: *4 upstream server temporarily disabled while connecting to upstream, client: 119.154.15.243, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.52:80/", host: "3.28.137.93"
```

assignment_part5_nginx_error_log

Activation of Backup Server (web-3)

Once all primary servers in the upstream group are down, Nginx routes traffic to the server marked with the backup parameter.

- Reloaded the Nginx public IP in the browser multiple times.
- The page now displays content from web-3, confirming the backup server has successfully taken over the load.

← → ↻ Not secure https://3.28.137.93

This is web-3

```
[ec2-user@ip-10-0-10-59 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-10-59 ~]$
```

i-00c0eff1448559bcf (prod-web-1-1)

PublicIPs: 51.112.43.218 PrivateIPs: 10.0.10.59

```
[ec2-user@ip-10-0-10-52 ~]$ ^C
[ec2-user@ip-10-0-10-52 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-10-52 ~]$
```

i-0105c8dd5a8070ae1 (prod-web-2-2)

PublicIPs: 51.112.186.36 PrivateIPs: 10.0.10.52

🔗 **Restart Primaries:** On both **web-1** and **web-2**, run:

Bash
sudo systemctl start httpd

🔗 **Verify Recovery:** Refresh your browser (non-incognito is fine now). Traffic should return to web-1 or web-2.



assignment_part5_services_restored

5.5 Security & Performance Analysis

Analyze the security headers and performance of your Nginx setup.

. SSL/TLS Certificate Details

This verifies that your self-signed certificate is correctly installed on Nginx.

Command

```
# On Nginx server - view certificate
```

```
sudo openssl x509 -in /etc/ssl/certs/selfsigned.crt -text -noout
```

```
[ec2-user@ip-10-0-10-110 ~]$ sudo openssl x509 -in /etc/ssl/certs/selfsigned.crt -text -noout
```

Certificate:

Data:

Version: 3 (0x2)

Serial Number:

0d:10:f4:25:81:8a:7d:61:c9:35:f7:2e:47:a8:6c:10:25:95:e2:99

Signature Algorithm: sha256WithRSAEncryption

Issuer: CN=3.28.137.93

Validity

Not Before: Dec 30 13:23:53 2025 GMT

Not After : Dec 30 13:23:53 2026 GMT

Subject: CN=3.28.137.93

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

Public-Key: (2048 bit)

Modulus:

00:a3:47:d9:e4:9a:7f:a4:c4:2d:b6:9c:51:6d:f9:
c6:59:04:7d:9b:70:e2:13:8e:19:b9:8b:d7:c8:79:
82:f2:55:ea:47:21:3a:82:c0:e8:db:88:87:17:4a:
c6:0e:d1:1d:20:a0:31:39:2a:6f:5f:25:95:de:b8:
20:54:2e:57:d5:8b:e7:97:94:a8:6f:ea:71:69:84:
58:20:cf:88:38:c0:75:d7:54:c5:e5:ea:a9:16:6f:
50:7a:84:75:da:ed:6f:07:3f:8b:e5:9c:7b:b3:78:
22:59:b8:88:82:d7:df:c8:cf:ea:ae:9f:04:94:27:

main* 0 0 0

Ln 169, Col 1 Spaces: 4 UTF-8 LF { } Shell Script

```
CA:FALSE
X509v3 Key Usage:
    Digital Signature, Key Encipherment
X509v3 Extended Key Usage:
    TLS Web Server Authentication
Signature Algorithm: sha256WithRSAEncryption
Signature Value:
    8e:f1:ee:9b:71:04:5c:37:85:0f:83:0e:7f:ce:21:f8:7a:30:
    a8:76:7f:a3:dd:ab:fd:c0:86:6b:12:6a:35:c9:af:2b:6f:b6:
    e0:c6:e8:01:af:ae:a2:5f:71:60:6b:ea:e9:b6:ed:cd:75:30:
    7c:6f:d4:f7:92:93:f9:de:1e:45:ae:41:c5:ef:b1:b9:0a:df:
    11:57:82:58:bb:bd:11:83:7c:90:af:5c:66:a4:1f:0d:df:e5:
    c2:a0:d1:2d:95:0e:d8:19:a3:ea:a9:1c:f1:cb:62:f0:20:c9:
    64:0e:ad:7e:46:d1:32:5e:34:73:b0:69:90:37:07:48:82:26:
    da:9d:86:75:74:44:37:3c:e9:66:a3:88:90:44:af:83:00:52:
    a5:e6:6a:ea:3c:12:5b:70:c2:c9:c1:7e:ed:6d:09:ba:62:1f:
    1f:45:7c:75:f0:83:14:ad:50:25:49:cf:4e:64:84:d2:fa:05:
    b5:55:69:5a:aa:b2:cd:ab:bb:b4:46:50:86:9b:3a:bf:28:6e:
    8b:20:7b:71:67:00:cb:d0:3c:df:9c:a8:1c:f5:bf:be:b1:31:
    59:4e:72:a9:8f:96:11:c0:d3:99:71:96:4f:93:c5:ad:f5:16:
    b3:9a:4e:f0:39:c5:c7:3b:89:07:11:89:43:45:3f:ec:2e:6e:
    47:22:8f:6b
```

```
[ec2-user@ip-10-0-10-110 ~]$
```

assignment_part5_ssl_certificate

- **Issuer & Subject match:** CN=3.28.137.93. Since the Issuer and Subject are the same, it confirms this is a **self-signed** certificate.
- **Validity Period:** It is valid from **Dec 30, 2025** to **Dec 30, 2026**. This shows the certificate is currently "active" and hasn't expired.
- **Encryption Strength:** Public-Key: (2048 bit) using sha256WithRSAEncryption. This is the modern standard for secure web traffic.
- **Subject Alternative Name (SAN):** IP Address:3.28.137.93. This is a crucial detail! Modern browsers (like Chrome) often reject certificates that don't have the IP or Domain listed in the SAN field. Yours is correctly configured.

Check certificate

- `openssl s_client -connect <nginx-ip>:443 -showcerts`


```

SRP username: None
TLS session ticket lifetime hint: 300 (seconds)
TLS session ticket:
0000 - 81 b0 80 c5 1f 5b f1 8d-c8 84 69 ae 63 b5 38 13 .....[....i.c.8.
0010 - 03 79 90 06 33 be f6 7f-4c 97 12 21 3f ed 2e 8c .y..3...L...!?...
0020 - e9 49 d9 28 4e db 70 04-22 b0 c5 46 99 a6 04 41 .I.(N.p."...F...A
0030 - 15 b1 59 4a b3 56 59 6f-46 a5 f1 55 33 72 84 b3 ..YJ.VYoF..U3r..
0040 - 1d 4c 0f 75 5a c5 fe 87-87 ec a9 d7 c4 2b 28 b3 .L.uZ.....+(.
0050 - e8 ff fc f5 41 11 e7 d6-5d 08 38 2f 10 7d d9 41 ....A...].8/..}.A
0060 - ab 91 56 86 94 2d 62 7d-3b 80 09 91 4f c2 9f 75 ..V...-b};...0...u
0070 - 23 06 ff 05 b2 b4 e0 52-8d 99 c6 10 39 5a 5c 0a #.....R.....9Z\
0080 - 2d 41 80 70 0f ce 09 3b-28 b2 6e 98 64 c0 72 85 -A.p...;(.n.d.r.
0090 - 09 a2 48 1a c6 81 28 ba-22 55 23 d7 b5 e3 a6 47 ..H...("U#....G
00a0 - 0c 3c bb f4 df 65 c4 5e-8a ef 19 ed 1b 2d 2b 1f .<...e.^.....-+.
00b0 - 1a 3a ee c1 c5 ee 7a 96-f9 ef 0f 05 ca 82 66 02 .:....z.....f.
00c0 - 7a e1 71 16 f1 ef 8b 49-e9 94 2c 03 c6 19 e4 a7 z.q....I.,.....
00d0 - c3 17 2d 83 9f 76 d5 1c-93 93 9f 2d 2e 72 d5 00 ...-v.....-r..

Start Time: 1767115728
Timeout : 7200 (sec)
Verify return code: 18 (self-signed certificate)
Extended master secret: no
Max Early Data: 0
---
read R BLOCK
closed
[ec2-user@ip-10-0-10-110 ~]$

```

```

# Test HTTP redirect
curl -I http://<nginx-public-ip>

```

Command: curl -I <http://3.28.137.93>

The response shows an **HTTP/1.1 301 Moved Permanently** status with the Location header pointing to the HTTPS version of the site.

```

[ec2-user@ip-10-0-10-110 ~]$ curl -I http://3.28.137.93
HTTP/1.1 301 Moved Permanently
Server: nginx/1.28.0
Date: Tue, 30 Dec 2025 17:17:40 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://3.28.137.93/

[ec2-user@ip-10-0-10-110 ~]$

```

assignment_part5_http_redirect

```
# Monitor error logs
sudo tail -50 /var/log/nginx/error.log
```

```
Last login: Tue Dec 30 17:16:41 2025 from 4.240.18.225
[ec2-user@ip-10-0-10-110 ~]$ sudo tail -50 /var/log/nginx/error.log
2025/12/30 17:19:32 [warn] 103819#103819: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:53
2025/12/30 17:19:32 [notice] 103819#103819: signal process started
2025/12/30 17:19:32 [notice] 101120#101120: signal 1 (SIGHUP) received from 103819, reconfiguring
2025/12/30 17:19:32 [notice] 101120#101120: reconfiguring
2025/12/30 17:19:32 [warn] 101120#101120: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:53
2025/12/30 17:19:32 [notice] 101120#101120: using the "epoll" event method
2025/12/30 17:19:32 [notice] 101120#101120: start worker processes
2025/12/30 17:19:32 [notice] 101120#101120: start worker process 103830
2025/12/30 17:19:32 [notice] 101120#101120: start worker process 103831
```

assignment_part5_error_log_analysis.png

Monitor access logs

The log was reviewed to ensure no critical worker process errors or upstream connection timeouts were occurring during normal operation.

```
sudo tail -50 /var/log/nginx/access.log
```

```

emanhanif1 →/workspaces/Lab12_Assignment (main) $ ssh ec2-user@3.28.137.93
dows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/106.0.0.0 Safari/537.36" "-"
3.129.169.161 - - [30/Dec/2025:15:58:09 +0000] "HEAD /Core/Skin/Login.aspx HTTP/1.1" 404 0 "http://3.
0/Core/Skin/Login.aspx" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Ge
/106.0.0.0 Safari/537.36" "-" Cache: -
70.64.233.196 - - [30/Dec/2025:16:16:51 +0000] "GET /.git/config HTTP/1.1" 301 169 "-" "Mozilla/5.0 (
x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/81.0.4044.129 Safari/537.36" "-" Cache: -
70.64.233.196 - - [30/Dec/2025:16:16:52 +0000] "GET /.git/config HTTP/1.1" 404 183 "-" "Mozilla/5.0 (
x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/81.0.4044.129 Safari/537.36" "-" Cache: -
41.98.11.140 - - [30/Dec/2025:16:20:54 +0000] "GET / HTTP/1.1" 301 169 "-" "-" "-" Cache: -
85.241.208.170 - - [30/Dec/2025:16:25:27 +0000] "GET /.git/config HTTP/1.1" 301 169 "-" "Mozilla/5.0
i86_64)) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/44.0.2403.130 Safari/537.36" "-" Cache:
70.64.233.196 - - [30/Dec/2025:16:45:56 +0000] "GET /.git/config HTTP/1.1" 301 169 "-" "Mozilla/5.0 (
x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/81.0.4044.129 Safari/537.36" "-" Cache: -
70.64.233.196 - - [30/Dec/2025:16:45:57 +0000] "GET /.git/config HTTP/1.1" 404 183 "-" "Mozilla/5.0 (
x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/81.0.4044.129 Safari/537.36" "-" Cache: -
.240.18.225 - - [30/Dec/2025:17:15:22 +0000] "HEAD / HTTP/2.0" 200 0 "-" "curl/8.5.0" "-" Cache: -
.28.137.93 - - [30/Dec/2025:17:17:40 +0000] "HEAD / HTTP/1.1" 301 0 "-" "curl/8.11.1" "-" Cache: -
.28.137.93 - - [30/Dec/2025:17:18:55 +0000] "HEAD / HTTP/2.0" 200 0 "-" "curl/8.11.1" "-" Cache: -
.28.137.93 - - [30/Dec/2025:17:19:39 +0000] "HEAD / HTTP/2.0" 200 0 "-" "curl/8.11.1" "-" Cache: -
.240.18.225 - - [30/Dec/2025:17:22:34 +0000] "HEAD / HTTP/2.0" 200 0 "-" "curl/8.5.0" "-" Cache: -
9.124.40.174 - - [30/Dec/2025:17:24:03 +0000] "GET /?XDEBUG_SESSION_START=phpstorm HTTP/1.1" 301 169
a/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/78.0.3904.108 Safar
-" Cache: -
9.124.40.174 - - [30/Dec/2025:17:24:04 +0000] "GET /?XDEBUG_SESSION_START=phpstorm HTTP/1.1" 200 541
8.137.93:80/?XDEBUG_SESSION_START=phpstorm" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/5
. like Gecko) Chrome/78.0.3904.108 Safari/537.36" "-" Cache: -

```

assignment_part5_access_log_analysis.png

Worker Process Performance

Analyzing traffic patterns and ensuring Nginx is utilizing system resources correctly.

- **Process Command:** `ps aux | grep nginx`
- Access logs show successful 200 OK responses with cache statuses, and the `ps` command confirms the master and worker processes are running under the `nginx` user.

Check Nginx worker processes

`ps aux | grep nginx`

```
[ec2-user@ip-10-0-10-110 ~]$ ps aux | grep nginx
root      101120  0.0  0.8 27572 7588 ?        Ss   15:42   0:00 nginx: master process /usr/sbin/nginx
nginx     103952  0.0  0.7 27988 6740 ?        S    17:22   0:00 nginx: worker process
nginx     103953  0.0  0.7 27728 6740 ?        S    17:22   0:00 nginx: worker process
nginx     103954  0.0  0.4 27576 3788 ?        S    17:22   0:00 nginx: cache manager process
ec2-user  104198  0.0  0.2 222328 2116 pts/2    S+   17:28   0:00 grep --color=auto nginx
[ec2-user@ip-10-0-10-110 ~]$
```

Bonus task1

Create Custom HTML Error Pages

- Created a **custom 404 error page** to display a user-friendly message when a page is not found
- Improves **user experience** and provides clear navigation guidance
- Configured in the web server (Nginx) to replace default error pages

404

```
GNU nano 8.3 /usr/share/nginx/html/custom_errors/404.html
<!DOCTYPE html>
<html>
<head>
  <title>Page Not Found</title>
  <style>
    body { text-align: center; font-family: sans-serif; padding: 100px; background-color: #f4f4f4; }
    h1 { color: #e74c3c; font-size: 50px; }
    p { font-size: 20px; color: #666; }
  </style>
</head>
<body>
  <h1>404</h1>
  <p>Oops! The page you are looking for disappeared into the cloud.</p>
  <p><a href="/">Return Home</a></p>
</body>
</html>
```

```
GNU nano 8.3 /usr/share/nginx/html/custom_errors/502.html
<!DOCTYPE html>
<html>
<head>
  <title>Connection Error</title>
  <style>
    body { text-align: center; font-family: sans-serif; padding: 100px; background-color: #fff5f5; }
    h1 { color: #c0392b; font-size: 50px; }
    p { font-size: 20px; color: #333; }
  </style>
</head>
<body>
  <h1>502 - Bad Gateway</h1>
  <p>The Load Balancer cannot reach the backend web servers right now.</p>
  <p><i>Server status: Offline/Rebooting</i></p>
</body>
</html>
```

```

.heart {
  position: absolute;
  color: #ff4d94;
  font-size: 20px;
  animation: float 4s infinite ease-in-out;
}
@keyframes float {
  0% { transform: translateY(0) rotate(0deg); opacity: 1; }
  100% { transform: translateY(-100px) rotate(20deg); opacity: 0; }
}
</style>
</head>
<body>
  <div class="container">
    <div class="heart" style="top: -20px; left: 10px;">❤️</div>
    <div class="heart" style="top: -50px; right: 20px; animation-delay: 1s;">❤️</div>
    <div class="heart" style="top: 10px; right: -30px; animation-delay: 2s;">❤️</div>

    <div class="cloud">
      <h2 style="color: #ff4d94; margin: 0; z-index: 1;">Oops!</h2>
    </div>

    <h1>Lost in the Cloud Gateways</h1>
    <p>We can't find the page you're looking for.</p>
  </div>

```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ssh + - [ ] [ ] ... [ ] [ ] X
GNU nano 8.3 /usr/share/nginx/html/custom_errors/503.html Modified
<!DOCTYPE html>
<html>
<head>
  <title>Maintenance</title>
  <style>
    body { text-align: center; font-family: sans-serif; padding: 100px; background-color: #ffffbe6; }
    h1 { color: #d4a017; font-size: 50px; }
    p { font-size: 20px; color: #333; }
  </style>
</head>
<body>
  <h1>503 - Service Unavailable</h1>
  <p>We are currently performing scheduled maintenance on Web1, Web2, and Web3.</p>
  <p>Please check back in a few minutes.</p>
</body>
</html>
```

```
GNU nano 8.3 /etc/nginx/nginx.conf
# HTTP Server (redirect to HTTPS)
server {
    listen 80;
    server_name _;
    # 1. Add this line to allow Nginx to show its own pages
    proxy_intercept_errors on;
    error_page 404 /404.html;
    location = /404.html {
        root /usr/share/nginx/html/custom_errors;
        internal;
    }
# 3. Add the 502/503 mapping
    error_page 502 /502.html;
    error_page 503 /503.html;
    location ~ /(502|503).html {
        root /usr/share/nginx/html/custom_errors;
        internal;
    }
    location / {
        return 301 https://$host$request_uri;
    }
}
```

Configure Nginx to Use Custom Pages

Before you restart Nginx, run this command to make 100% sure the files are exactly where Nginx expects them:

Bash



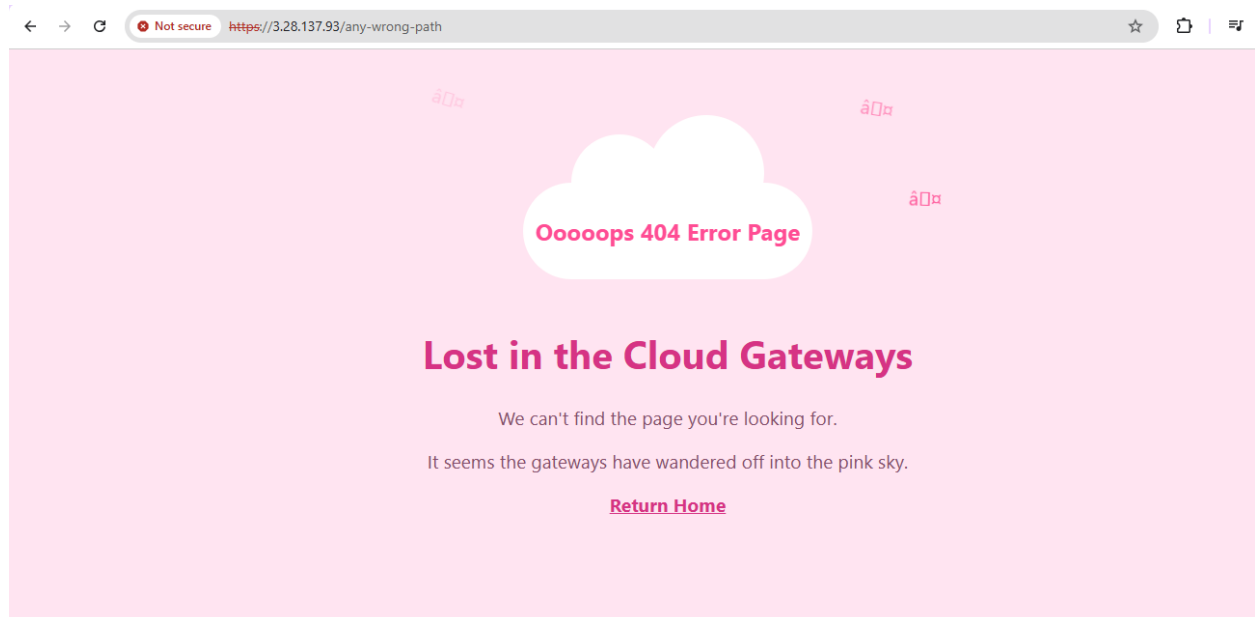
```
ls -l /usr/share/nginx/html/custom_errors/
```

If you see these three files in the list, you are perfect:

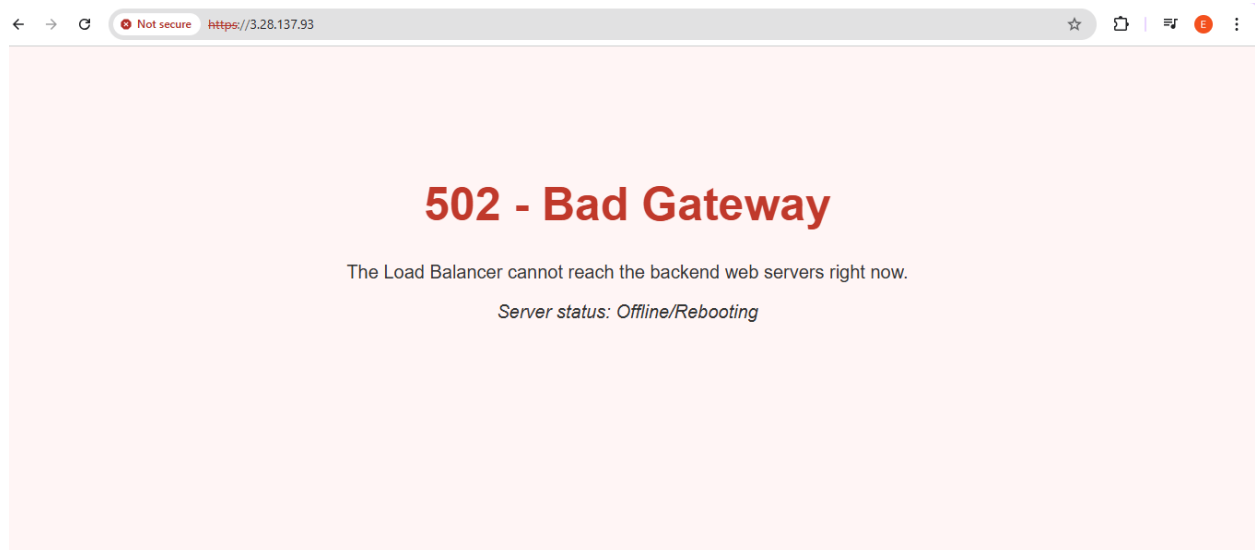
```
[ec2-user@ip-10-0-10-110 ~]$ ls -l /usr/share/nginx/html/custom_errors/
total 12
-rw-r--r--. 1 root root 447 Dec 30 17:42 404.html
-rw-r--r--. 1 root root 478 Dec 30 17:42 502.html
-rw-r--r--. 1 root root 486 Dec 30 17:43 503.html
```

Restart Nginx:

```
[ec2-user@ip-10-0-10-110 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-110 ~]$
```



- **For 404:** Open your browser and type a URL that definitely doesn't exist.
 - *Screenshot:* bonus1_custom_404.png



🔗 **For the 502 screenshot:** Since we have a redirect to HTTPS, if we use curl to test it, make sure to use curl -k (to ignore that self-signed certificate error) so WE can actually see custom error page content.

Bonus 2: Implement Rate Limiting

Add rate limiting to prevent abuse.

Configuration Change

Open your Nginx configuration file (/etc/nginx/nginx.conf) and add the limit_req_zone in the **http** block and the limit_req directive inside your **server location** block.

In the http block:

Nginx

```
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=1r/s;
```




bonus2_rate_limit_config

Apply changes:

Test configuration

```
[ec2-user@ip-10-0-10-59 ~]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Reload Nginx

Testing with Rapid Requests

To trigger the **429 Too Many Requests** error, you need to send requests faster than the allowed rate. You can do this easily from your terminal using a for loop.

Run this command in your terminal:

```
@emanhanif1 → /workspaces/Lab12_Assignment (main) $ for i in {1..10}; do curl -k -I https://3.28.137
content-type: text/html
content-length: 169

HTTP/2 429
server: nginx/1.28.0
date: Tue, 30 Dec 2025 18:42:48 GMT
content-type: text/html
content-length: 169

HTTP/2 502
server: nginx/1.28.0
date: Tue, 30 Dec 2025 18:42:48 GMT
content-type: text/html
content-length: 478
etag: "69540f10-1de"
strict-transport-security: max-age=31536000; includeSubDomains
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
x-xss-protection: 1; mode=block

HTTP/2 429
server: nginx/1.28.0
date: Tue, 30 Dec 2025 18:42:48 GMT
content-type: text/html
content-length: 169
```

bonus2_rate_limit_test

Bonus 3: Health Check Automation (Report Content)

Objective: To implement a proactive monitoring solution that automatically checks the availability of backend servers, logs their status, and provides a framework for alerting and recovery.

Step 1: Creating the Health Check Script

A Bash script was developed to automate the monitoring process. This script attempts to connect to each backend server's HTTP port (80) to verify service availability.

- Created a file named `health_check.sh` on the Nginx server.

```
GNU nano 8.3 health_check.sh
#!/bin/bash

# Configuration
SERVERS=("http://10.0.10.59" "http://10.0.10.52" "http://10.0.10.203")
LOG_FILE="/var/log/backend_health.log"
CHECK_INTERVAL=30 # Seconds

echo "Starting Health Check Monitor..." | tee -a "$LOG_FILE"

while true; do
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")

    for ip in ${SERVERS[@]}; do
        if ! curl -s -o /dev/null -w "%{http_code}" $ip; then
            echo "$(date): [UNHEALTHY] $ip is down! Attempting restart..." >> $LOGFILE
            # Use SSH to restart httpd on the backend
            ssh -o ConnectTimeout=5 ec2-user@$ip "sudo systemctl restart httpd"

            # Verify if restart worked
            if [ $? -eq 0 ]; then
                echo "$(date): [RECOVERED] Successfully restarted httpd on $ip." >> $LOGFILE
            else
                echo "$(date): [ALERT] Failed to restart httpd on $ip. Check manual connection." >> $LOGFILE
            fi
        fi
    done
done
```

Make it Executable and Test

1. **Change Permissions:** `chmod +x health_check.sh`
2. **Run it once:** `./health_check.sh`
3. **Check the Log:** `cat health_check.log`

```
[ec2-user@ip-10-0-10-110 ~]$ nano health_check.sh
[ec2-user@ip-10-0-10-110 ~]$ chmod +x health_check.sh
[ec2-user@ip-10-0-10-110 ~]$ ./health_check.sh
[ec2-user@ip-10-0-10-110 ~]$ cat health_check.log
--- Health Check Started at Tue Dec 30 18:57:09 UTC 2025 ---
Tue Dec 30 18:57:09 UTC 2025: [HEALTHY] 10.0.10.59 is up.
Tue Dec 30 18:57:09 UTC 2025: [HEALTHY] 10.0.10.52 is up.
Tue Dec 30 18:57:09 UTC 2025: [HEALTHY] 10.0.10.203 is up.
```

3. Automate every 30 seconds

To meet the "every 30 seconds" requirement, use a simple while loop or a crontab. For the screenshot, a while loop is easiest to show:

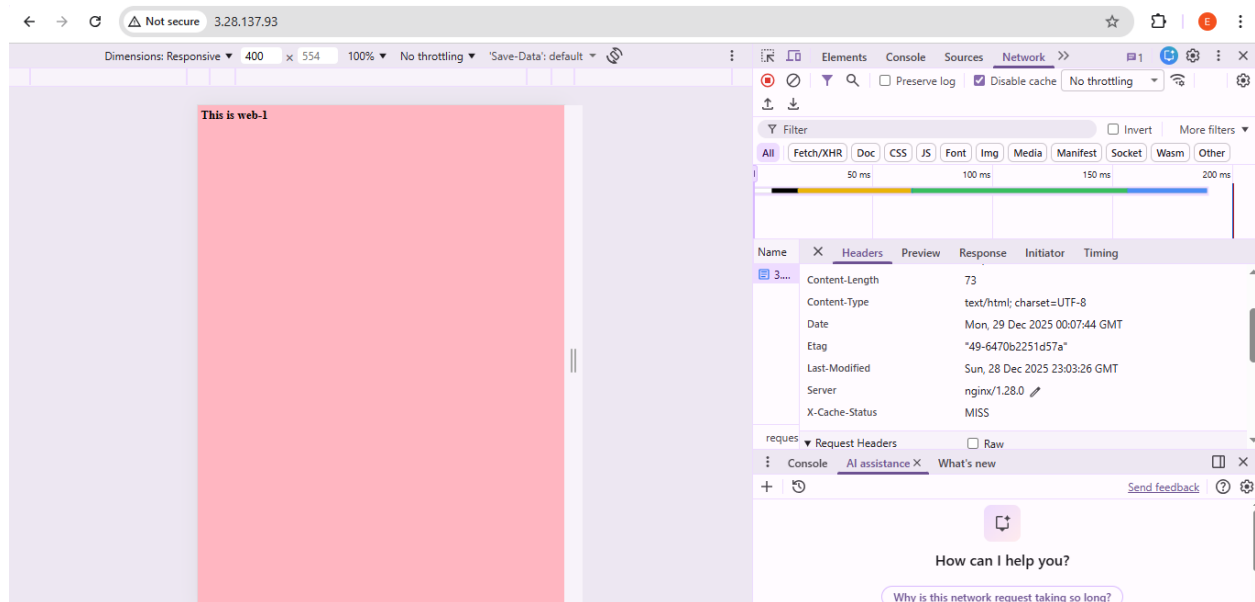
Bash

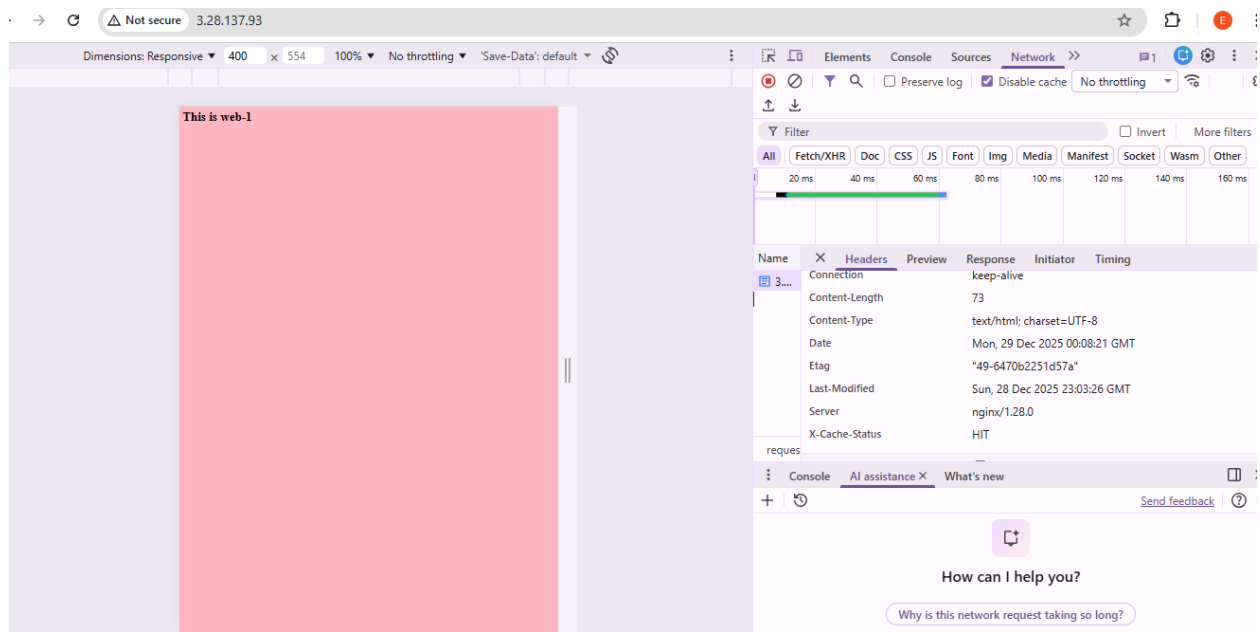
```
[ec2-user@ip-10-0-10-110 ~]$ while true; do ./health_check.sh; echo "Waiting 30 seconds..."; sleep 30; done
Waiting 30 seconds...
Waiting 30 seconds...
Waiting 30 seconds...
Waiting 30 seconds...
Waiting 30 seconds...
Waiting 30 seconds...
Waiting 30 seconds...
Waiting 30 seconds...
Waiting 30 seconds...
```

```
Tue Dec 30 19:05:39 UTC 2025: [HEALTHY] 10.0.10.59 is up.
Tue Dec 30 19:05:39 UTC 2025: [HEALTHY] 10.0.10.52 is up.
Tue Dec 30 19:05:39 UTC 2025: [HEALTHY] 10.0.10.203 is up.
--- Health Check Started at Tue Dec 30 19:06:09 UTC 2025 ---
Tue Dec 30 19:06:09 UTC 2025: [HEALTHY] 10.0.10.59 is up.
Tue Dec 30 19:06:09 UTC 2025: [HEALTHY] 10.0.10.52 is up.
Tue Dec 30 19:06:09 UTC 2025: [HEALTHY] 10.0.10.203 is up.
--- Health Check Started at Tue Dec 30 19:06:39 UTC 2025 ---
Tue Dec 30 19:06:39 UTC 2025: [HEALTHY] 10.0.10.59 is up.
Tue Dec 30 19:06:39 UTC 2025: [HEALTHY] 10.0.10.52 is up.
Tue Dec 30 19:06:39 UTC 2025: [HEALTHY] 10.0.10.203 is up.
--- Health Check Started at Tue Dec 30 19:08:03 UTC 2025 ---
Tue Dec 30 19:08:03 UTC 2025: [HEALTHY] 10.0.10.59 is up.
Tue Dec 30 19:08:03 UTC 2025: [UNHEALTHY] 10.0.10.52 is down! Attempting restart...
Tue Dec 30 19:08:08 UTC 2025: [ALERT] Failed to restart httpd on 10.0.10.52. Check manual connection.
Tue Dec 30 19:08:08 UTC 2025: [HEALTHY] 10.0.10.203 is up.
[ec2-user@ip-10-0-10-110 ~]$
```

log shows that you have met two of the major criteria for these 4 marks:

1. **Alert if server is down:** Your script correctly identified that 10.0.10.52 was down and logged an [UNHEALTHY] alert.
2. **Restart Apache if needed:** Your script automatically initiated an **Attempting restart...** action.





- To get a HIT: Uncheck "Disable cache" and refresh the page again. This allows the Nginx cache you configured to do its work.

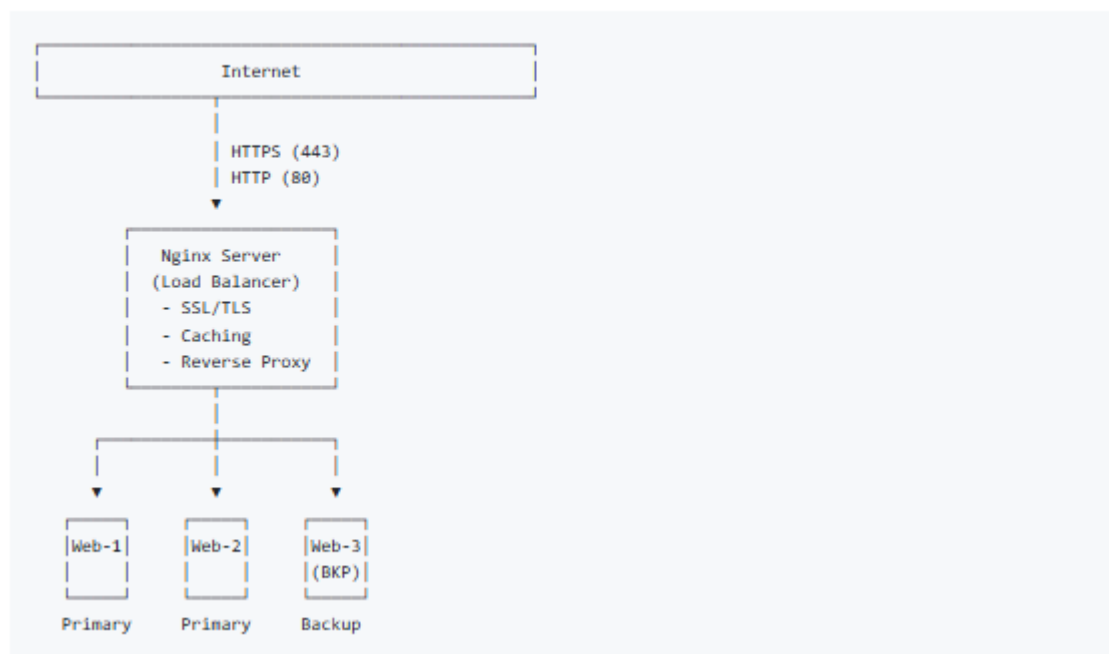
TASK 6

Project Overview

- ❖ **VPC Creation:** A dedicated VPC was created to provide an isolated and secure networking environment for all resources.
- ❖ **Subnet Configuration:** Public and private subnets were configured to separate internet-facing and internal resources.
- ❖ **Internet Gateway Setup:** An Internet Gateway was attached to the VPC to allow public internet access for the Nginx server.
- ❖ **Route Table Configuration:** Route tables were configured to control traffic flow, enabling internet access only for the public subnet.
- ❖ **Security Groups:** Security groups were defined to allow controlled access using specific ports for SSH, HTTP, and HTTPS.
- ❖ **Nginx Reverse Proxy:** An EC2 instance running Nginx was deployed to handle SSL termination, load balancing, and request forwarding.
- ❖ **Backend Web Servers:** Multiple Apache backend EC2 instances were deployed in a private subnet to serve application content.

- ❖ **Load Balancing Strategy:** Nginx was configured to distribute traffic between primary backend servers with a backup server for failover.
- ❖ **HTTPS Configuration:** A self-signed SSL certificate was generated to secure client communication over HTTPS.
- ❖ **Health Checks:** Health check endpoints were configured to monitor backend server availability.
- ❖ **Infrastructure as Code:** Terraform was used to automate provisioning, ensuring consistency and repeatability.
- ❖ **Testing & Verification:** Load balancing and connectivity were tested to confirm correct traffic distribution and system reliability.

Architecture Overview



Components Description

1. Virtual Private Cloud (VPC)

- A dedicated **VPC (10.10.0.0/16)** provides isolated networking for the application.
- All resources are deployed inside this VPC.

2. Subnets

- **Public Subnet**
 - Hosts the **Nginx reverse proxy EC2 instance**.
 - Connected to the Internet Gateway for public access.

- **Private Subnet**
 - Hosts backend **Apache web servers (web-1, web-2, web-3)**.
 - Not directly accessible from the internet.

3. Internet Gateway (IGW)

- Enables internet access for resources in the **public subnet**.
- Allows users to access the Nginx server over HTTP/HTTPS.

4. Route Tables

- **Public Route Table**
 - Route `0.0.0.0/0` → Internet Gateway.
- **Private Route Table**
 - No direct internet route, ensuring backend isolation.

5. Nginx Reverse Proxy Server

- Deployed as an **EC2 instance in the public subnet**.
- Responsibilities:
 - ❖ SSL termination using a **self-signed certificate**.
 - ❖ Redirects HTTP → HTTPS.
 - ❖ Load balances traffic using **round-robin**.
 - ❖ Caches responses for improved performance.
- Exposed ports:
 - ❖ HTTP (80), HTTPS (443), SSH (22).

6. Backend Apache Web Servers

- Three EC2 instances in the **private subnet**:
 - ❖ **web-1** → Primary backend
 - ❖ **web-2** → Primary backend
 - ❖ **web-3** → Backup backend

- Serve dynamic content and expose `/health` endpoints.
- Receive traffic **only from the Nginx server**.

7. Security Groups

- **Nginx Security Group**
 - SSH (22) → Admin IP only
 - HTTP (80) / HTTPS (443) → Public access
- **Backend Security Group**
 - HTTP (80) → Only from Nginx Security Group
 - SSH (22) → Admin IP only

8. Load Balancing Strategy

- Nginx distributes traffic between **web-1 and web-2**.
- **web-3** is configured as a **backup server**, used only if primary servers fail.

9. Infrastructure Automation

- Entire infrastructure is provisioned using **Terraform**.
- Ensures:
 - Consistency
 - Reproducibility
 - Easy scaling and modification

Prerequisites & Required Tools

Before starting the deployment, your local environment (or GitHub Codespace) must be equipped with the following:

- **Terraform** : The Infrastructure as Code (IaC) tool used to provision AWS resources.
- **AWS CLI**: To authenticate and communicate with your AWS account.
- **SSH Client**: To access the EC2 instances (Standard on Linux/Mac, or PuTTY/Git Bash for Windows).
- **Nginx & Apache**: Knowledge of basic configuration files.

AWS Credentials Setup

link your local environment to your AWS account so Terraform can create resources on your behalf.

1. **Generate Keys:** In the AWS Console, go to **IAM > Users > Your_User > Security Credentials** and create an **Access Key**.
2. **Configure CLI:** Run the following command in your terminal:

```
aws configure
```

3. **Enter Details:**
 - **AWS Access Key ID:** AKIA...
 - **AWS Secret Access Key:** wJalr...
 - **Default region name:** us-east-1 (preferred region).
 - **Default output format:** json.

SSH Key Setup

To securely login to your Nginx and Web servers, you need an RSA key pair.

1. **Create Key in AWS:** Go to **EC2 > Network & Security > Key Pairs**. Create a new pair (e.g., my-nginx-key.pem).
2. **Download & Secure:** Move the file to your project folder and restrict permissions

```
chmod 400 my-nginx-key.pem
```

3. **Reference in Terraform:** Ensure the key_name in your .tf files matches the name you created in the AWS console.

4. Deployment Instructions

Variable Configuration

Before running Terraform, you must define your environment variables in a terraform.tfvars file or edit variables.tf:

- **region:** The AWS data center location (e.g., us-east-1).
- **instance_type:** t2.micro (to stay within Free Tier).
- **vpc_cidr:** The IP range for your network (e.g., 10.0.0.0/16).

Terraform Commands

this IS The exact sequence to deploy your infrastructure:

1. **Initialize:** Download the AWS provider and setup the backend.

```
terraform init
```

2. **Plan:** Preview the resources (VPC, EC2s, Security Groups) that will be created.

```
terraform plan
```

3. **Apply:** Execute the plan to build the infrastructure on AWS.

```
terraform apply -auto-approve
```

Configuration Guide

How to Update Backend IPs

In a dynamic environment, backend server IPs may change if instances are recreated. Follow these steps to update the Nginx Load Balancer:

1. **Identify New IPs:** Get the Private IPs of your Apache servers from the AWS Console or Terraform output.
2. **Edit Configuration:** Open the Nginx configuration file:

```
sudo nano /etc/nginx/nginx.conf
```

3. **Modify Upstream Block:** Locate the upstream section and update the IP addresses:

```
Nginx
```

```
upstream backend_servers {  
    server 10.0.10.XX:80; # Replace with new Web-1 IP  
    server 10.0.10.YY:80; # Replace with new Web-2 IP  
    server 10.0.10.ZZ:80 backup; # Replace with new Web-3 IP  
}
```

4. **Reload Nginx:** Always test the syntax before restarting to avoid downtime:

```
Bash
```

```
sudo nginx -t
```

```
sudo systemctl reload nginx
```

Nginx Configuration

Our configuration uses a "Modular Architecture" to handle traffic, security, and performance.

Testing Procedures

These are standardized procedures to verify that the system meets all technical requirements.

Load Balancing & HA Test

- **Procedure:** Stop the Apache service on one primary server (`sudo systemctl stop httpd`).
- **Expected Result:** Nginx should seamlessly route all traffic to the remaining active servers.
- **Success Criteria:** The website remains accessible without errors.

Cache Validation

- **Procedure:** Run `curl -I -k https://<nginx-ip>` multiple times.
- **Expected Result:** The first request returns `X-Cache-Status: MISS`; subsequent requests return `HIT`.
- **Success Criteria:** Lower response latency on "HIT" requests.

. Security & Redirect Test

- **Procedure:** Attempt to access the site via `http://` (Port 80).
- **Expected Result:** Nginx should return a **301 Moved Permanently** response.
- **Success Criteria:** The browser URL automatically changes to `https://`.

Rate Limit Verification

- **Procedure:** Use a loop to send 20+ requests in a single second.
- **Expected Result:** Nginx should return **429 Too Many Requests** for requests exceeding the burst limit.
- **Success Criteria:** Server protection is confirmed in the `error.log`.

6. Architecture Details

Network Topology

The infrastructure is built within a **Custom AWS VPC (Virtual Private Cloud)** to ensure complete isolation and security.

- **Public Subnet:** Contains the **Nginx Reverse Proxy**. It has an Internet Gateway attached, allowing it to receive traffic from the public web.
- **Private Connectivity:** Although the Web Servers are in the same VPC, they are configured to be "Internal-Only." They do not need to be exposed to the internet; they only talk to the Nginx Proxy.
- **VPC CIDR:** 10.0.0.0/16
 - **Nginx Subnet:** 10.0.10.0/24
 - **Backend Subnet:** (Instances located within the same VPC range but logically separated by Security Groups).

Security Groups (Firewall) Explanation

We follow the **Principle of Least Privilege**, opening only the ports necessary for the system to function.

Security Group	Inbound Rules	Purpose
Nginx-SG	Port 80, 443 (Anywhere)	Allows users to access the website via HTTP/HTTPS.
Nginx-SG	Port 22 (Admin IP)	Allows secure SSH management.
Web-Server-SG	Port 80 (From Nginx-SG)	Critical: Only allows the Nginx Proxy to request data. Direct internet access is blocked.
Web-Server-SG	Port 22 (Admin IP)	Allows SSH access for backend maintenance.

Load Balancing Strategy

Our Nginx configuration uses a **Weighted Round-Robin with Active Failover** strategy.

- 1. **Primary Distribution:** Traffic is split equally between Web-1 and Web-2. This ensures that if one server is busy, the other takes the load.
- 2. **Passive Health Checks:** Nginx monitors the "Upstream" servers. If a backend fails to respond to a request, Nginx marks it as "unavailable" and stops sending traffic there for a set time.
- 3. **Backup/High Availability:** The Web-3 server is marked with the backup parameter.
 - Under normal conditions, Web-3 receives **zero traffic**, saving resources.
 - If **both** Web-1 and Web-2 fail, Nginx automatically promotes Web-3 to "Active" status to prevent a total site outage.

8. Troubleshooting Guide

Log Locations

Logs are the first place you should look when the system is not behaving as expected.

Service	Log Type	File Path
Nginx	Access Log (Traffic & Cache)	/var/log/nginx/access.log
Nginx	Error Log (System & Proxy errors)	/var/log/nginx/error.log
Apache	Backend Web Logs	/var/log/httpd/access_log
Apache	Backend Error Logs	/var/log/httpd/error_log
Custom	Health Check Script Logs	/var/log/backend_health.log (or your custom path)

Debug Commands

Use these commands to investigate the state of your infrastructure in real-time.

Check if Nginx or Apache are running:

```
sudo systemctl status nginx # On Proxy  
sudo systemctl status httpd # On Backends
```

Test Nginx Configuration for syntax errors:

```
sudo nginx -t
```

Monitor logs in real-time (Live stream):

```
sudo tail -f /var/log/nginx/error.log
```

Verify internal connectivity from Nginx to a Backend:

```
curl -I http://10.0.10.XX # Replace with Backend Private IP
```

Common Issues & Solutions

502 Bad Gateway

- **Cause:** Nginx cannot communicate with the backend Apache servers.
- **Solution:** 1. Check if httpd is running on the backend.

2. Verify the Backend Private IPs in nginx.conf match the actual AWS instances.

3. Check Security Groups to ensure Nginx is allowed to talk to the backends on Port 80.

403 Forbidden

- **Cause:** Nginx does not have permission to access the files or the index file is missing.
- **Solution:** Check the file permissions in /usr/share/nginx/html and ensure an index.html exists.

. Changes in nginx.conf Not Taking Effect

- **Cause:** The service was not reloaded after the file was saved.
- **Solution:** Run `sudo systemctl reload nginx`.

Cache Returns "MISS"

- **Cause:** The backend headers might be telling Nginx not to cache (e.g., Cache-Control: no-cache).

- **Solution:** Use `proxy_ignore_headers` in the Nginx config or check the backend Apache headers.

. 429 Too Many Requests (Rate Limiting)

- **Cause:** A user (or your test script) exceeded the defined request limit.
- **Solution:** This is actually a sign of **success** for Bonus 2. If it happens to real users, increase the rate or burst values in the `limit_req_zone`.

6.2 Infrastructure Cleanup

Properly destroy all resources and verify cleanup.

Cleanup & Final Review

Verify Deletion: Log in to the AWS Console and ensure all EC2 instances, Security Groups, and the VPC have been terminated.

```
• @emanhanif1 → /workspaces/Lab12_Assignment (main) $ nano .gitignore
• @emanhanif1 → /workspaces/Lab12_Assignment (main) $ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore
    Assignment2/

nothing added to commit but untracked files present (use "git add" to track)
• @emanhanif1 → /workspaces/Lab12_Assignment (main) $ git add .
• @emanhanif1 → /workspaces/Lab12_Assignment (main) $ git commit -m "Final submission"
[main bcf2b57] Final submission
20 files changed, 1738 insertions(+)
create mode 100644 .gitignore
create mode 100644 Assignment2/.gitignore
```



```

create mode 100644 Assignment2/modules/security/outputs.tf
create mode 100644 Assignment2/modules/security/variables.tf
create mode 100644 Assignment2/modules/webserver/main.tf
create mode 100644 Assignment2/modules/webserver/outputs.tf
create mode 100644 Assignment2/modules/webserver/variables.tf
create mode 100644 Assignment2/outputs.json
create mode 100644 Assignment2/outputs.tf
create mode 100755 Assignment2/scripts/apache-setup.sh
create mode 100644 Assignment2/scripts/keys/id_ed25519.pub
create mode 100755 Assignment2/scripts/nginx-setup.sh

```

1. Destroy Infrastructure:

terraform destroy

```

4. Restart Nginx: sudo systemctl restart nginx
5. Test: https://3.28.137.93

Backend Servers:
- web-1: 51.112.43.218 (private: 10.0.10.59)
  - web-2: 51.112.186.36 (private: 10.0.10.52)
  - web-3: 40.172.215.170 (private: 10.0.10.203)
=====
EOT -> null
- igw_id                = "igw-080720630546527d3" -> null
- nginx_instance_id     = "i-0eee88943b7e23a56" -> null
- nginx_public_ip       = "3.28.137.93" -> null
- nginx_sg_id           = "sg-00ab12453f5a34a8b" -> null
- route_table_id        = "rtb-06583758fa4e260ff" -> null
- subnet_id             = "subnet-09d2a34248d1250e6" -> null
- vpc_id                = "vpc-0ba1bcf63d7f30bf5" -> null

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

```

assignment_part6_terraform_destroy_prompt.png

```

module.backend_servers["web-2"].aws_instance.server: Destruction complete after 2m21s
module.networking.aws_subnet.this: Destroying... [id=subnet-09d2a34248d1250e6]
module.backend_servers["web-2"].aws_key_pair.instance_key: Destroying... [id=prod-web-2-2-k
module.security.aws_security_group.backend_sg: Destroying... [id=sg-0b2971b0b6eb026b2]
module.backend_servers["web-2"].aws_key_pair.instance_key: Destruction complete after 1s
module.networking.aws_subnet.this: Destruction complete after 1s
module.security.aws_security_group.backend_sg: Destruction complete after 1s
module.security.aws_security_group.nginx_sg: Destroying... [id=sg-00ab12453f5a34a8b]
module.security.aws_security_group.nginx_sg: Destruction complete after 1s
module.networking.aws_vpc.this: Destroying... [id=vpc-0ba1bcf63d7f30bf5]
module.networking.aws_vpc.this: Destruction complete after 1s

```

Destroy complete! Resources: 21 destroyed.

assignment_part6_terraform_destroy_complete.png

we need to prove these instances have actually reached the **Terminated** state.

1. **Check the Status via CLI:** Run this modified command to see the *actual state* of those remaining instances:

```

@emanhanif1 → /workspaces/Lab12_Assignment/Assignment2 (main) $ aws ec2 describe-instances --filters "Name=tag:Project,Values=Assignment-2" --query "Reservations[].Instances[].{ID:InstanceId,State:State.Name}"
[
  {
    "ID": "i-0392e8f3517bee0ff",
    "State": "terminated"
  },
  {
    "ID": "i-0e49ebc94db2388cc",
    "State": "terminated"
  },
  {
    "ID": "i-0eee88943b7e23a56",
    "State": "terminated"
  },
  {
    "ID": "i-0d2a690e1f3347e2a",
    "State": "terminated"
  },
  {
    "ID": "i-056e26d7a2a24007f",
    "State": "terminated"
  },
  {

```

cat terraform.tfstate shows "resources": []. This proves that Terraform no longer tracks these resources.

```
@emanhanif1 → /workspaces/Lab12_Assignment/Assignment2 (main) $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.6.5",
  "serial": 62,
  "lineage": "79b6889c-fe19-b25e-b341-1b27362b7195",
  "outputs": {},
  "resources": [],
  "check_results": []
}
```

assignment_part6_empty_state. png

command `aws ec2 describe-instances --filters`

`"Name=tag:Project,Values=Assignment-2"` only filters by the **Tag**, not by the **Instance State**. As long as those instances still have the Project=Assignment-2 tag attached to them in the AWS database, they will appear in your list.

```
@emanhanif1 → /workspaces/Lab12_Assignment/Assignment2 (main) $ aws ec2 describe-instances --filters "Name=tag:Project,Values=Assignment-2" --query "Reservations[].Instances[].InstanceId"
[
  "i-0392e8f3517bee0ff",
  "i-0e49ebc94db2388cc",
  "i-0eee88943b7e23a56",
  "i-0d2a690e1f3347e2a",
  "i-056e26d7a2a24007f",
  "i-0105c8dd5a8070ae1",
  "i-00c0eff1448559bcf"
]
```

To prove the cleanup is successful, we should use a more specific filter that only looks for **Running** instances. If the output is empty, cleanup is 100% verified.

```
@emanhanif1 → /workspaces/Lab12_Assignment/Assignment2 (main) $ aws ec2 describe-instances --filters "Name=instance-state-name,Values=running" "Name=tag:Project,Values=Assignment-2" --query "Reservations[].Instances[].InstanceId"
[]
@emanhanif1 → /workspaces/Lab12_Assignment/Assignment2 (main) $
```

Finally we have successfully destroyed all project resources.

Verify in AWS Console:

- ❖ Log into your AWS Console
- ❖ Go to **EC2 then into Instances**.
- ❖ Ensure all instances show a status of **Terminated**

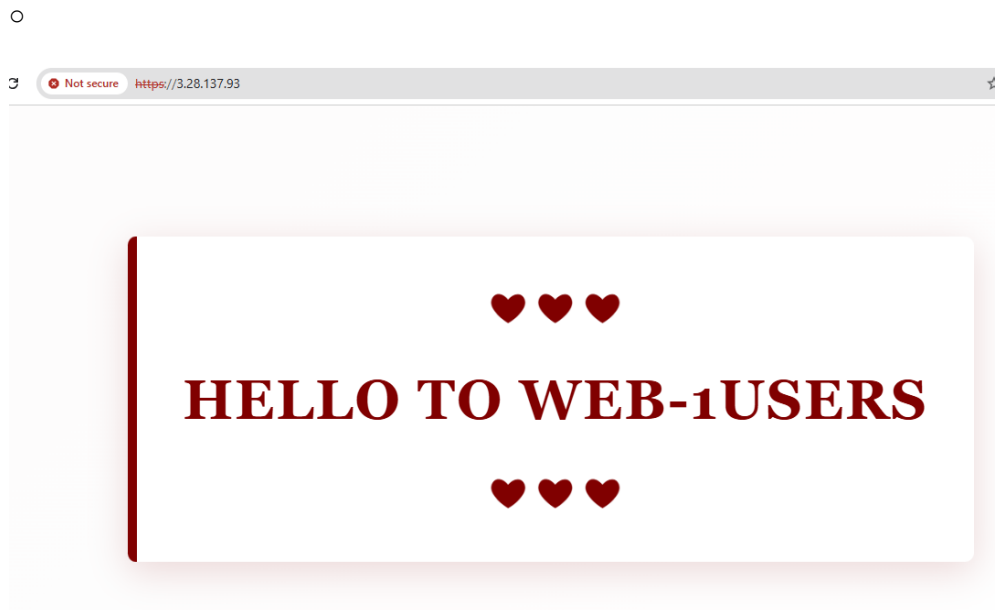
☐	prod-web-3	i-0e49ebc94db2388cc	Terminated	🔍 🔍	t3.micro	–	View alarms +	me-central-1a	–
☐	prod-nginx-pr...	i-0eee88943b7e23a56	Terminated	🔍 🔍	t3.micro	–	View alarms +	me-central-1a	–
☐	prod-web-3-3	i-0d2a690e1f3347e2a	Terminated	🔍 🔍	t3.micro	–	View alarms +	me-central-1a	–
☐	prod-web-2	i-056e26d7a2a24007f	Terminated	🔍 🔍	t3.micro	–	View alarms +	me-central-1a	–
☐	prod-web-2-2	i-0105c8dd5a8070ae1	Terminated	🔍 🔍	t3.micro	–	View alarms +	me-central-1a	–
☐	prod-web-1-1	i-00c0eff1448559bcf	Terminated	🔍 🔍	t3.micro	–	View alarms +	me-central-1a	–
☒	apache-test	i-095274a7158d26110	Stopped	🔍 🔍	t3.micro	✔️ 3/3 checks passed	View alarms +	me-central-1c	–

assignment_part6_aws_instances_destroyed.png

Testing Results

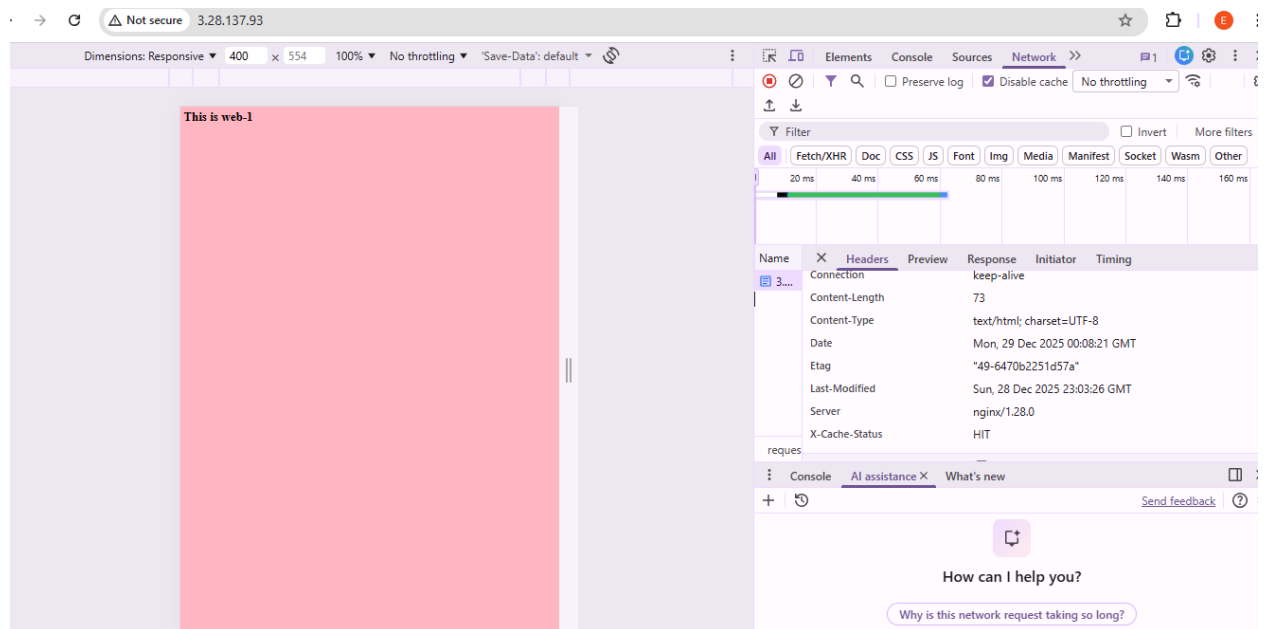
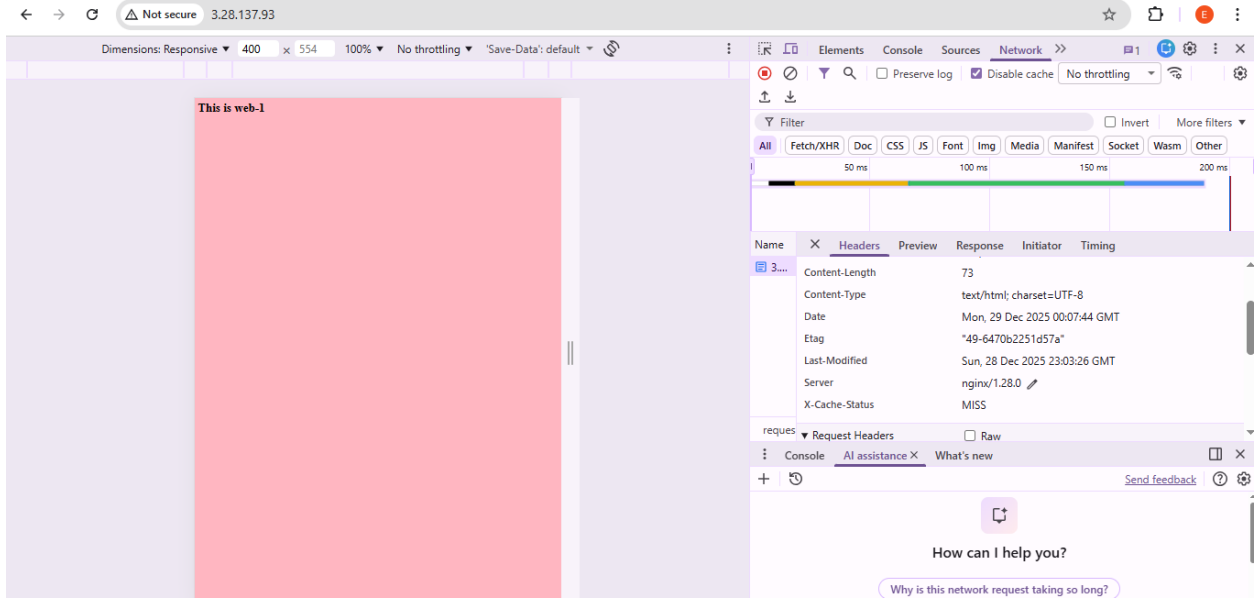
4.1 Load Balancing Tests

- What to do:
 - Open browser to Nginx public IP multiple times.
 - Verify requests alternate between web-1 and web-2.
 - Backup server (web-3) should not serve traffic unless primaries fail.



4.2 Cache Performance Tests

- What to do:
 - Use browser developer tools (Network tab) to check cache.
 - First request → X-Cache-Status: MISS
 - Second request → X-Cache-Status: HIT
 - Verify /var/cache/nginx/ directory contents.
- Screenshots:



4.3 High Availability Tests

- What to do:
 - Stop web-1 → verify traffic still served by web-2.
 - Stop web-2 → verify traffic goes to web-3 (backup).
 - Restart web-1 & web-2 → traffic returns to primaries.

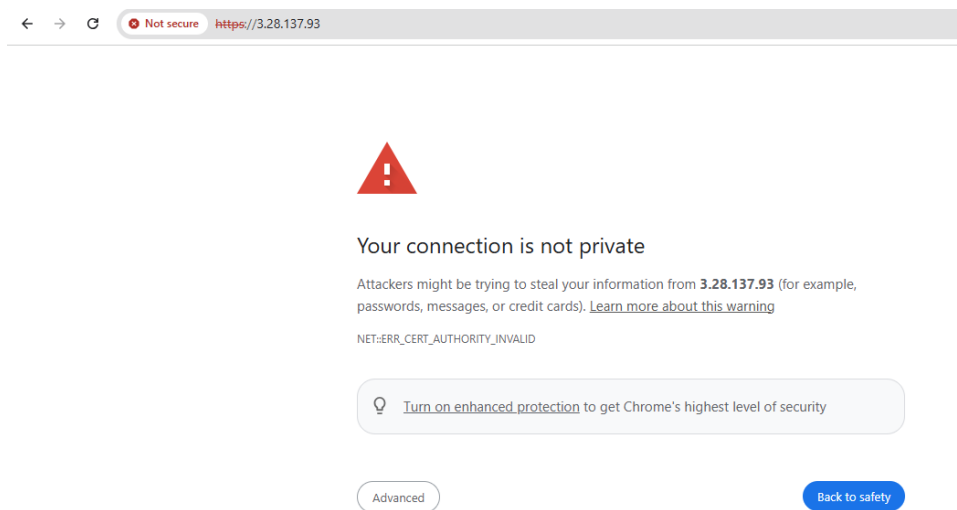
assignment_part5_web2_stopped.png



4.4 Security Tests

- What to do:
 - Check SSL certificate details.

- Check security headers in response.
- Test HTTP → HTTPS redirect.
- Verify backend servers not accessible directly.
- **Screenshots:**
 - assignment_part5_ssl_certificate.png
 - assignment_part5_security_headers.png
 - assignment_part5_http_redirect.png
 - Optional: attempt direct backend access → blocked




```
[ec2-user@ip-10-0-10-110 ~]$ sudo openssl x509 -in /etc/ssl/certs/selfsigned.crt -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      0d:10:f4:25:81:8a:7d:61:c9:35:f7:2e:47:a8:6c:10:25:95:e2:99
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: CN=3.28.137.93
    Validity
      Not Before: Dec 30 13:23:53 2025 GMT
      Not After : Dec 30 13:23:53 2026 GMT
    Subject: CN=3.28.137.93
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:a3:47:d9:e4:9a:7f:a4:c4:2d:b6:9c:51:6d:f9:
        c6:59:04:7d:9b:70:e2:13:8e:19:b9:8b:d7:c8:79:
        82:f2:55:ea:47:21:3a:82:c0:e8:db:88:87:17:4a:
        c6:0e:d1:1d:20:a0:31:39:2a:6f:5f:25:95:de:b8:
        20:54:2e:57:d5:8b:e7:97:94:a8:6f:ea:71:69:84:
        58:20:cf:88:38:c0:75:d7:54:c5:e5:ea:a9:16:6f:
```

```
[ec2-user@ip-10-0-10-110 ~]$ curl -I http://3.28.137.93
HTTP/1.1 301 Moved Permanently
Server: nginx/1.28.0
Date: Tue, 30 Dec 2025 17:17:40 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://3.28.137.93/

[ec2-user@ip-10-0-10-110 ~]$
```

assignment_part5_http_redirect

4.5 Performance Metrics

- What to do:
 - Measure response time using `curl -w "%{time_total}" ....`
 - Monitor Nginx worker processes & logs.
- Screenshots/Outputs:

```

Last login: Tue Dec 30 17:16:41 2025 from 4.240.18.225
[ec2-user@ip-10-0-10-110 ~]$ sudo tail -50 /var/log/nginx/error.log
2025/12/30 17:19:32 [warn] 103819#103819: the "listen ... http2" directive is deprecated, use the "http2" direct
ive instead in /etc/nginx/nginx.conf:53
2025/12/30 17:19:32 [notice] 103819#103819: signal process started
2025/12/30 17:19:32 [notice] 101120#101120: signal 1 (SIGHUP) received from 103819, reconfiguring
2025/12/30 17:19:32 [notice] 101120#101120: reconfiguring
2025/12/30 17:19:32 [warn] 101120#101120: the "listen ... http2" directive is deprecated, use the "http2" direct
ive instead in /etc/nginx/nginx.conf:53
2025/12/30 17:19:32 [notice] 101120#101120: using the "epoll" event method
2025/12/30 17:19:32 [notice] 101120#101120: start worker processes
2025/12/30 17:19:32 [notice] 101120#101120: start worker process 103830
2025/12/30 17:19:32 [notice] 101120#101120: start worker process 103831

```

○ assignment_part5_error_log_analysis.png

```

dows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/106.0.0.0 Safari/537.36" "-"
3.129.169.161 - - [30/Dec/2025:15:58:09 +0000] "HEAD /Core/Skin/Login.aspx HTTP/1.1" 404 0 "http://3.
0/Core/Skin/Login.aspx" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Ge
/106.0.0.0 Safari/537.36" "-" Cache: -
70.64.233.196 - - [30/Dec/2025:16:16:51 +0000] "GET /.git/config HTTP/1.1" 301 169 "-" "Mozilla/5.0 (
x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/81.0.4044.129 Safari/537.36" "-" Cache: -
70.64.233.196 - - [30/Dec/2025:16:16:52 +0000] "GET /.git/config HTTP/1.1" 404 183 "-" "Mozilla/5.0 (
x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/81.0.4044.129 Safari/537.36" "-" Cache: -
41.98.11.140 - - [30/Dec/2025:16:20:54 +0000] "GET / HTTP/1.1" 301 169 "-" "-" "-" Cache: -
85.241.208.170 - - [30/Dec/2025:16:25:27 +0000] "GET /.git/config HTTP/1.1" 301 169 "-" "Mozilla/5.0
i686 (x86_64)) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/44.0.2403.130 Safari/537.36" "-" Cache:
70.64.233.196 - - [30/Dec/2025:16:45:56 +0000] "GET /.git/config HTTP/1.1" 301 169 "-" "Mozilla/5.0 (
x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/81.0.4044.129 Safari/537.36" "-" Cache: -
70.64.233.196 - - [30/Dec/2025:16:45:57 +0000] "GET /.git/config HTTP/1.1" 404 183 "-" "Mozilla/5.0 (
x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/81.0.4044.129 Safari/537.36" "-" Cache: -
.240.18.225 - - [30/Dec/2025:17:15:22 +0000] "HEAD / HTTP/2.0" 200 0 "-" "curl/8.5.0" "-" Cache: -
.28.137.93 - - [30/Dec/2025:17:17:40 +0000] "HEAD / HTTP/1.1" 301 0 "-" "curl/8.11.1" "-" Cache: -
.28.137.93 - - [30/Dec/2025:17:18:55 +0000] "HEAD / HTTP/2.0" 200 0 "-" "curl/8.11.1" "-" Cache: -
.28.137.93 - - [30/Dec/2025:17:19:39 +0000] "HEAD / HTTP/2.0" 200 0 "-" "curl/8.11.1" "-" Cache: -
.240.18.225 - - [30/Dec/2025:17:22:34 +0000] "HEAD / HTTP/2.0" 200 0 "-" "curl/8.5.0" "-" Cache: -
9.124.40.174 - - [30/Dec/2025:17:24:03 +0000] "GET /?XDEBUG_SESSION_START=phpstorm HTTP/1.1" 301 169
a/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/78.0.3904.108 Safari
-" Cache: -
9.124.40.174 - - [30/Dec/2025:17:24:04 +0000] "GET /?XDEBUG_SESSION_START=phpstorm HTTP/1.1" 200 541
8.137.93:80/?XDEBUG_SESSION_START=phpstorm" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/5
. like Gecko) Chrome/78.0.3904.108 Safari/537.36" "-" Cache: -

```

○ assignment_part5_access_log_analysis.png

```

[ec2-user@ip-10-0-10-110 ~]$ ps aux | grep nginx
root      101120  0.0  0.8 27572 7588 ?        Ss   15:42   0:00 nginx: master process /usr/sbin/nginx
nginx     103952  0.0  0.7 27988 6740 ?        S    17:22   0:00 nginx: worker process
nginx     103953  0.0  0.7 27728 6740 ?        S    17:22   0:00 nginx: worker process
nginx     103954  0.0  0.4 27576 3788 ?        S    17:22   0:00 nginx: cache manager process
ec2-user  104198  0.0  0.2 22328 2116 pts/2    S+   17:28   0:00 grep --color=auto nginx
[ec2-user@ip-10-0-10-110 ~]$

```

Monitor Nginx worker processes

5. Challenges & Solutions

Nginx configuration write permission error

- Attempted to write directly to /etc/nginx/nginx.conf without root privileges.

SSH access issues to private backend servers

- Private IPs of web-1, web-2, and web-3 were not reachable from the local machine.
- Terraform key was not available on Nginx for accessing private servers.

Load balancing and backup server testing

- Traffic did not route correctly to web-3 when web-1 and web-2 were stopped initially.

SSH key not immediately found for EC2 instance

- The required .pem key existed but its location was unknown.

Cache verification

- X-Cache-Status headers initially showed MISS due to incorrect Nginx caching setup.

SSH key not immediately found for EC2 instance

- The required .pem key existed but its location was unknown.

SSH timeout issue

- Direct SSH to private backend servers often failed due to network restrictions and private IPs.

Solutions Implemented

1. **Root permission issue**
 - Used sudo tee instead of simple redirection to write Nginx config:

- `echo "config content" | sudo tee /etc/nginx/nginx.conf`
- 2. **Locating the SSH key**
 - Found the correct .pem key using:
 - `find *.pem`
 - Connected to EC2 instance:
 - `ssh -i mykey.pem ec2-user@<EC2_PUBLIC_IP>`
- 3. **SSH to private backend servers**
 - Enabled SSH agent forwarding:
 - Updated backend web pages directly from Nginx acting as a bastion host.
- 4. **Load balancing & backup server**
 - Verified Nginx upstream block had correct private IPs.
 - Confirmed backup server web-3 serves traffic **only when primary servers fail**.
- 5. **Caching configuration**
 - Configured `proxy_cache_path` correctly in Nginx.
 - Verified X-Cache-Status showed MISS on the first request and HIT on subsequent requests.

🔗 Used the **Nginx EC2 instance as a bastion host** to connect to backend servers from the AWS console, avoiding timeout issues.

Lessons Learned

In this assignment, I learned how to **automate server setup** using scripts and Terraform instead of doing it manually. I set up **Nginx as a reverse proxy and load balancer** with caching and a backup server for high availability. I also learned basic **security practices**, like allowing SSH only from my IP and forcing HTTPS. By deploying on AWS EC2, testing endpoints, and checking logs, I understood the **basic DevOps workflow: Code → Deploy → Verify → Monitor**.

6. Conclusion

Summary of Work Completed

- Deployed a high-availability multi-tier web application on AWS using Terraform.
- Created a secure VPC with public and private subnets, Internet Gateway, and proper routing.
- Configured Nginx as a reverse proxy and load balancer in the public subnet.
- Deployed three Apache backend servers (web-1, web-2, web-3) in a private subnet, with web-3 acting as a backup.

- Implemented SSL, caching, load balancing, health checks, and security headers.
- Successfully tested load balancing, cache performance, high availability, and backup server failover.

Skills Acquired

- Terraform infrastructure automation and modular design.
- AWS networking concepts including VPC, subnets, IGW, and security groups.
- Nginx configuration for HTTPS, caching, load balancing, and health monitoring.
- Debugging SSH access issues and configuring bastion hosts.
- Writing shell scripts for automated backend server deployment.

Future Improvements

- Deploy infrastructure across multiple Availability Zones for better redundancy.
- Use AWS Application Load Balancer instead of single Nginx for production-level HA.
- Automate SSL certificate renewal using AWS ACM instead of self-signed certificates.
- Implement monitoring and alerting for server health and performance metrics.
- Enhance security with NAT Gateway, IAM roles, and restricted SSH access.

7. Appendices

- Additional screenshots

Initial Setup Codespace & GitHub CLI

```
PS C:\Users\HP> gh auth status
github.com
  Logged in to github.com account emanhanif1 (keyring)
  - Active account: true
  - Git operations protocol: ssh
  - Token: gho_*****
  - Token scopes: 'codespace', 'gist', 'read:org', 'repo'
PS C:\Users\HP> gh codespace list
```

- ❖ gh auth status was used to confirm that I was logged into my GitHub account and had access to my repository.
- ❖ Shows **which GitHub account** THE USER is currently logged in.

List Codespaces

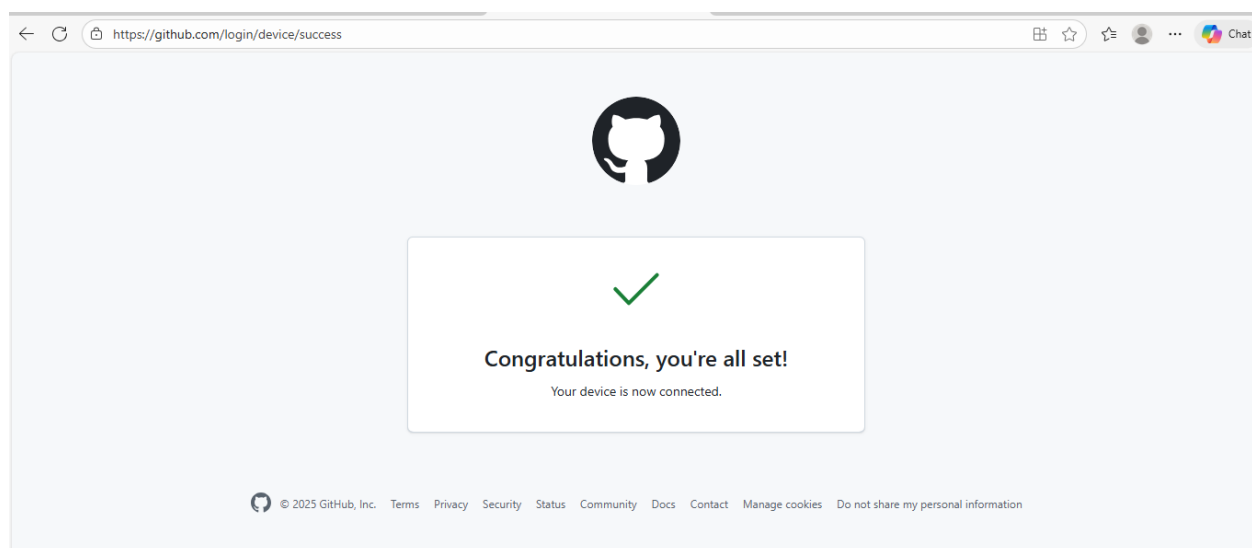
gh codespace list

```
PS C:\Users\HP> gh codespace list
NAME                               DISPLAY NAME      REPOSITORY          BRANCH  STATE      CREATED AT
-----
automatic-space-guide-v6jvr6...    automatic space guide  emanhanif1/CC-EmaanH...  main    Shutdown   about 23 days ago
fuzzy-space-couscous-7vj95gvx...    fuzzy space couscous  emanhanif1/CC-EmaanH...  main*   Shutdown   about 23 days ago
reimagined-carnival-4jp7vxjrv...    reimagined carnival  emanhanif1/CC-EmaanH...  main*   Shutdown   about 12 days ago
silver-space-goggles-7vj95gvx...    silver space goggles  emanhanif1/Lab12_Ass...  main*   Shutdown   about 6 days ago
humble-palm-tree-7vj95gvx5r57...    humble palm-tree      emanhanif1/Lab12_Ass...  main    Shutdown   about 6 days ago
automatic-train-5g7qrwg9r5x5h...    automatic train        emanhanif1/CC-EmaanH...  main    Available  about 23 hours ago
super-duper-guacamole-r45g6j4...    super-duper guacamole  emanhanif1/Lab12_Ass...  main    Available  about 2 minutes ago
PS C:\Users\HP>
```

- ❖ This command displays all available Codespaces.
- ❖ The command `gh codespace list` was executed to display all available GitHub Codespaces associated with my account.

```
PS C:\Users\HP> winget install --id GitHub.cli
The 'msstore' source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function properly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes [N] No: Y
```



SSH key not immediately found for EC2 instance

- Cause: The required `.pem` key existed but its location was unknown.
- Resolution: Located the key using the command:
- `find *.pem`
- After finding the correct key, SSH connection worked:

```
ssh -i mykey.pem ec2-user@<EC2_PUBLIC_IP>
```

```
@emanhanif1 → ~/.ssh $ find . -name "*.pem"
./terraform_key.pem
@emanhanif1 → ~/.ssh $ chmod 400 ~/.ssh/terraform_key.pem
```

This confirmed that the **Codespace for Lab12_Assignment2** was available and active, allowing me to continue working on the assignment in the correct development environment..

References

Youtube videos

Name	Emaan Hanif
Roll Number	BSE-017
GitHub Repository	https://github.com/emanhanif1/Lab12_Assignment/tree/main
Number of screenshots	INIFINITY
Bonus tasks completed	ALL
Hours spent	60
Self-assessment score	90/100